

算法 Algorithm

最小生成树

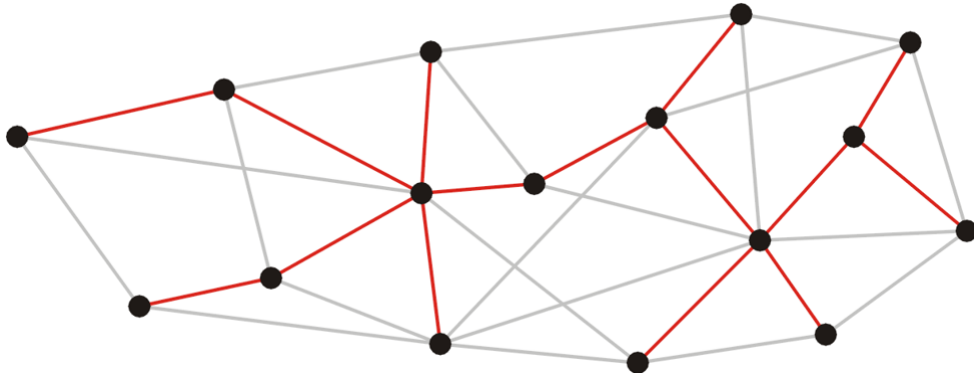
Spanning trees

什么是spanning trees最小生成树？首先，生成树的定义如下：

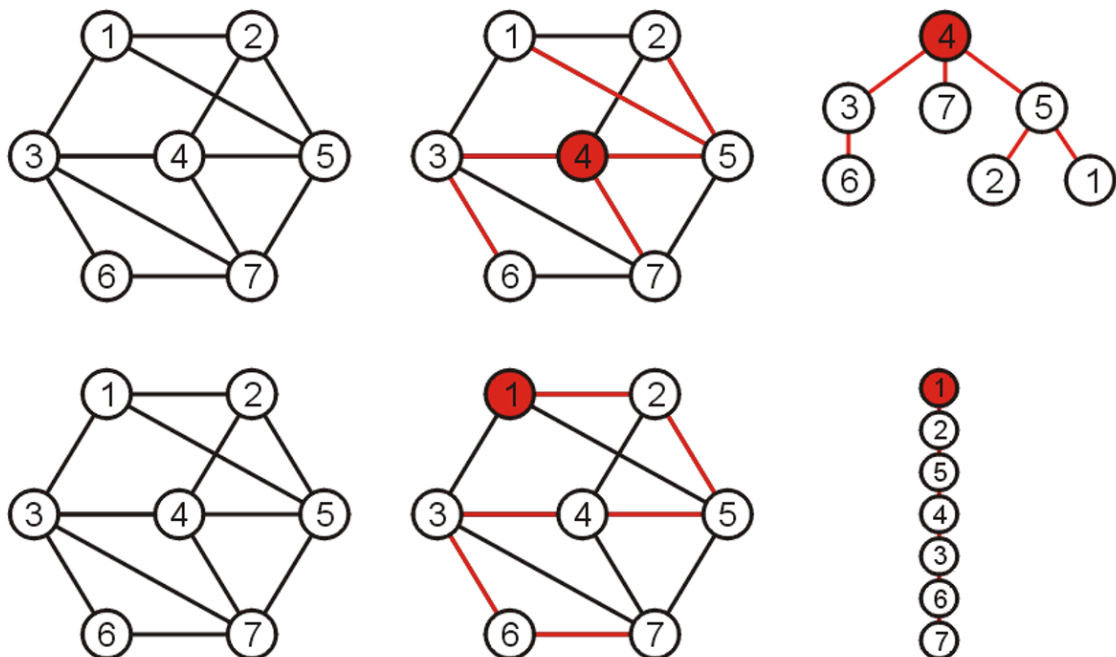
Given a connected graph with n vertices, a spanning tree is defined as a subgraph that is a tree and includes all the n vertices

生成树有 $n-1$ 个边，并且一个图的生成树并不是唯一的

由上述定义可知：生成树是原来图中的一个子图，并且包含了所有的顶点，而且边的数量正好是顶点数量-1。下面是一个生成树的例子：原来的图中共有16个顶点，然后红线连接成的生成树里面有15个顶点：



下图又展示了：一个图的生成树并不唯一！可能有多种生成树，都满足定义的要求；同时也展示了为什么这个子图的称呼可以是“树”：



同时，生成树的定义支持权重的概念（weight）：一个生成树的权重是所有生成树上的边长的权重总和。因此引入了权重的概念之后，我们发现：生成树可以有很多种，但是其中最特殊的便是权重最小的生成树。因此，我们引申出了最小生成树的概念：拥有着最小权重的生成树。

另外，生成树的概念存在于连通图中，那么如果不是连通图，而是一个个连通分量组成的呢？可想而知，每一个单独的连通分量都能生成自己的生成树，那么一个个生成树在一起就是spanning forest:

Suppose that a graph is composed of N connected vertex-induced sub-graphs. In this case, we may define a spanning forest as a collection of N spanning trees, one for each connected vertex-induced sub-graph.

A minimum spanning forest is therefore a collection of N minimum spanning trees, one for each connected vertex-induced sub-graph

那么在unweighted graph中也有最小生成树的概念，只不过是在无权重的图中，我们认为每一条边长的权重都是1。那么在unweighted graph里面，因此可想而知：所有的生成树其实都是最小生成树，而最小生成树的权重都是 $|V| - 1$ 。

Theorem : 在weighted graph中，如果每一条边长的权重都是不一样的，那么这能够保证这个weighted connected图一定有一个唯一的最小生成树。

那么接下来将介绍两种寻找最小生成树的算法。在不特别说明的情况下，默认有权连通图的每一条边长权重都是不一样的，从而确保最终只有一个最小生成树。

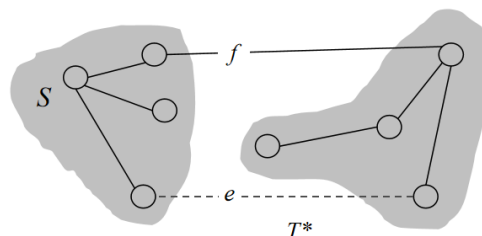
Prim's Algorithm

Strategy

思想：假如说我们对于这个图的一部分顶点 $k < n$ ，已经知道了一个最小生成树，那么我们如何让这个 k 个顶点的最小生成树进行扩张呢？我们的策略是：对于每一个可能的顶点 V_{k+1} 来说，我们可以遍历这个顶点到 k 个点的所有可能存在的边长的权重，然后找到权重最小的边，在这些对一个可能新顶点的最小权重边中再找到拥有最小权重值的顶点，然后进行连接。当然，这个流程也可以换个方式：对于最小生成树里面的所有的顶点，列出所有的和非最小生成树顶点连接的边的权重，然后找到最小值，并且连接这两个顶点。这样我们就有了这 $k+1$ 个顶点的最小生成树了。这个扩张思路有一个专门的属于叫做cut property，来说明这种扩张策略的合理性：

Cut property

- Let S be any subset of nodes, and let e be the least weight edge with exactly one endpoint in S . Then the MST T^* contains e .
- Proof
 - Suppose e does not belong to T^* .
 - Adding e to T^* creates a cycle C in T^* .
 - e is in a cycle C with exactly one endpoint in $S \Rightarrow$ there exists another edge f in C with exactly one endpoint in S .
 - $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
 - Since $w_e < w_f$, the weight of T' is smaller than that of T^* .
 - This is a contradiction



直觉上来说，如果这种性质是错的话，那么这个顶点只能通过其他的方式和这 k 个顶点的最小生成树进行连接，那么对于另外一条线路来说，一定有一个另外的顶点和这个最小的生成树进行连接，而这条边长的权重就至少比那个 V_{k+1} 的对应的边长的权重高，这就是矛盾点

那么在之前的流程中都是默认知道了 k 个顶点的最小生成树，那么这个 k 从1遍历到 n 不就找到了最小生成树吗？对于 $k=1$ 的情况，认为一个顶点的最小生成树就是一个顶点本身。因此我们得到了**Prim's algorithm**：

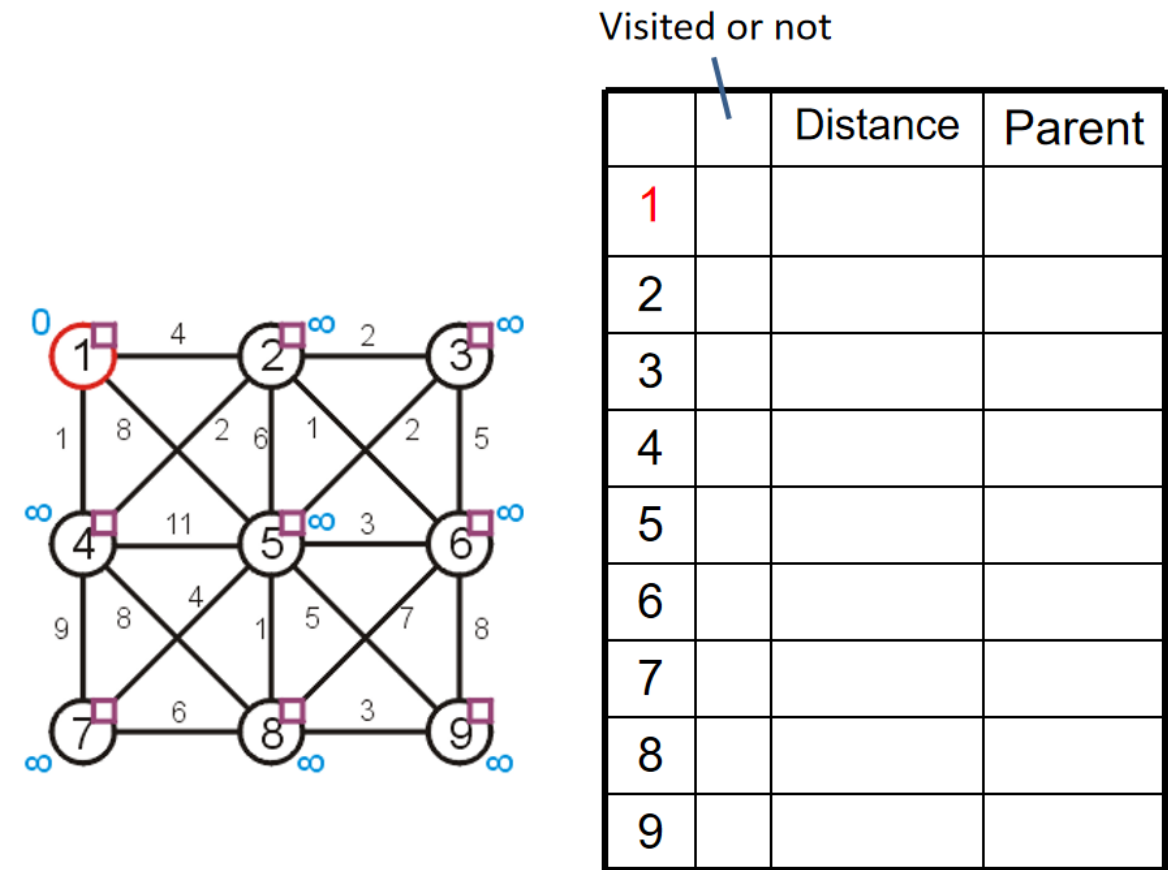
Prim's algorithm for finding the minimum spanning tree states:

- Start with an arbitrary vertex to form a minimum spanning tree on one vertex
- At each step, add the edge with least weight that connects the current minimum spanning tree to a new vertex
- Continue until we have $n - 1$ edges and n vertices

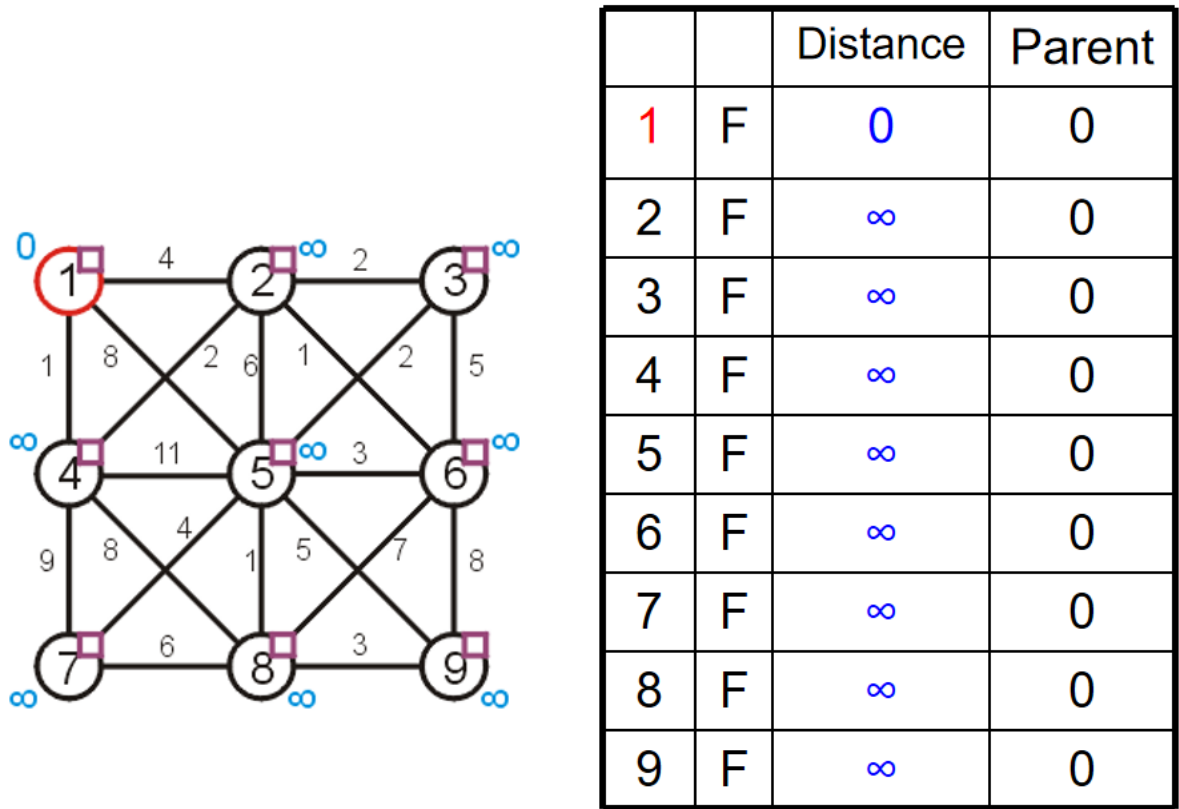
有更多的细节值得探讨：

- 对于每一个顶点来说，在结构体里面可以设置一个Boolean Flag，来记录这个订单是否已经被访问，i.e., 是否已经在 k 个顶点的最小生成树里面了
- 记录每一个顶点到最小生成树任意一个节点的最小的distance
- 同时要记录父节点的指针

下面将通过例子来生动地演示prim algorithm并且将展示更多的实现细节：



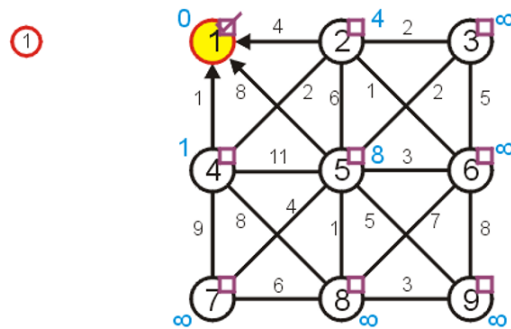
关于空初始化，随便选择一个顶点，然后标记这个顶点的距离为0（合理，因为自己到自己的距离当然是0），同时其他的所有顶点到这个顶点的距离都是无穷（合理，因为其他的顶点都还没有编入到最小生成树里面），设置所有的顶点都是unvisited状态1，然后所有的顶点的父节点指针都置为空指针。当所有的距离小于正无穷的顶点都是visited的时候，算法结束。所以，第一步初始化的结果如下：



那么第一步，遍历最小生成树里面所有顶点到非最小生成树顶点的距离，找到最小值和其对应的另外一个顶点：这里是4，权重最小是1，因此1 4进行连接。同时，对于2 5这两个顶点来说，我们依然记录这两个顶点到1的距离，并且将父节点设置为1节点的指针。这是算法上的习惯罢了，不妨碍我们流程的核心：记录最小生成树里面的顶点更新：1和4. 因此在这一步结束之后，我们更新了2 4 5节点的信息，但是还没更新最小生成树，只是把根节点置入——4的置入在下一步中进行。

每一步访问一个节点，然后在下一步这个节点放进最小生成树，同时访问一个新的满足要求的节点；只有放进最小生成树的时候，Boolean Flag才置为True

Visiting vertex 1, we update vertices 2, 4, and 5

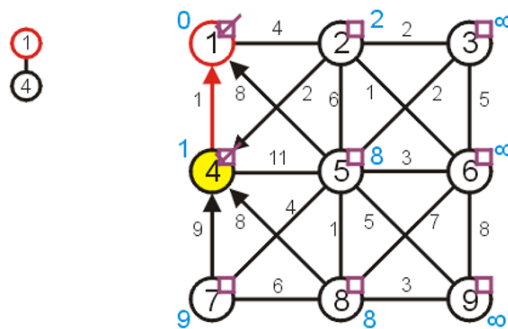


		Distance	Parent
1	T	0	0
2	F	4	1
3	F	∞	0
4	F	1	1
5	F	8	1
6	F	∞	0
7	F	∞	0
8	F	∞	0
9	F	∞	0

第二步，遍历1 4节点和其他非最小生成树里面的节点的边长的权重，然后更新2 5 7 8四个节点的数据。当然，从数据上来看，5节点的信息不更新。更新完成之后，把4这个节点放进最小生成树里面。同时根据剩下的2 5 7 8四个节点的distance，我们知道下一步将访问2号节点。

The next unvisited vertex with minimum distance is vertex 4

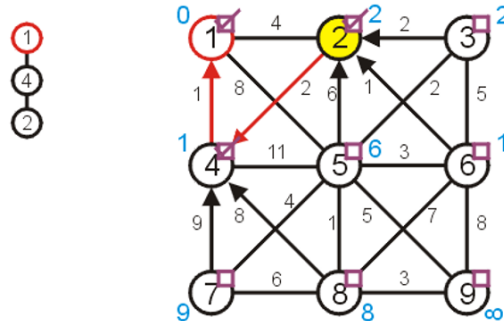
- Update vertices 2, 7, 8
- Don't update vertex 5



		Distance	Parent
1	T	0	0
2	F	2	4
3	F	∞	0
4	T	1	1
5	F	8	1
6	F	∞	0
7	F	9	4
8	F	8	4
9	F	∞	0

第三步，访问2号节点，然后更新3 5 6节点的信息，当然从数据的角度上来看，3 5 6节点的数据进行了更新。更新之后，把2号节点放进最小生成树里面。并且我们知道即将访问6号节点。

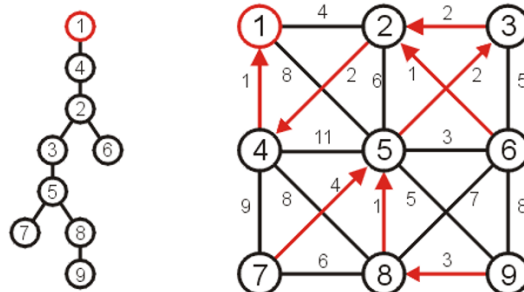
Next visit vertex 2
 – Update 3, 5, and 6



		Distance	Parent
1	T	0	0
2	T	2	4
3	F	2	2
4	T	1	1
5	F	6	2
6	F	1	2
7	F	9	4
8	F	8	4
9	F	∞	0

不断重复操作，最后我们将会得到如下的结果：

Using the parent pointers, we can now construct the minimum spanning tree



		Distance	Parent
1	T	0	0
2	T	2	4
3	T	2	2
4	T	1	1
5	T	2	3
6	T	1	2
7	T	4	5
8	T	1	5
9	T	3	8

总而言之：

- we begin with a vertex which represents the root
- starting with this trivial tree and iteration, we find the shortest edge which we can add to this already existing tree to expand it

Implementation and Analysis

The initialization requires $\Theta(|V|)$ memory and run time

We iterate $|V| - 1$ times, each time finding the *closest* vertex

- Iterating through the table requires $\Theta(|V|)$ time
- Each time we find a vertex, we must check all of its neighbors

With an adjacency list, the run time is $\Theta(|V|^2 + |E|) = \Theta(|V|^2)$ as $|E| = O(|V|^2)$

Can we do better?

- At each iteration, we need to find the shortest edge
- How about a priority queue?
 - Assume we are using a binary heap

We iterate $|V| - 1$ times, each time finding the *closest* vertex

- The size of the priority queue is $O(|V|)$
- Pop the closest vertex from the priority queue is $O(\ln(|V|))$
- For each of its neighbors, we may update the distance, which is $O(\ln(|V|))$ (“decrease key” can be done with $O(\ln(|V|))$ in a heap).

With an adjacency list, the total run time is $O(|V| \ln(|V|) + |E| \ln(|V|)) = O(|E| \ln(|V|))$

Thus, we have two run times when using

- A binary heap: $O(|E| \ln(|V|))$
- A Fibonacci heap: $O(|E| + |V| \ln(|V|))$

Questions: Which is faster if $|E| = \Theta(|V|)$? How about if $|E| = \Theta(|V|^2)$?

Summary

We have seen an algorithm for finding minimum spanning trees

- Start with a trivial minimum spanning tree and grow it
- An alternate algorithm, Kruskal’s algorithm, uses a different approach

Prim’s algorithm finds an edge with least weight which grows an already existing tree

- It solves the problem in $O(|E| \ln(|V|))$ time

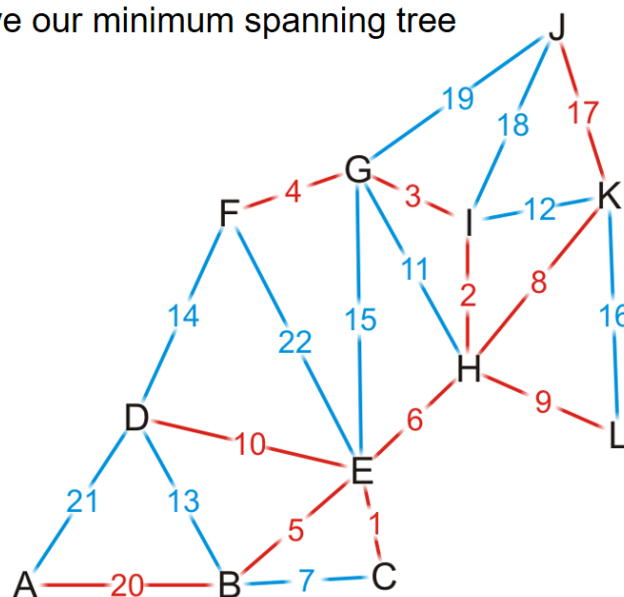
Kruskal's Algorithm

核心思想是：将所有的边按照权重进行排序，然后按照权重最小到最大的顺序遍历所有的边。对于遍历的边来说，**如果添加之后子图没有成环**，那么就把这条边加进最小生成树里面。不断这样加边，直到有 $|V| - 1$ 条边被加入，这样我们就有了一个最小生成树。例子如下：

Example

Having added {A, B}, we now have 11 edges

- We terminate the loop
- We have our minimum spanning tree



{C, E}
{H, I}
{G, I}
{F, G}
{B, E}
{E, H}
{B, C}
{H, K}
{H, L}
{D, E}
{G, H}
{I, K}
{B, D}
{D, F}
{E, G}
{K, L}
{J, K}
{J, I}
{J, G}
{A, D}
{E, F}

在具体的实践中，我们用数组储存边长和它们对应的权重，然后通过任一排序算法进行sorting。但是最关键的在于：如何判断一条边的加入会导致环的出现？我们可以检查这条边的两个端点在先前的生成树中是否已经是连通的。那么这样的话，分析如下：

Implementation

- We would store the edges and their weights in an array
- We would sort the edges using some sorting algorithm: $O(|E| \ln(|E|))$
- For each edge, add it if no cycle is created.
 - How do we determine if a cycle would be created?
 - Check if the two vertices of the edge are already connected by the added edges.

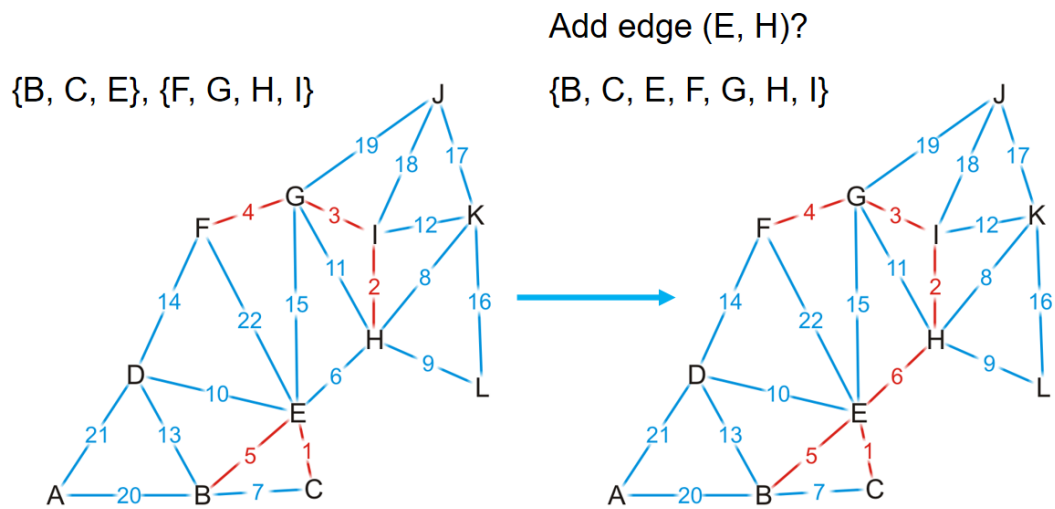
The critical operation is determining if two vertices are connected

- If we perform a traversal on the added edges, it is $O(|V|)$. Consequently, the total run-time would be $O(|E| \ln(|E|) + |E| \cdot |V|) = O(|E| \cdot |V|)$

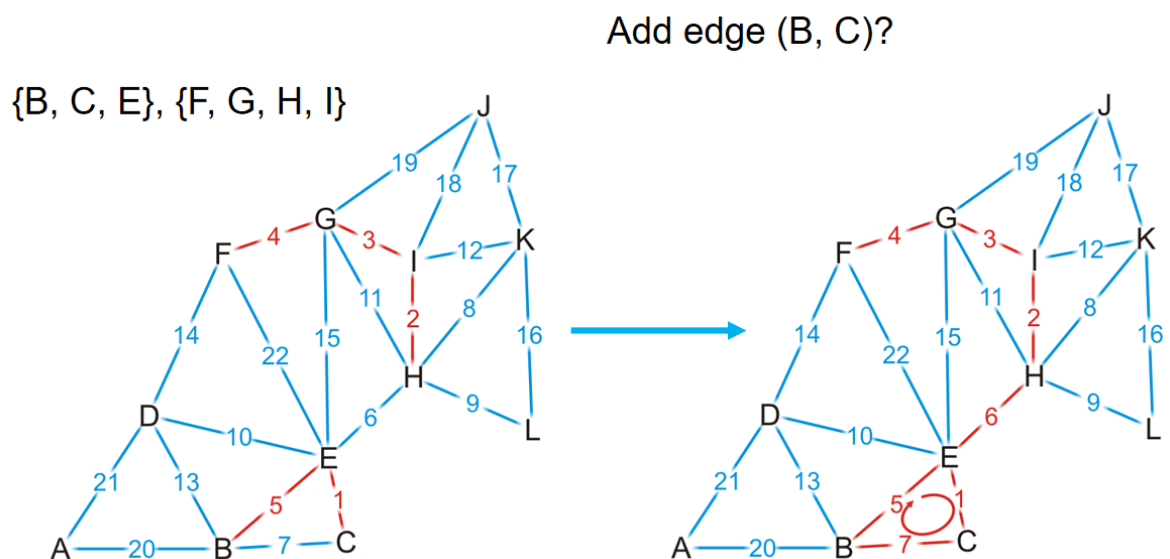
有没有更好的办法来帮助快速判断是否成环呢？**可以考虑使用并查集**。如果两个点的代表元素都是一样的话，说明这个并查集组成的树中，两个点是连通的，这两个点之间加边就会形成环。

Instead, we could use disjoint sets

- Consider edges in the same connected sub-graph as forming a set
- If the vertices of the next edge are in different sets, take the union of the two sets



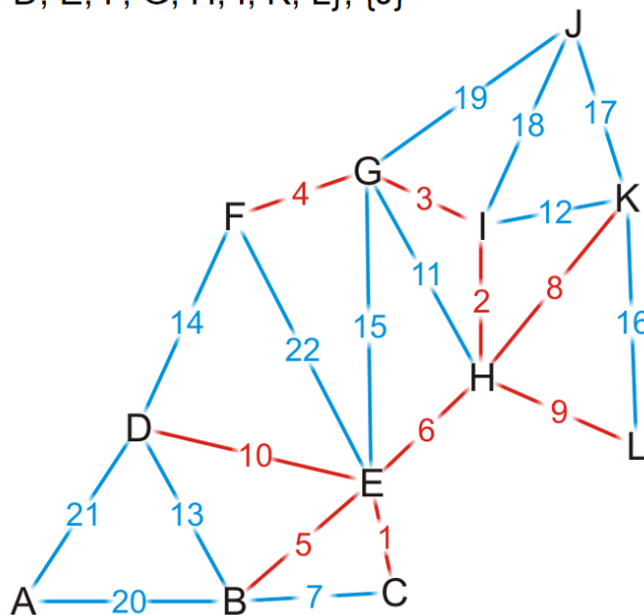
- Do not add an edge if both vertices are in the same set



Example

We add edge {H, K}, {H, L} and {D, E}

{A}, {B, C, D, E, F, G, H, I, K, L}, {J}



- {C, E}
- {H, I}
- {G, I}
- {F, G}
- {B, E}
- {E, H}
- {B, C}
- {H, K}
- {H, L}
- {D, E}
- {G, H}
- {I, K}
- {B, D}
- {D, F}
- {E, G}
- {K, L}
- {J, K}
- {J, I}
- {J, G}
- {A, B}
- {A, D}
- {E, F}

在上面的这个例子中，在GH边的判断中，由于G H在一个并查集里面，我们就会知道GH这条边如果加上将会成环。而事实也如图所示。

这样一来，由于并查集的操作在使用了路径压缩和优先级策略后，时间复杂度近似常数，因此检查和构建最小生成树的时间复杂度都是 $O(|E|)$ ；而dominant time就是sort这些边长了，时间复杂度是 $O(|E| \ln |E|) = O(|E| \ln |V|)$ 。

Uniqueness of MST

Lemma: If all edge weights in a connected graph G are distinct, then G has a unique minimum spanning tree. 这条定理的证明如下：

Proof. Greedy exchange argument → Contradiction

Step 1: Assume T and T' are two different MSTs of G , then we have

$$e \in T \setminus T' \text{ and } e' \in T' \setminus T.$$

Moreover, let e and e' be the edges with minimum weights in $T \setminus T'$ and $T' \setminus T$, respectively. Assume $w(e) \leq w(e')$.

Step 2: The subgraph $T' \cup \{e\}$ contains exactly one cycle. Let e'' be any edge of this cycle that is not in T , we have

$$w(e) \leq w(e') \leq w(e'').$$

Step 3: Construct a new MST $T'' = T' + e - e''$, which has smaller weight than T' . However, T' is a MST and we have

$$w(e) = w(e''). \text{ Contradiction!}$$

上面是经典的贪心交换论证。假设有两个最小的生成树，那么将至少会有两条边，它仅存在于其中的一棵树而不会存在于另外一颗树中。那么假设对于两棵树来说，满足上述条件的边中，权重最小的分别是 e 和 e' 边。此时将 e 并到 T' 树中注意到由于这里是树结构，任意两点之间加边都会形成一条环，因此在加上了 e 这条边之后，会包含一个环。这个环除了 e 还有至少另外一条边 e'' ，而这条边大于等于 e' 。那么构建 $T' + e - e''$ 的树，那么权重不大于 T' ，表明 T' 不会是最小生成树，形成了矛盾。

这种贪心思想和贪心论证方式如下图所示：非常的重要！

Greedy algorithms

Suppose it is possible to build a solution through a sequence of partial solutions

- At each step, we focus on one particular partial solution and we attempt to extend that solution
- Ultimately, the partial solutions should lead to a feasible solution which is also optimal

Greedy exchange argument

- Modify a solution incrementally by any other algorithm into the solution by your greedy algorithm
- Justify the modification doesn't make the solution worsen.
- Your solution is at least as good as that of any other solution (optimal solution).

Painpoint

1. Using **Cayley's formula**, the number of spanning trees for a complete graph K_n is n^{n-2}
2. Kruskal and Prim都能够处理multi-edge的情况
3. After Kruskal's algorithm, we choose m edges, then the number of connected components of G is $|V| - m$.
4. Kirchhoff's Matrix-Tree Theorem: Algorithm Workflow
Input: A graph $(G = (V, E))$ with $(|V| = n)$ vertices and $(|E|)$ edges.

Steps:

a. Construct Matrices:

- **Degree Matrix ((D)):**
A diagonal matrix where $(D[i][i])$ is the degree of vertex (i) .
- **Adjacency Matrix ((A)):**
A matrix where $(A[i][j] = 1)$ if there is an edge between (i) and (j) , and (0) otherwise.

b. Compute the Laplacian Matrix ((L)):

$$L = D - A$$

c. Select a Cofactor ((L*)):

- Remove any one row and the corresponding column from (L) .

d. Compute the Determinant:

- Calculate the determinant of (L^*) , which gives the number of spanning trees $(\tau(G))$.

Output: The number of spanning trees $(\tau(G))$.

5. 使用二叉堆优化Prim算法并不是任何时候都是更好的方法，因为如果 $|E| = \Theta(v^2)$ ，那么时间复杂度反而更高了。
6. **Kruskal使用并查集优化的具体细节：**

```
首先对所有的点进行建堆。然后对所有的边的权重进行sort。
然后每条出来一个边，有u v两个顶点，如果：
Find-Set(u) != Find-Set(v):
那么就把这条边加进answer里面，并且Union(Find-Set(u), Find-Set(v))
// 惨痛的教训。。。
```


7. 在undirected connected graph里面，最小的边并不一定会出现在所有的最小生成树里面，因为可能权重不是distinct；同时，undirected connected graph中权重最大的边也可能出现在MST里面，因为可能边长数量恰好就是顶点数量-1（树）。
8. 如何求出修改一个边长的允许范围，从而不改变MST呢？对于MST里面的边长来说，如果删掉，那么MST就会边长两个联通分量，两个联通分量之间的边长的最小值，就是这个边长的最大值；对于不在MST里面的边长来说，如果加上，那么MST中就会有环，这个环除开这条边的所有的边长的最大值，就是这个边长的最小值。
9. 如果G是一个无向图， $f(\cdot)$ 是一个严格单增的函数， $F(G)$ 是G图中所有边长权重w换成 $f(w)$ 后的图。那么如果T1 T2是G的生成树，且 $W(T_1) > W(T_2)$ ，那么 $W(F(T_1)) > W(F(T_2))$ 这个结论看似是正确的，但是实则是不一定成立的！可以通过举反例说明。
10. If G is connected, then $|E| \geq |V| - 1$; If G is acyclic, then $|E| \leq |V| - 1$
11. $G = (V, E)$'s adjacency matrix is A, where $A_{uv} = 1$ if $u, v \in E$, and 0 otherwise. Then: $\sum_{u,v \in V} A_{uv} = 2|E|$, and $(A_{uu})^2 = \text{degree}(u)$
12. If we add a new edge $e = (u, v)$ into a graph $G = (V, E)$ with unique MST to get a new graph $(G' = (V, E \cup \{e\}))$. There is at most 1 edge difference between the MST of G and G' .

贪心算法

Greedy algorithms select the locally optimal choice at each step, hoping that these local choices will lead to a global optimum.

引子——Coin changing

假设我们要找零钱，那么如何用最少的硬币来支付零钱呢？已知硬币的面值有1 2 5 10 20 50（这里举的例子是欧元）。那么假设我们每一种硬币都有很多很多，那么直接总是加尽可能大的面值的硬币，只要最后钱的金额没有超就可以了。这种策略貌似是有用的。

但是这种策略并不是任何情况都奏效：假如说硬币的面值为1 4 9 16 25 36 49呢（平方关系）。假如说要找72块钱，那么直接两个36不就好了吗？但是按照之前的策略， $72 = 49 + 16 + 4 + 1 + 1 + 1$ ，而这显然不是最优解。很明显，我们的算法并没有那么的简单，这里我们需要使用到**贪心算法**：

A greedy algorithm is an algorithm which has:

- A set of partial solutions from which a solution is built
- An *objective function* which assigns a value to any partial solution

Then given a partial solution, we

- Consider possible extensions of the partial solution
- Discard any extensions which are not feasible
- Choose that extension which minimizes the object function

This continues until some criteria has been reached

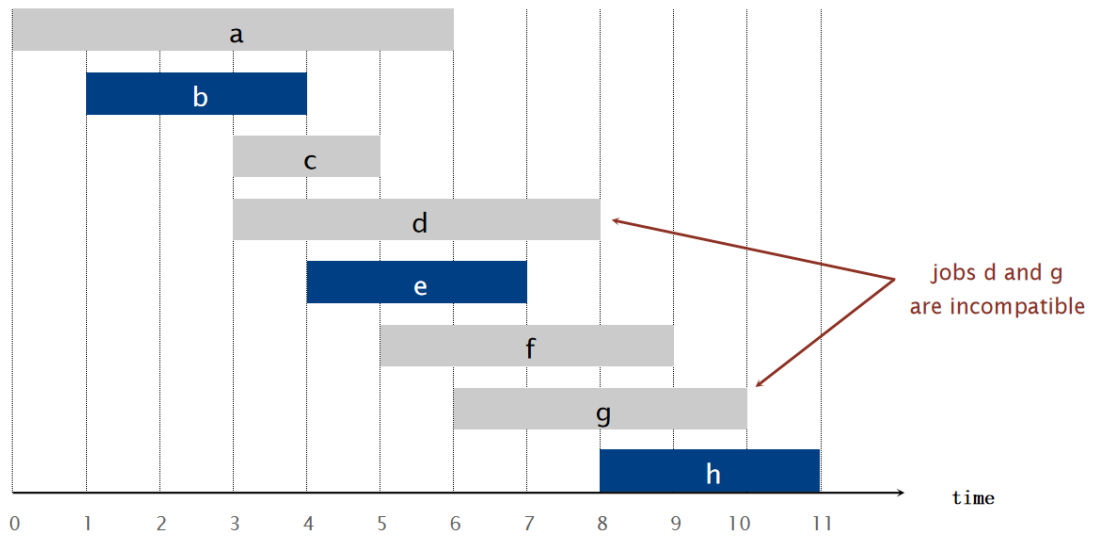
例如Prim算法就是一个贪心算法：在策略中考虑的是已经知道k个点的最小生成树，然后尝试不断expand这棵树。同样在找零钱的例子中也是用了贪心算法：在欧元的例子中，任何一个k个硬币的子集都是一种partial solution，在我们拥有一个partial solution之后，我们尝试将一个最大化零钱面额增长且不超过总金额的硬币加入到子集中。这种算法在欧元的情况能够奏效，但是在quadratic面额的硬币中却不奏效。

下面将举一些贪心算法最优的例子：

Interval scheduling

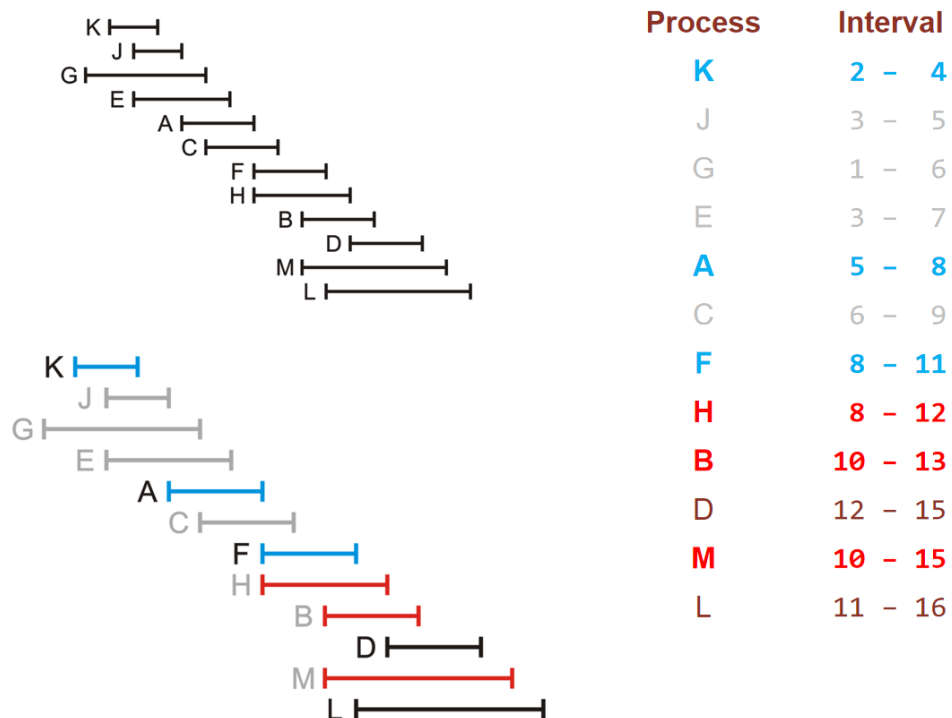
考虑如下图所示的任务：

- Job j starts at s_j and finishes at f_j .
- Two jobs are **compatible** if they don't overlap.
- Goal: find maximum subset of mutually compatible jobs.



对于这种任务，我们设计：EARLIEST-FINISH-TIME-FIRST算法：将所有的工作按照**结束时间**早晚来进行排序。然后对于这样的一个 sorted order，挑出一个元素，然后剔除掉这个 order 下之后**所有的**和这个元素的时间段有重合的元素；重复这样的操作一直到没有元素可以操作位置，然后挑出来是多少个那么就是工作子集的最大数量。示意图如下：

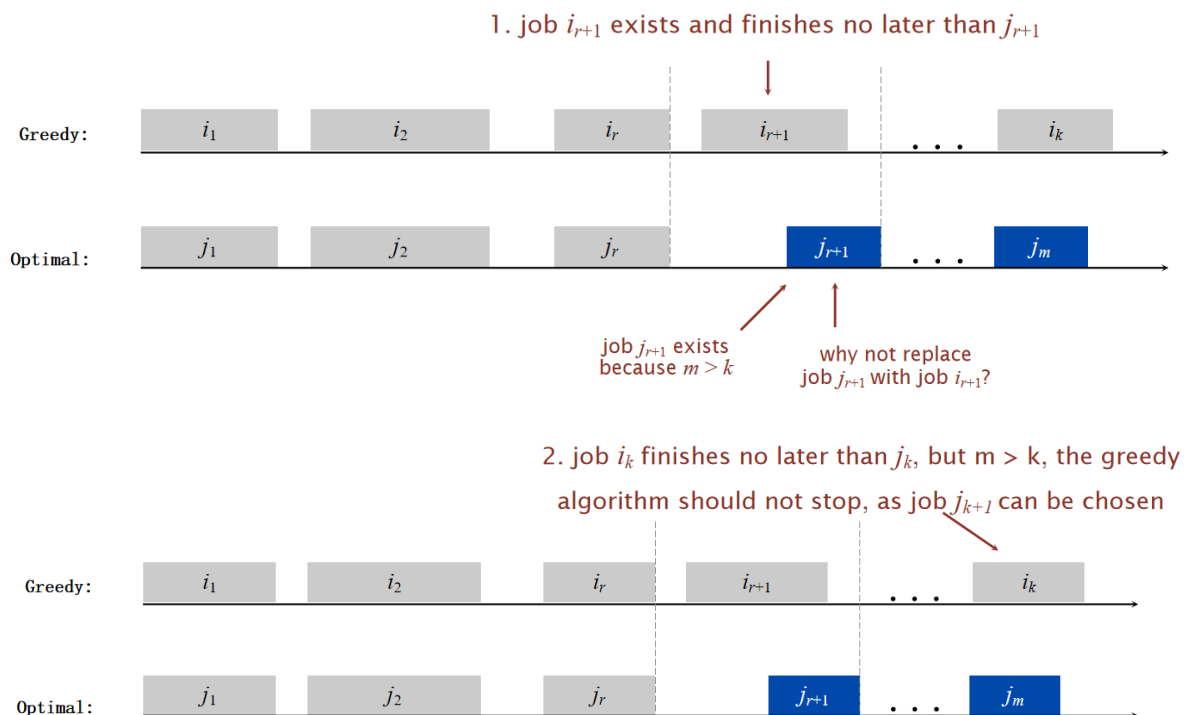
This restricts us from running
Processes H, B and M



这个算法貌似还是很简单的，但是非常奇怪的问题在于：为什么这种算法就是最优的？

Pf. [by contradiction]

- Assume greedy is not optimal, and let's see what happens.
- Let $i_1, i_2, \dots, i_k$ denote set of jobs selected by greedy.
- Let $j_1, j_2, \dots, j_m$ denote set of jobs in an optimal solution with $i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$ for the largest possible value of r .



EARLIEST-FINISH-TIME-FIRST (最早完成时间优先) 算法被认为是最优的，主要是因为它能够最大化不相互重叠的任务集合。这种算法的核心思想是：在所有可行的任务中，总是选择最早结束的任务进行调度。以下是为什么这种算法是最优的几个关键点：

1. **最小反例证明**：通过假设贪心算法不是最优解，并尝试找到一个与贪心算法结果不同的最优解，来证明这种算法的最优性。如果在每一步选择中，贪心算法选择的任务都是最早结束的，那么任何其他选择都不会比这个选择更好，因为选择更晚结束的任务会减少后续任务的可用时间窗口，从而可能导致更少的任务被安排 ”。
2. **通过矛盾证明**：假设贪心算法不是最优的，并设贪心算法选择的任务集合为 $i_1, i_2, \dots, i_k$ ，而最优解的任务集合为 $j_1, j_2, \dots, j_m$ ，其中 $i_1 = j_1, i_2 = j_2, \dots, i_r = j_r$ 对于最大的 r 值。由于贪心算法总是选择最早结束的任务，所以 $f_{i_{r+1}} < f_{j_{r+1}}$ ，且 $i_r = j_r$ 。如果用 i_{r+1} 替换 j_{r+1} ，不改变最优解的结果，并且更改后的最优解与贪心算法更相似，这说明找不到与贪心算法不同的最优解 ”。

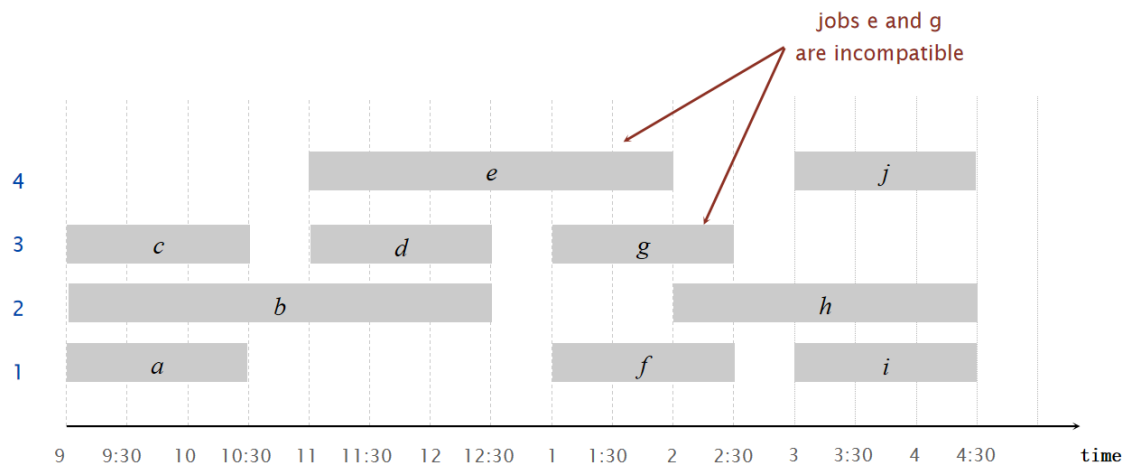
Reference: kimi.moonshot.cn

Interval Partitioning

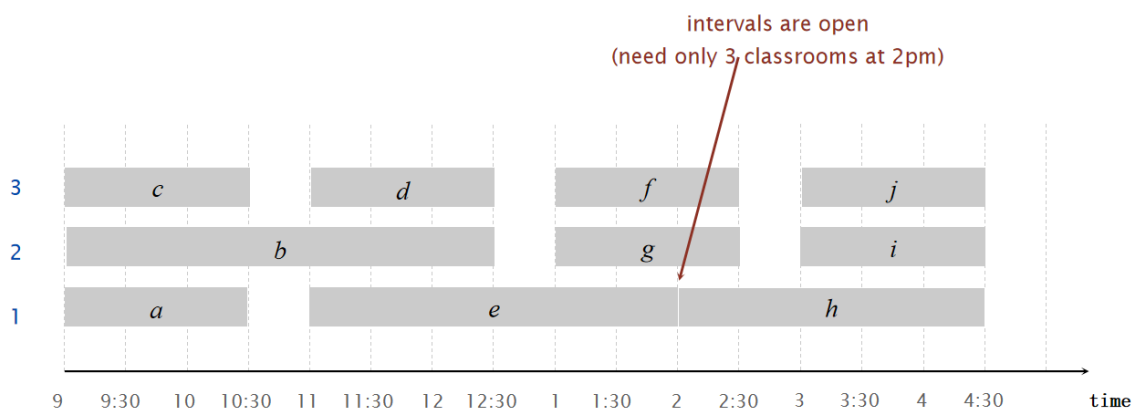
考虑下面所示的任务：

- Lecture j starts at s_j and finishes at f_j .
- Goal: find minimum number of classrooms to schedule all lectures so that no two lectures occur at the same time in the same room.

Ex. This schedule uses 4 classrooms to schedule 10 lectures.



Ex. This schedule uses 3 classrooms to schedule 10 lectures.



对于这种任务，我们设计：EARLIEST-START-TIME-FIRST算法：

EARLIEST-START-TIME-FIRST ($n, s_1, s_2, \dots, s_n, f_1, f_2, \dots, f_n$)

Sort lectures by start times and renumber so that $s_1 \leq s_2 \leq \dots \leq s_n$.

$d \leftarrow 0$. ← number of allocated classrooms

FOR $j = 1$ **TO** n

IF (lecture j is compatible with some classroom)

 Schedule lecture j in any such classroom k .

ELSE

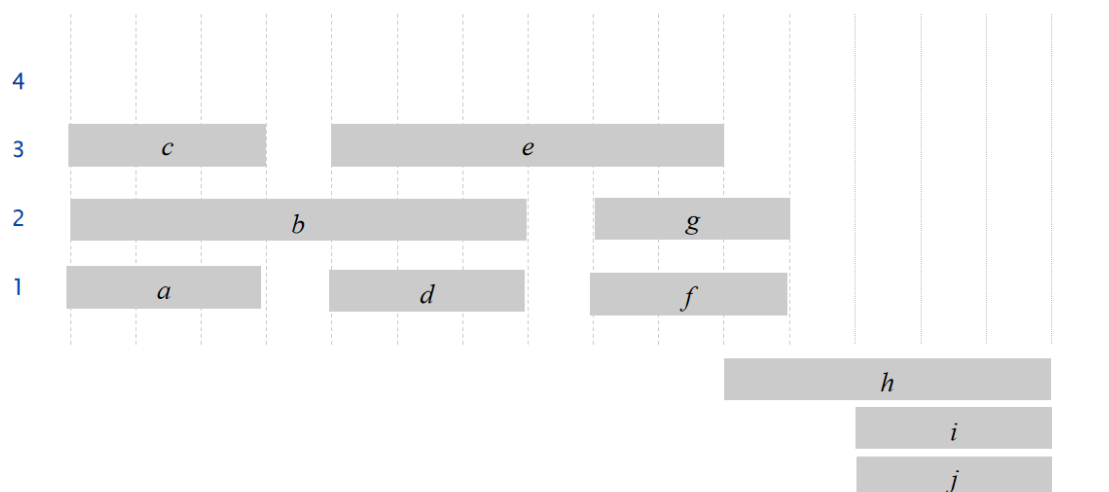
 Allocate a new classroom $d + 1$.

 Schedule lecture j in classroom $d + 1$.

$d \leftarrow d + 1$.

RETURN schedule.

如下面是流程进行到一半的实例：



Proposition :The earliest-start-time-first algorithm can be implemented in $O(n \log n)$ time.

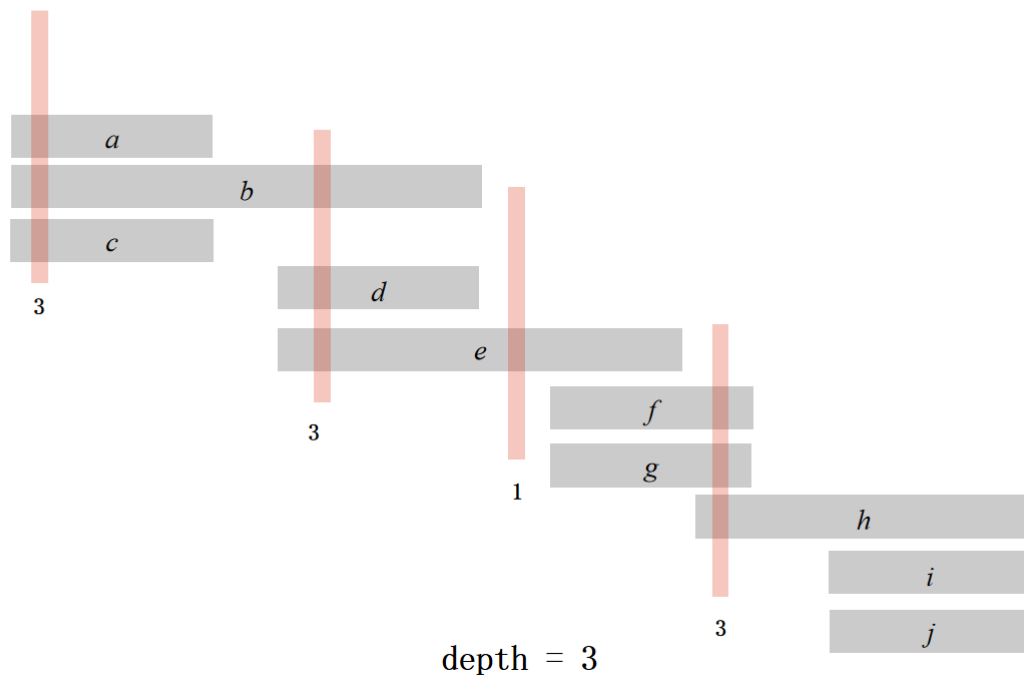
Pf. Store classrooms in a priority queue (key = finish time of its last lecture).

- To determine whether lecture j is compatible with some classroom, compare s_j to key of min classroom k in priority queue.
- To add lecture j to classroom k , increase key of classroom k to f_j .
- Total number of priority queue operations is $O(n)$.
- Sorting by start times takes $O(n \log n)$ time. ▀

同时，我们定义open interval的深度，而且能观察到：需要的教室数量至少为深度：

Def. The **depth** of a set of open intervals is the maximum number of intervals that contain any given point.

Key observation. Number of classrooms needed $\geq$ depth.



因此有 *Theorem* : Earliest-start-time-first algorithm is optimal.

Pf.

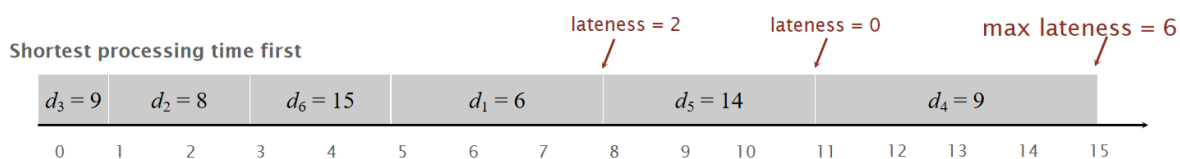
- Let d = number of classrooms that the algorithm allocates.
- Classroom d is opened because we needed to schedule a lecture, say j , that is incompatible with a lecture in each of $d - 1$ other classrooms.
- Thus, these $d - 1$ lectures each end after s_j .
- Since we sorted by start time, each of these incompatible lectures start no later than s_j .
- Thus, we have d lectures overlapping at time $s_j + \varepsilon$.
- Key observation $\Rightarrow$ all schedules use $\geq d$ classrooms. ■

Scheduling to minimize lateness

考虑下图所示的任务：

- Single resource processes one job at a time.
- Job j requires t_j units of processing time and is due at time d_j .
- If j starts at time s_j , it finishes at time $f_j = s_j + t_j$.
- Lateness: $\ell_j = \max \{ 0, f_j - d_j \}$.
- Goal: schedule all jobs to minimize **maximum** lateness $L = \max_j \ell_j$.

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



我们希望安排任务，从而导致超过预期due time的时间尽可能的小。那么对于这个任务，我们可以设计EARLIEST-DEADLINE-FIRST算法：

EARLIEST-DEADLINE-FIRST ($n, t_1, t_2, \dots, t_n, d_1, d_2, \dots, d_n$)

SORT jobs by due times and renumber so that $d_1 \leq d_2 \leq \dots \leq d_n$.

$t \leftarrow 0$.

FOR $j = 1$ **TO** n

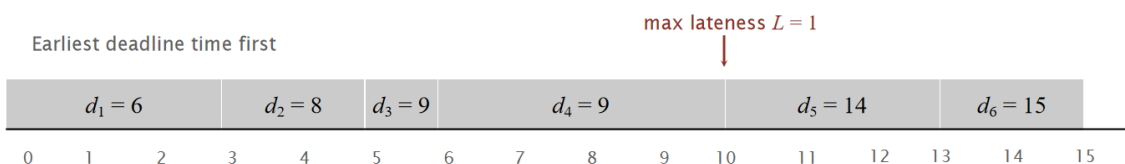
Assign job j to interval $[t, t + t_j]$.

$s_j \leftarrow t$; $f_j \leftarrow t + t_j$.

$t \leftarrow t + t_j$.

RETURN intervals $[s_1, f_1], [s_2, f_2], \dots, [s_n, f_n]$.

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15



关于这个有两个关键观察：

- 存在一种最好的安排，使得整个任务流程不存在闲置时间
- EARLIEST-DEADLINE-FIRST算法没有闲置时间
- The earliest-deadline-first schedule is the unique idle-free schedule with no inversions.

Given a schedule S , an inversion is a pair of jobs i and j such that: $d_i \leq d_j$ but j is scheduled before i .

- If an idle-free schedule has an inversion, then it has an adjacent inversion.

adjacent inversion: two inverted jobs scheduled consecutively

Theorem : The earliest-deadline-first schedule S is optimal.

Pf. [by contradiction]

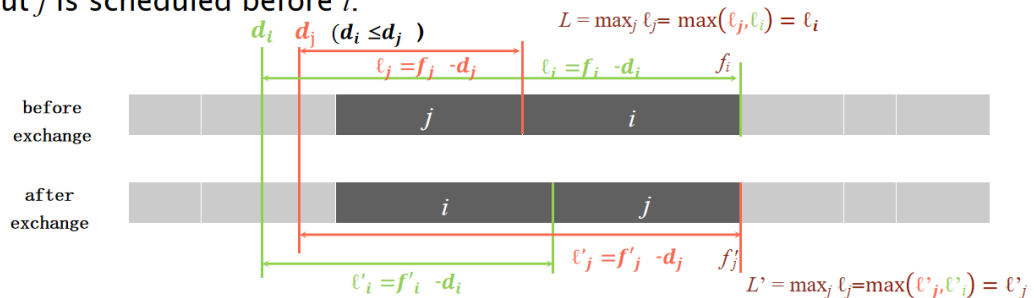
optimal schedule can have inversions

Define S^* to be an optimal schedule with the fewest inversions.

- Can assume S^* has no idle time. ← Observation 1
- Case 1. [S^* has no inversions] Then $S = S^*$. ← Observation 3
- Case 2. [S^* has an inversion]
 - let $i-j$ be an adjacent inversion ← Observation 4
 - exchanging jobs i and j decreases the number of inversions by 1 without increasing the max lateness ← key claim
 - contradicts “fewest inversions” part of the definition of S^* ✖

Def. Given a schedule S , an **inversion** is a pair of jobs i and j such that:

$d_i \leq d_j$ but j is scheduled before i .



Key claim. Exchanging two adjacent, inverted jobs i and j reduces the number of inversions by 1 and does not increase the max lateness.

Pf. Let ℓ be the lateness before the swap, and let ℓ' be it afterwards.

- $\ell'_k = \ell_k$ for all $k \neq i, j$.
- $\ell'_i \leq \ell_i$.
- If job j is late, $\ell'_j = f'_j - d_j$ ← definition
 - $= f_i - d_j$ ← j now finishes at time f_i
 - $\leq f_i - d_i$ ← $i < j \Rightarrow d_i \leq d_j$
 - $\leq \ell_i$. ← definition

Optimal Caching

在贪心算法中，“Optimal Caching”（最优缓存）问题涉及到如何在有限的缓存空间内，通过选择性地存储和替换数据，来最小化缓存未命中（cache misses）的次数。这个问题的核心是决定在缓存满时，哪个数据应该被替换出去，以便为新数据腾出空间。

根据搜索结果，最优缓存问题可以这样描述”：假设我们有一个主内存大小为 n ，缓存大小为 k ，以及一个必须处理的数据序列 $D = d_1, d_2, \dots, d_m$ 。当一个数据项不在缓存中时，会发生缓存未命中，我们需要将该数据项加载到缓存中，如果缓存已满，则需要替换掉缓存中的某个数据项。我们的目标是给出一个算法，该算法最小化缓存未命中的次数。

一个最优的贪心缓存替换策略被称为“Farthest-in-Future Algorithm”（最远未来算法）。当需要将新数据项加载到缓存中时，这个算法会贪心地替换掉在未来最迟被需要的数据项”。

为了证明“Farthest-in-Future Algorithm”是最优的，我们可以采用贪心交换的方法。这种方法的基本思想是，对于任何非简化的（非最优的）缓存维护算法，都存在一个同样好的简化（最优的）算法，该算法在给定步骤中执行最少的工作”。通过这种方式，我们可以证明，对于任何给定的缓存大小和数据访问序列，使用“Farthest-in-Future Algorithm”都能达到最少的缓存未命中次数，因此它是最优的。

Caching.

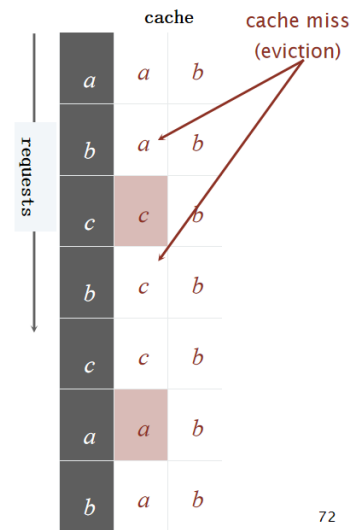
- Cache with capacity to store k items.
- Sequence of m item requests $d_1, d_2, \dots, d_m$.
- Cache hit: item in cache when requested.
- Cache miss: item not in cache when requested.
(must evict some item from cache and bring requested item into cache)

Applications. CPU, RAM, hard drive, web, browser,

Goal. Eviction schedule that minimizes the number of evictions.

Ex. $k = 2$, initial cache = ab , requests: a, b, c, b, c, a, b .

Optimal eviction schedule. 2 evictions.



直觉上，如果是最优解，那么只有在需要将新数据储存到内存中的时候，才需要进行新数据的caching，而这样的安排我们定义为 reduced schedule。

Theorem : Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions.

结论“Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions”意味着对于任何非简化的（非最优的）调度计划 S ，我们都可以将其转换为一个简化的（最优的）调度计划 S' ，而不会有更多的驱逐（替换）操作。这里的“简化的调度计划”指的是仅在必须替换时（即发生缓存未命中且缓存已满）才进行数据替换的调度计划”。

证明这个结论的思路如下：

- 1. 定义和假设：**假设 S 是一个非简化的调度计划，而 S' 是对应的简化的调度计划。我们假设在某个时间点 t ， S 在没有请求的情况下将数据 d 放入缓存，并替换掉了数据 c 。
- 2. 情况分析：**
 - **Case 1:** 在下次请求 d 之前， d 被 S 驱逐出缓存。在这种情况下， S' 可以在 d 被请求时再将其放入缓存，这样就减少了一次不必要的替换操作。
 - **Case 2:** 在 d 被驱逐之前， d 被请求。在这种情况下， S' 和 S 在 d 被请求时都会进行替换操作，因此替换次数相同。

Theorem : Farthest in Future algorithm is optimal eviction algorithm.

Topological Sort

Definition

考虑一种实际情况的任务场景：不同的task之间存在的dependency relationship，即对于一个任务来说，必须要先完成另外一个任务，才能进而完成这个任务。这种依赖关系可以形成一个部分的排序：A partial ordering on a finite number of objects can be represented as a directed acyclic graph (DAG)

同时，dependency中的环也可能造成问题，比如说完成一个任务的前提任务就是这个任务本身，那么这种关系其实就是不可取的。

Topological sorting 定义如下：

Given a set of tasks with dependencies, is there an order in which we can complete the tasks?

A topological sorting of the vertices in a DAG is an ordering

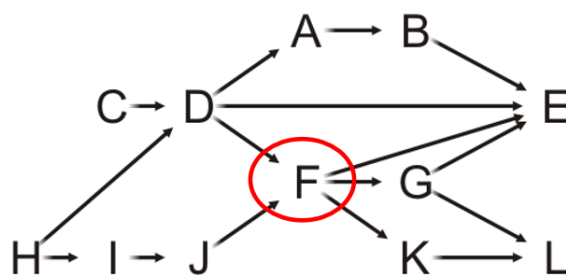
$$v_1, v_2, v_3, \dots, v_{|V|}$$

such that v_j appears before v_k if there is a path from v_j to v_k

对于任意两个连通的点，那么topological order中的相关子序列，会呈现和path中一样的dependency关系顺序。简化一下就是：如果两个点之间是连通的，那么这两个点在topological order中谁在前，谁就是被依赖的，也就是上图中的定义。下面是一个例子：

For example, there are paths from H, C, I, D and J to F, so all these must come before F in a topological sort

H, C, I, D, J, A, F, B, G, K, E, L



重要！Topological sorting的结果并不是唯一的！

Theorem : A graph is a DAG if and only if it has a topological sorting

这一点直觉上很容易认同，given that：如果一个有向图有topo sorting，但是里面有环，那么将会产生自己依赖于自己的问题，而这种关系是不可取的

但是实际的证明比较复杂：

lemma1: A DAG always has at least one vertex with in-degree zero (source)

lemma2: Any sub-graph of a DAG is a DAG

然后利用两个引理1，按照if and only if的证明逻辑去进行证明

We will start with showing $a \rightarrow b$:

If a graph is a DAG, it has a topological sort

Proof by induction:

A graph with one vertex is a DAG and it has a topological sort

Assume a DAG with n vertices has a topological sort

A DAG with $n + 1$ vertices must **have at least one vertex v of in-degree zero**

Removing the vertex v and consider the **vertex-induced** sub-graph with the remaining n vertices

- If this sub-graph has a cycle, so would the original graph—contradiction
- Thus, the graph with n vertices is also a DAG, therefore it has a topological sort

Add the vertex v to the start of the topological sort to get one for the graph of size $n + 1$

Next, we will show that $b \rightarrow a$:

If a graph has a topological ordering, it must be a DAG

We will show this by showing the contrapositive: $\neg a \rightarrow \neg b$:

If a graph is not a DAG, it does not have a topological sort

By definition, it has a cycle: $(v_1, v_2, v_3, \dots, v_k, v_1)$

- In any topological sort, v_1 must appear before v_2 , because (v_1, v_2) is a path
- However, there is also a path from v_2 to v_1 : $(v_2, v_3, \dots, v_k, v_1)$
- Therefore, v_2 must appear in the topological sort before v_1

This is a contradiction, therefore the graph cannot have a topological sort

$\therefore a \leftrightarrow b$: A graph is a DAG if and only if it has a topological sorting

Algorithm

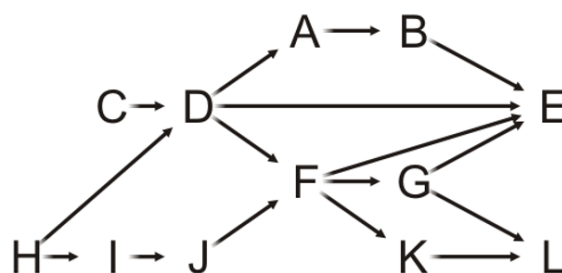
算法的idea非常的简单：找到图中的任意一个in-degree为0的节点（source），然后放入topological sorting里面，然后删除掉这个点和与这个点相关的所有的边。这个步骤重复 $|V|$ 次。

从上面的算法就能看出来：topo sorting的顺序当然不是唯一的，因为source可能不止一个

idea is cheap, 那么implementation中仍然有许多的细节：为了支持上述的算法，我们应该存储什么信息？以什么方式存储？如何更新信息？这些都是问题：

What are the tools necessary for a topological sort?

- We must know and be able to update the in-degrees of each of the vertices
- We could do this with a table of the in-degrees of each of the vertices
- This requires $\Theta(|V|)$ memory



A	1
B	1
C	0
D	2
E	4
F	2
G	1
H	0
I	1
J	1
K	1
L	2

同时，为了找到当前图中哪一个点是source，每一次sort的时候，都要遍历存储了入度的列表来发现谁可以是source，那么这样的话时间复杂的将会是 $O(|V|^2)$ 级别。当然有更好的实现方式：用一个队列来暂时储存那些source点，然后每一次有点的入度降为0那么就将它Push进队列里面。那么关于队列的时间复杂度：最开始遍历入度列表，需要 $\Theta(|V|)$ ；然后对于图中的每一个顶点来说，它们都会被push进队列一次，且被pop off一次，因此所有点的Push pop需要 $\Theta(|V|)$ 。

最后，关于全部的‘删除顶点及其对应边，从而更新入度信息的时间复杂度’还未进行分析：

Finally, every time we remove a vertex v , all its edges shall also be removed and the in-degree table be updated

- The run time of these operations is $\Omega(|E|)$
- If we are using an adjacency matrix: $\Theta(|V|^2)$
- If we are using an adjacency list: $\Theta(|E|)$

最后综合一下：Topological sort的算法复杂度为：

$\Theta(|V| + |E|)$ if we use an adjacency list; $\Theta(|V|^2)$ if we use an adjacency matrix; Memory requirement: $\Theta(|V|)$

可以看出：如果是稠密图，那么使用adjacency matrix or list都是无所谓；但是如果是稀疏图，那么就是adjacency list更好。

同时值得注意的是：如果在算法运行的过程中，发现所有剩余的顶点，它们的入度都不是0，那么就说明一定有环的出现，就可以停止 topo sorting了。换言之，**我们拥有一个 $\Theta(|V| + |E|)$ 的算法来判断图里面是否有环！**

总而言之，implementation如下：

Thus, to implement a topological sort:

- Allocate memory for and initialize an array of in-degrees
- Create a queue and initialize it with all vertices that have in-degree zero

While the queue is not empty:

- Pop a vertex from the queue
- Decrement the in-degree of each neighbor
- Those neighbors whose in-degree was decremented to zero are pushed onto the queue

更好的是：如果使用的是数组实现的queue而非STL中的队列，那么最后这个数组储存的就是topological sorting的顺序！

Finding the critical path

考虑一种实际的情景：假如说每一个任务的完成都需要对应的时间呢？如果同一时间段只能干一个任务，毫无疑问，最后的总用时就是所有任务的要求时间总和，而实现任务的顺序是任意的一个topological order。但是如果我们能够同时做多个任务呢？（注意依赖性）首先我们定义：

The critical time of each task is the earliest time that it could be completed after the start of execution

The critical path is the sequence of tasks determining the minimum time needed to complete the project

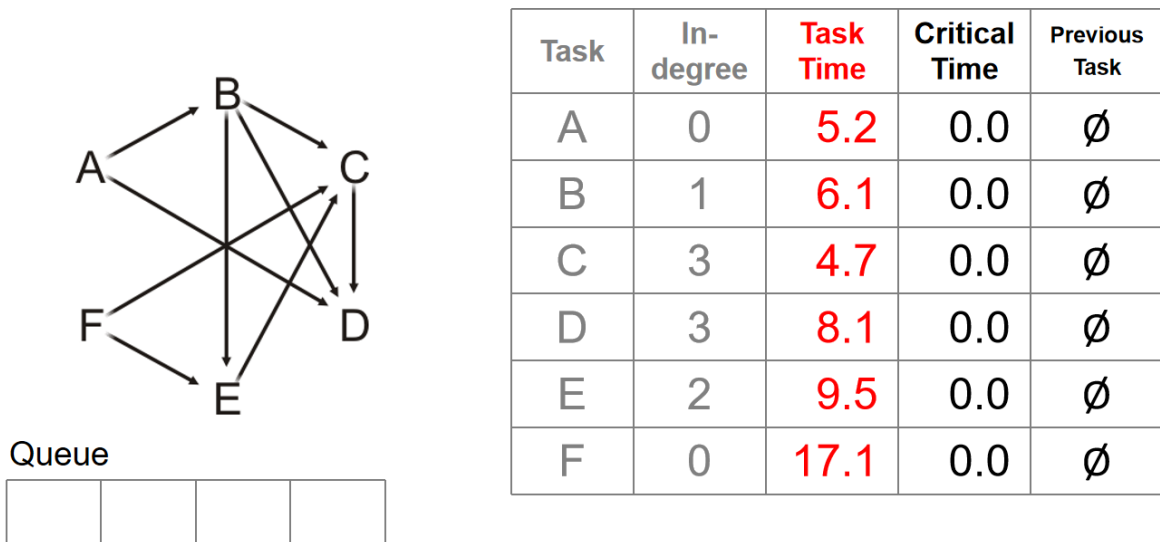
If a task on the critical path is delayed, the entire project will be delayed

具体如何找到critical path从而求出完成所有任务的最小时间要求如下图：

To find the critical time/path, we run topological sorting and require the following additional information:

- We must know the execution time of each task
- We will have to record the critical time for each task
 - Initialize these to zero
- We will need to know the previous task with the longest critical time to determine the critical path
 - Set these to null

Suppose we have the following times for the tasks



Each time we pop a vertex v , in addition to what we already do:

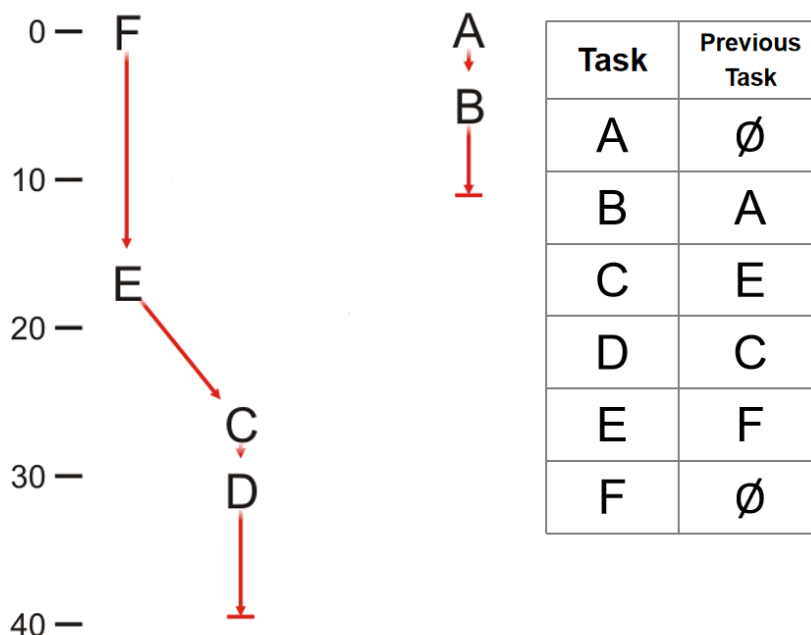
- For v , add the task time onto the critical time for that vertex:
 - That is the critical time for v
- For each adjacent vertex w :
 - If the critical time for v is greater than the currently stored critical time for w
 - Update the critical time with the critical time for v
 - Set the previous pointer to the vertex v

逻辑顺序：弹出一个顶点之后，首先更新顶点自己的critical time += task time；然后对于这个顶点的所有的邻居来说，更新入度，如果这个邻居的critical time比这个顶点的critical time小，那么Previous Task指针更新为这个顶点，更新critical time = 顶点的critical time，并且如果有入度为0的就Push进队列。

该算法的时间复杂度和topological sort一样，都是 $\Theta(|V| + |E|)$!

最后根据表格中的task and previous task信息，我们能够创建一个森林来表示平行作业的顺序：

Incidentally, the task and previous task defines a **forest** using the parental tree data structure



Painpoint

1. A critical path in a DAG is a path from the source to the sink with the maximum total weights
2. Any directed tree has a topological sorting, because they are DAG as well.
3. A DAG with all different weighted edges **may have more than one** critical path.
4. A directed graph with n vertices and $n-1$ edges **may not** have a topological sorting. Consider a triangle plus one edge, with 5 nodes in total and 4 edges in total.
5. Given a directed acyclic graph G . If u appears before v in a topological sort, there **won't** be any path from v to u in G .
6. Topological Sort的核心是：任何一条路径上的点，在拓扑排序中这些点的顺序应该保持一样的order
7. Given a DAG with probably negative edge weight and a source vertex s , we want to find the shortest path from s to every vertex. 对于这个任务，可以使用基于拓扑循环的松弛方法。给出拓扑排序后的order，然后类似于bellman ford方法，按照topological order对顶点们依次松弛。（注意将 s 的距离标为0，然后其他的顶点距离都标记为无穷）这个算法比Bellman-Ford时间复杂度较低。

Shortest Path

Definition

给出一个有权有向图，一个经典的问题就是找到两点之间shortest path。

In a weighted graph, the **length** of a path is the sum of the weights of each of the edges in that path

我们的目标就是找到shortest path并找到它的length。我们将假设所有边长的权重都是一个正数。能够实现这个目标的算法有很多种，如A*算法、Dijkstra、Bellman-Ford、Floyd-Warshall算法。

Dijkstra's Algorithm

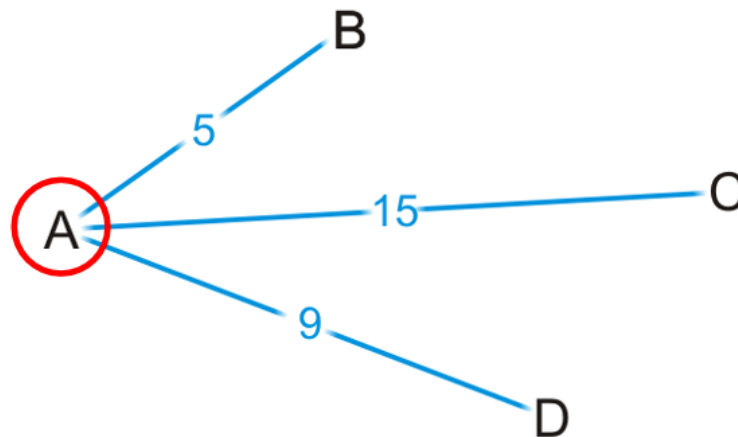
该算法只能找到一个source的最短路径。（single-source）

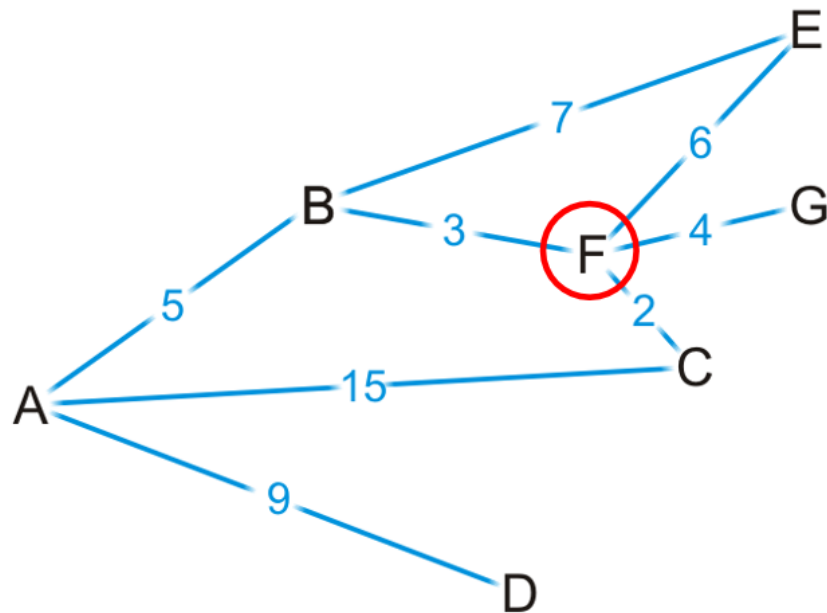
strategy

Dijkstra's algorithm solves the **single-source shortest path problem**. 这个思维和Prim算法很相似。

核心策略是：假设你現在在A顶点，我们能够知道所有的adjacent的点（这个信息是adjacency list or matrix给出来的）。假设遇到了下图的情景（只截取了A顶点相关的顶点），那么：

1. 我们能够断言从A出发到达B的最短的path就是边长AB吗？是可以的，因为假设是通过C D这两个顶点之后再到达B，那么15 and $9 > 5$
2. 但是我们断言从A出发到达C的path最短的就是AC边长吗？当然不可以。因为有可能走B D会遇到权重更短的边，从而实现shortest。如第二章图中的反例

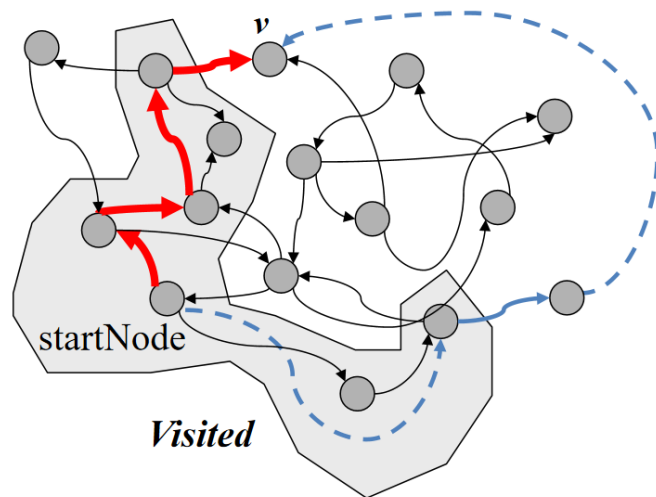




可以看出：我们对于部分的顶点来说，我们能够知道最短的路径（如上图的B），但是还有很多是无法断言的，因此我们规定：前者 marked as visited；后者 marked as unvisited。能够注意到：我们也可以通过 visited vertices 知道 the shortest distance to each unvisited vertex。

Consider the unvisited vertex v that has the shortest known distance

- We are guaranteed that the known distance to v is the shortest distance from the start node to it
- Proof by contradiction



因此：

We need to track the known shortest distance to each vertex

- We require an array of distances, all initialized to infinity except for the source vertex, which is initialized to 0

Do we need to track the shortest path to each vertex?

- Ex: do I have to store (A, B, F) as the shortest path to vertex F?
- No. We only have to record that the shortest path to vertex F came from vertex B
 - The shortest path to F is the shortest path to B followed by the edge (B, F)
- Thus, we need an array of previous vertices, all initialized to null

We need to track visited vertices whose shortest paths have been found

- a Boolean table of size $|V|$

We will iterate $|V|$ times:

- Find the unvisited vertex v that has a minimum distance to it
- Mark it as visited
- Consider its every adjacent vertex w that is unvisited:
 - Is the distance to v plus the weight of the edge (v, w) less than our currently known shortest distance to w ?
 - If so, update the shortest distance to w and record v as the previous pointer

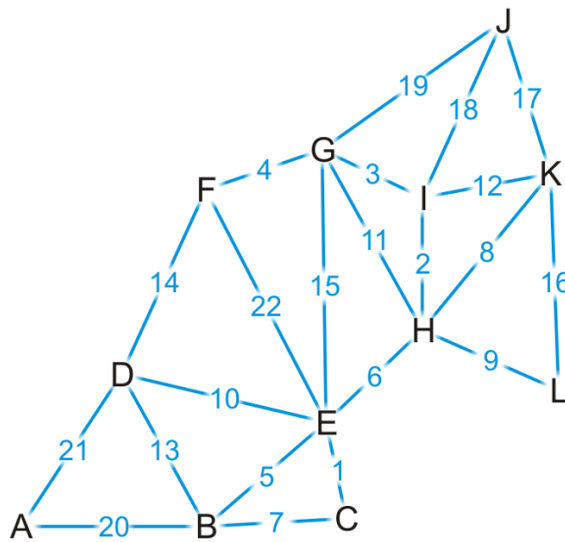
Continue iterating until all vertices are visited or **all remaining vertices have a distance of infinity**

如下图是一个寻找每一个顶点到K点的shortest path:

We will iterate $|V|$ times:

- Find the unvisited vertex v that has a minimum distance to it
- Mark it as visited
- Consider its every adjacent vertex w that is unvisited:
 - Is the distance to v plus the weight of the edge (v, w) less than our currently known shortest distance to w ?
 - If so, update the shortest distance to w and record v as the previous pointer

Continue iterating until all vertices are visited or **all remaining vertices have a distance of infinity**



Vertex	Visited	Distance	Previous
A	F	∞	$\emptyset$
B	F	∞	$\emptyset$
C	F	∞	$\emptyset$
D	F	∞	$\emptyset$
E	F	∞	$\emptyset$
F	F	∞	$\emptyset$
G	F	∞	$\emptyset$
H	F	∞	$\emptyset$
I	F	∞	$\emptyset$
J	F	∞	$\emptyset$
K	F	0	$\emptyset$
L	F	∞	$\emptyset$

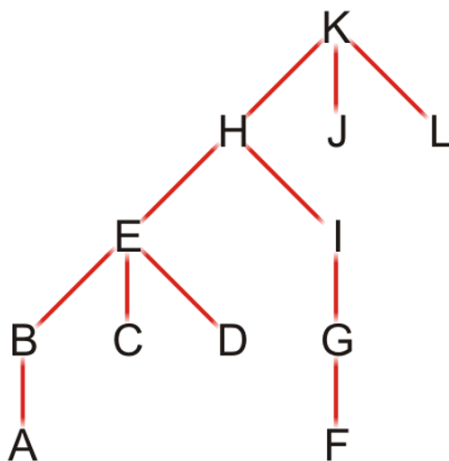
流程：每一次的操作是从一个点开始，找到所有的邻居，用这个点的distance和邻居们边的数据计算startnode到这些*unvisited*邻居的距离，然后更新表格中的distance；如果这个数据更小，那么更新distance并且previous改成这次操作的点，否则就distance previous都不修改；最后在表格visited为false的点中找到distance最小的点，作为下一个操作的点。重复操作，直到所有的点是visited。

注意：修改数据的点一定unvisited的。如果一个点最终被定为了visited，它的数据不再修改

利用表格中的previous信息，就能够找到每一个顶点到startnode最短的路径，进而可以生成一颗树。如下图所示。**值得注意的是：Dijkstra算法产生的生成树不一定是最小生成树！**

Note that this table defines a rooted parental tree (is it also a minimum spanning tree?)

- The source vertex K is at the root
- The previous pointer is the *parent* of the vertex in the tree



Vertex	Previous
A	B
B	E
C	E
D	E
E	H
F	G
G	I
H	K
I	H
J	K
K	$\emptyset$
L	K

在算法的过程中，如果出现了所有unvisited vertices的距离都是正无穷的情况，那就说明这个图并不是连通的，而目前状态找到是连通分量里面所有的点到起始点的最短路径。而如果只想找到两个点之间的最短路径，那么就选择其中的一个点进行算法，然后当另外一个点被标记为visited的时候，算法停止。而且如果图是有向图，那么这个算法依然是合理的。

implementation and analysis

The initialization requires $\Theta(|V|)$ memory and run time

We iterate $|V|$ times, each time finding next closest vertex to the source

- Iterating through the table requires $\Theta(|V|)$ time
- Each time we find a vertex, we must check all of its neighbors
 - With an adjacency matrix, the run time is $\Theta(|V|(|V| + |V|)) = \Theta(|V|^2)$
 - With an adjacency list, the run time is $\Theta(|V|^2 + |E|) = \Theta(|V|^2)$ as $|E| = O(|V|^2)$

Can we do better?

- How about using a priority queue to find the closest vertex?
 - Assume we are using a binary heap

The initialization still requires $\Theta(|V|)$ memory and run time

- The priority queue will also requires $O(|V|)$ memory
- We must use an adjacency list, not an adjacency matrix

We iterate $|V|$ times, each time finding the *closest* vertex to the source

- Place the distances into a priority queue
- The size of the priority queue is $O(|V|)$
- Thus, the work required for this is $O(|V| \ln(|V|))$

Is this all the work that is necessary?

- Recall that each edge visited may result in a distance being updated
- Thus, the work required for this is $O(|E| \ln(|V|))$

Thus, the total run time is $O(|V| \ln(|V|) + |E| \ln(|V|)) = O(|E| \ln(|V|))$?

(注意上图中Dijkstra时间复杂度的分析哪些是O哪些是 Θ)(下图中全都是O, 但是这并不是准确的; 以上图为准!)

有多种方法来维护 1 操作中最短路长度最小的结点，不同的实现导致了 Dijkstra 算法时间复杂度上的差异。

- 暴力：不使用任何数据结构进行维护，每次 2 操作执行完毕后，直接在 T 集合中暴力寻找最短路长度最小的结点。2 操作总时间复杂度为 $O(m)$ ，1 操作总时间复杂度为 $O(n^2)$ ，全过程的时间复杂度为 $O(n^2 + m) = O(n^2)$ 。
- 二叉堆：每成功松弛一条边 (u, v) ，就将 v 插入二叉堆中（如果 v 已经在二叉堆中，直接修改相应元素的权值即可），1 操作直接取堆顶结点即可。共计 $O(m)$ 次二叉堆上的插入（修改）操作， $O(n)$ 次删除堆顶操作，而插入（修改）和删除的时间复杂度均为 $O(\log n)$ ，时间复杂度为 $O((n + m) \log n) = O(m \log n)$ 。
- 优先队列：和二叉堆类似，但使用优先队列时，如果同一个点的最短路被更新多次，因为先前更新时插入的元素不能被删除，也不能被修改，只能留在优先队列中，故优先队列内的元素个数是 $O(m)$ 的，时间复杂度为 $O(m \log m)$ 。
- Fibonacci 堆：和前面二者类似，但 Fibonacci 堆插入的时间复杂度为 $O(1)$ ，故时间复杂度为 $O(n \log n + m)$ ，时间复杂度最优。但因为 Fibonacci 堆较二叉堆不易实现，效率优势也不够大¹，算法竞赛中较少使用。
- 线段树：和二叉堆原理类似，不过将每次成功松弛后插入二叉堆的操作改为在线段树上执行单点修改，而 1 操作则是线段树上的全局查询最小值。时间复杂度为 $O(m \log n)$ 。

在稀疏图中， $m = O(n)$ ，使用二叉堆实现的 Dijkstra 算法较 Bellman-Ford 算法具有较大的效率优势；而在稠密图中， $m = O(n^2)$ ，这时候使用暴力做法较二叉堆实现更优。

Reference: oi.wiki

值得注意的是，如果是稠密图，即 $|E| = \Theta(V^2)$ ，则暴力算法反而是更优的。这一点在最小生成树的时间复杂度分析中也是如此。但是斐波那契堆的使用可以保证无论是稠密图还是稀疏图，时间复杂度 $\Theta(|E| + |V| \ln(|V|))$ 都不是更差的。

Bellman-Ford Algorithm

如果有些边长是负数的话，而且只要图里面的环都不含负数边，那么 Bellman-Ford 就可以找到最短的距离。值得注意的是，这个算法比 Dijkstra's Algorithm 更慢，但是这个算法可以处理负数权重的边长，但是 Dijkstra 做不到。该算法只能找到一个 source 的最短路径。（single-source）

先介绍 Bellman-Ford 算法要用到的松弛操作（Dijkstra 算法也会用到松弛操作）。

对于边 (u, v) ，松弛操作对应下面的式子： $dis(v) = \min(dis(v), dis(u) + w(u, v))$ 。

这么做的含义是显然的：我们尝试用 $S \rightarrow u \rightarrow v$ （其中 $S \rightarrow u$ 的路径取最短路）这条路径去更新 v 点最短路的长度，如果这条路径更优，就进行更新。

Bellman-Ford 算法所做的，就是不断尝试对图上每一条边进行松弛。我们每进行一轮循环，就对图上所有的边都尝试进行一次松弛操作，当一次循环中没有成功的松弛操作时，算法停止。

每次循环是 $O(m)$ 的，那么最多会循环多少次呢？

在最短路存在的情况下，由于一次松弛操作会使最短路的边数至少 $+1$ ，而最短路的边数最多为 $n - 1$ ，因此整个算法最多执行 $n - 1$ 轮松弛操作。故总时间复杂度为 $O(nm)$ 。

但还有一种情况，如果从 S 点出发，抵达一个负环时，松弛操作会无休止地进行下去。注意到前面的论证中已经说明了，对于最短路存在的图，松弛操作最多只会执行 $n - 1$ 轮，因此如果第 n 轮循环时仍然存在能松弛的边，说明从 S 点出发，能够抵达一个负环。

⚠ 负环判断中存在的常见误区

需要注意的是，以 S 点为源点跑 Bellman-Ford 算法时，如果没有给出存在负环的结果，只能说明从 S 点出发不能抵达一个负环，而不能说明图上不存在负环。

因此如果需要判断整个图上是否存在负环，最严谨的做法是建立一个超级源点，向图上每个节点连一条权值为 0 的边，然后以超级源点为起点执行 Bellman-Ford 算法。

Reference:oi.wiki

BELLMAN-FORD(G, s)

```
1  for all  $v \in V$ 
2       $dist[v] \leftarrow \infty$ 
3       $prev[v] \leftarrow null$ 
4   $dist[s] \leftarrow 0$ 
5  for  $i \leftarrow 1$  to  $|V| - 1$ 
6      for all edges  $(u, v) \in E$ 
7          if  $dist[v] > dist[u] + w(u, v)$ 
8               $dist[v] \leftarrow dist[u] + w(u, v)$ 
9               $prev[v] \leftarrow u$ 
10 for all edges  $(u, v) \in E$ 
11     if  $dist[v] > dist[u] + w(u, v)$ 
12         return false
```

check for negative
cycles

A* search

Background

A星算法解决的是single-source图中的最短路径问题，和Dijkstra算法较为类似。假设我们对于任意两个顶点之间的路径的距离都能有一个heuristic lower bound distance，这就和地图上直接两点之间肯定直线最短，因此路径肯定至少比这条两点间的线段长。

考虑从A出发到达B的情景：假设A B两点之间的直线距离是53，那么说明AB路径至少是53；又假设我们已知A地到C地的最短路径是28，而C B两点之间的直线距离是32，那么就说明：延长A C路径从而抵达B的路径的距离至少是60。我们用 $w(C)$ 记号来表示：在从A出发沿着某条路径到达C点，并且将继续延伸从而达到B点的情境约束下，A B两点之间的可能的最小路径距离。即：

$$w(A) = d(A, A) + h(A, B) = 0 + 53 = 53;$$

$$w(C) = d(A, C) + h(C, B) = 28 + 32 = 60;$$

where d stands for distance and h stands for heuristic

Strategy

Suppose we are finding the shortest path from vertex a to a vertex z

The A* search algorithm initially:

- Marks each vertex as unvisited
- Starts with a priority queue containing only the initial vertex a
 - The priority of any vertex v in the queue is the weight $w(v)$ which assumes we have found the shortest path to v (initialize it to be infinity except for the initial vertex a)
 - Shortest weights have highest priority
- For each vertex v , $d(a, v)$ is the shortest known distance from a to v , $d(a, a) = 0$ and $d(a, v) = \text{infinity}$ for all $v \neq a$
- For each vertex v , $h(v, z)$ is the heuristic distance from v to z

接着，再迭代的过程中，可以考虑Tree Search的角度：

The algorithm then iterates:

- Pop the vertex u with highest priority
- For each adjacent vertex (neighbor) v of u :
 - If $w(v) = d(a, u) + d(u, v) + h(v, z)$ is less than the current weight/priority of v , update the path leading to v and its priority
 - If v is not in the queue, push v into the queue

Continue iterating until the item popped from the priority queue is the destination vertex z

也可以考虑Graph Search的角度：

The algorithm then iterates:

- Pop the vertex u with highest priority
 - mark u as visited
- For each **unvisited** adjacent vertex (neighbor) v of u :
 - If $w(v) = d(a, u) + d(u, v) + h(v, z)$ is less than the current weight/priority of v , update the path leading to v and its priority
 - If v is not in the queue, push v into the queue

Continue iterating until the item popped from the priority queue is the destination vertex z

在实现的过程中：The A star search algorithm directs its search towards the destination, whereas Dijkstra's algorithm radiates out from the initial vertex.

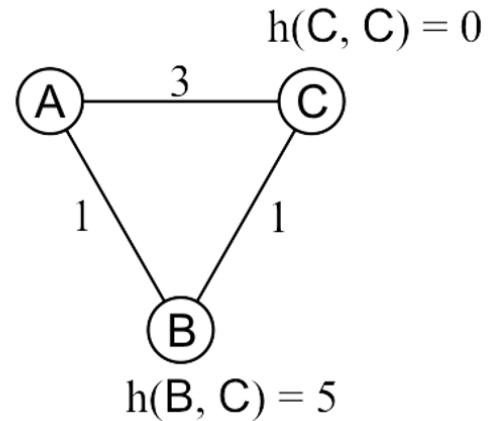
但是在上述的实现过程中，我们发现：这个heuristic lower bound信息保不保熟？如果是地图上面寻找最短路径，尚可保熟，因为Euclidean距离不会骗人。

The path the crow flies is always shorter than the road the wolf runs.

但是如果是更为广泛的情景呢？A星算法真的能总是给出最优解吗？考虑下面的场景：

- Find the shortest path from A to C:
 - B is enqueued with weight $w(B) = 1 + 5 = 6$
 - C is enqueued with weight $w(C) = 3 + 0 = 3$

- Therefore, C is dequeued next and as it is the destination, we are finished



Reference: Andrew Moore, Carnegie Mellon University

在上面的场景中, $h(B, C)$ 过度预计了 BC 两节点之间的真实距离, 从而导致最终算法没有倾向于 ABC 的路径。那么什么样的 heuristics 是非常合适的呢?

第一种: $d(u, v)$ 代表 u, v 两点之间的最短路径距离, 那么如果 $h(u, v) \leq d(u, v)$, 那么 $h(u, v)$ 就称为 admissible。换言之, 这样的 heuristic is optimistic or a lower bound on the distance。从这个观点重新看 Euclidean distance: 地图上两点之间的欧几里得距离一定是最短的, 明显 $h(u, v) \leq d(u, v)$ 。那么这样的 heuristic 表现如何呢? 这些条件表现很好! 有以下的定理:

Theorem : If $h(n)$ is admissible, A star algorithm using TREE-SEARCH is optimal.

第二种: 我们定义: A heuristic is consistent if for every node n , every successor n' of n generated by any action a , we have: $h(n) \leq c(n, a, n') + h(n')$ 。那么因此: $w(n') \geq w(n)$, i.e., $w(n)$ 沿着路径不会下降。这样的 heuristic 也表现的很好, 有以下的定理:

Theorem : If $h(n)$ is consistent, A star algorithm using GRAPH-SEARCH is optimal.

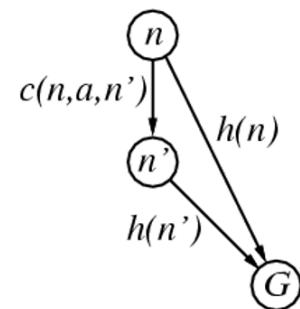
A heuristic is **consistent** if for every node n , every successor n' of n generated by any action a , we have

$$h(n) \leq c(n, a, n') + h(n')$$

If h is consistent, we have

$$\begin{aligned} w(n') &= d(n') + h(n') && \text{(by def.)} \\ &= d(n) + c(n, a, n') + h(n') && (d(n') = d(n) + c(n, a, n')) \\ &\geq d(n) + h(n) = w(n) && \text{(consistency)} \\ w(n') &\geq w(n) \end{aligned}$$

i.e., $w(n)$ is non-decreasing along any path.



It's the triangle inequality !

Time Complexity

Exponential: $O(b^d)$ where b is the branching factor (the average number of successors per state) and d is the depth of the solution

Can be shown to run in polynomial time if

$$|h(u, v) - d(u, v)| = O(\ln(d(u, v)))$$

where $d(u, v)$ is the length of the actual shortest path¹

- I.e., doubling the length of the optimal solution only increases the error by a constant
- Not likely with a road map and the Euclidean distance
- E.g. when $h(u, v) = d(u, v)$

A* for N puzzle problem

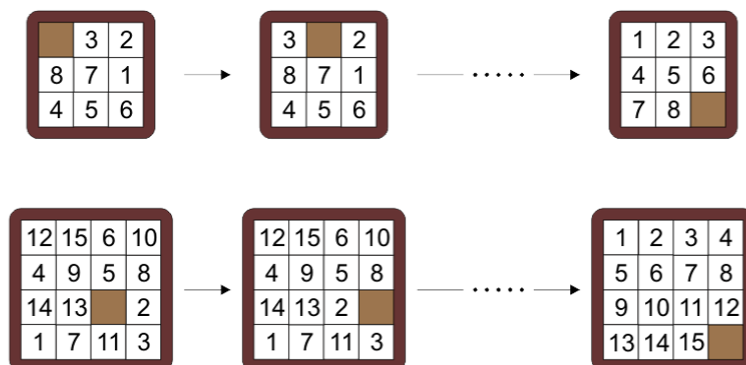
Consider find the solution to the following puzzle

- Take a random permutation of 8 or 15 numbered tiles and a blank formed in a rigid square

	3	2
8	7	1
4	5	6

12	15	6	10
4	9	5	8
14	13		2
1	7	11	3

The objective is to find a minimum number of moves which will transform the tiles into a standard solution



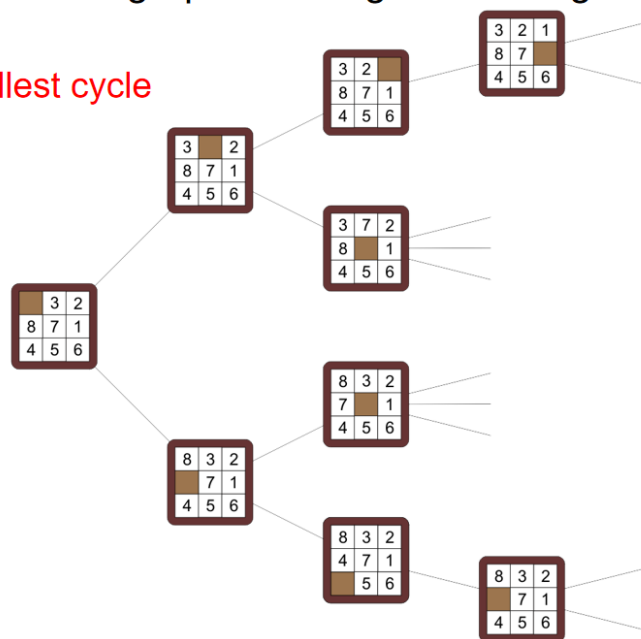
对于3×3和4×4的puzzle，分别共有9! 和16! 种状态，每走一步就会完成一次状态的转化。因此可以用下面的状态节点关系图来表示 solution:

注意，这个图不是树，因为可以通过一系列的操作回到原来的状态

Each solution defines a vertex in a graph with edges denoting allowable moves

- It is not a tree, but the smallest cycle has a length of 12

- E.g., cycle 8, 7, and 3

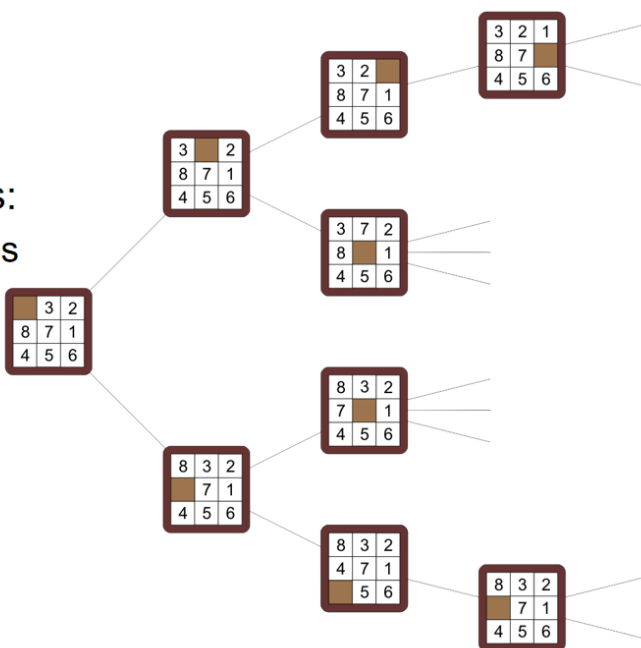


The graph of solvable eight puzzles has:

- 181 440 vertices
- 241 920 edges

The fifteen puzzle graph has:

- 10 461 394 944 000 vertices
- 15 692 092 416 000 edges



In general and in the limit:

- $(N + 1)!/2$ vertices
- $(N + 1)!$ edges

那么问题就转化成了：在上述的状态节点关系图中找到固定两个状态之间的最短路径（所有边的权重都是1）。那么可想而知的是，散发式的Dijkstra算法将会非常的慢。那么倘若使用A星算法，那么这个heuristic lower bound应该如何定义呢？我们可以考虑三种距离：Discrete Distance, Hamming Distance and Manhattan Distance。

The discrete distance

- If two objects are equal, the distance is 0, otherwise it is 1

The Hamming distance

- The number of tiles (including the blank) which are in an incorrect location

The Manhattan distance

- The sum of the minimum number of moves required to put a tile in its correct location

Hamming heuristic是一种简单的启发式函数，它通过计算当前状态中方块位置与目标状态中相应方块位置不匹配的数量来评估当前状态与目标状态的距离。这种启发式函数对于指导A星算法在搜索树中选择最有希望的路径非常有效。

曼哈顿距离 (Manhattan Distance) 启发式函数是一种常用的估计函数。它通过计算每个方块从当前位置到目标位置在水平和垂直方向上的距离之和来评估当前状态与目标状态之间的距离。具体来说, 对于每个方块, 曼哈顿距离是其当前位置与目标位置在行和列上差异的绝对值之和。

Reference: wiki.moonshot.cn

For example, consider this permutation:

- It does not equal the solution, so the discrete distance is 1

5	2	7
8	4	
1	3	6

- 8 out of 9 tiles/blanks are in the incorrect location; therefore the Hamming distance is 8

5	2	7
8	4	
1	3	6

- It requires $2 + 0 + 4 + 2 + 1 + 1 + 2 + 3 + 1 = 16$ moves to move each tile into the correct location, therefore the Manhattan distance is 16

5	2	7
8	4	
1	3	6

如果使用的是discrete distance, 那么相当于是Dijkstra算法, 表现依然很差; 如果使用的是Hamming Distance, 那么表现也并没有改善很多 (only a small change), 仅仅只在当前状态和目标状态已经十分接近的时候才会表现良好; 而**Manhattan Distance**能够使得**A星算法拥有很好的表现**。上述的内容很好地、感性地解释了'heuristic,启发式'的真正含义是什么。

Floyd-Warshall

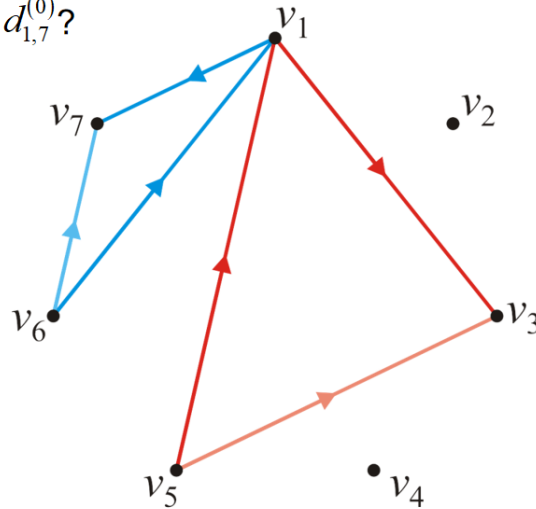
Dijkstra算法找到一个顶点到另外所有顶点的最短路径所需要的时间复杂度是: $O(|E|\ln|V|)$ 。也就为着, 找到所有的顶点对之间的最短路径, 需要的时间复杂度是 $O(|V||E|\ln|V|)$ 。在稠密图中, 最坏的情况便是 $O(|V|^3\ln|V|)$ 的时间复杂度。有没有更好的算法呢? Floyd-Warshall算法应运而生: It works with positive or negative weights with no negative cycle. 这个算法能够找到任意一个source的最短距离。

Strategy

考虑两点之间的距离关系: 如果两个点其实是一个点, 那么距离就是0; 如果两个点不同, 而且之间有一条边, 那么距离就是这条边长的权重; 如果两个不同的点, 而且之间没有边长, 那么距离就是无穷。明显, 在上述的考虑中, 我们只考虑了直接连接两个顶点的边长。

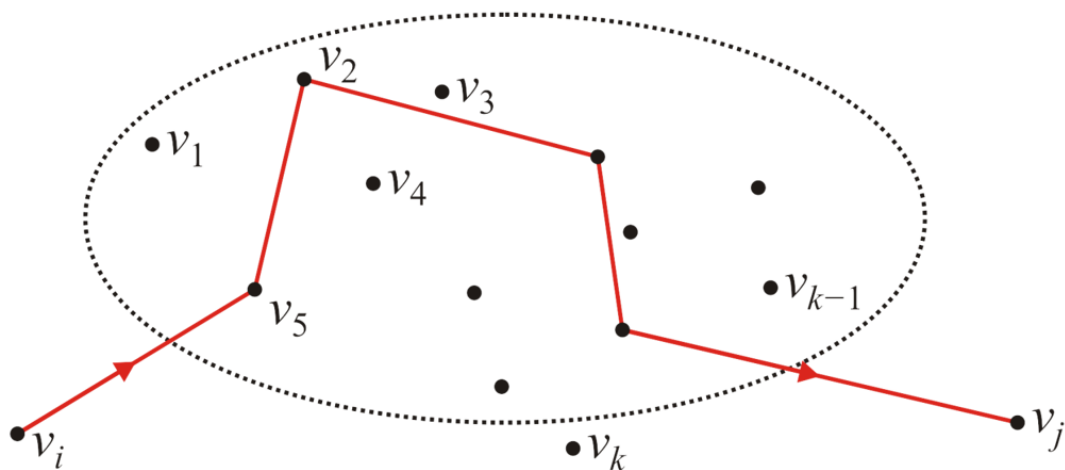
Suppose now, we want to see whether or not the path going through vertex v_1 is shorter than a direct edge?

- Is $d_{5,3}^{(0)} > d_{5,1}^{(0)} + d_{1,3}^{(0)}$?
- Is $d_{6,7}^{(0)} > d_{6,1}^{(0)} + d_{1,7}^{(0)}$?



但是两点之间的最短距离用上述的定义明显是不妥的。如上图所示，虽然我们可能知道67 53两点之间的边长的权重，但是我们也可能会怀疑：经过了其他点的两点之间的路径，会不会比这条边长的权重更短？

因此在上述的情境中，我们定义 $d_{i,j}^{(1)} = \min\{d_{i,j}^{(0)}, d_{i,1}^{(0)} + d_{1,j}^{(0)}\}$. 同时我们定义： $d_{i,j}^{(k-1)}$ 是只允许中途访问顶点 $v_1, \dots, v_{k-1}$ 的最短距离。



假设现在我们已经知道了所有顶点对的 $d_{i,j}^{(k-1)}$ ，那么对于 v_i, v_j 这两个顶点来说，假如说出现了一个新的定点 v_k ，那么应该如何更新 i, j 两点的 $d_{i,j}^{(k)}$ ？核心的观察是：两点之间的最短路径一定只有两个可能：一种是包含了顶点 v_k ，一种是不包含。如果包含，那么可想而知的是， i 到 k 与 k 到 j 的路径的距离都应该是最短的，i.e., $d_{i,k}^{(k-1)}$ & $d_{k,j}^{(k-1)}$ ，而这些信息我们都应该是知道的。因此我们有了更新 $d_{i,j}^{(k)}$ 的逻辑：

$$d_{i,j}^{(k)} = \min\{d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}\}$$

而用这种逻辑就能够更新所有的顶点对的 $d_{i,j}^{(k)}$ 。而 k 可以从 0 一直遍历到顶点数量。因此 C++ 示意代码如下：

```
for ( int k = 0; k < num_vertices; ++k ) {
    for ( int i = 0; i < num_vertices; ++i ) {
        for ( int j = 0; j < num_vertices; ++j ) {
            d[i][j] = std::min( d[i][j], d[i][k] + d[k][j] );
        }
    }
}
```

可以看出，算法的复杂度是 $\Theta(|V|^3)$ 。可以通过下列的一系列图管中窥豹：

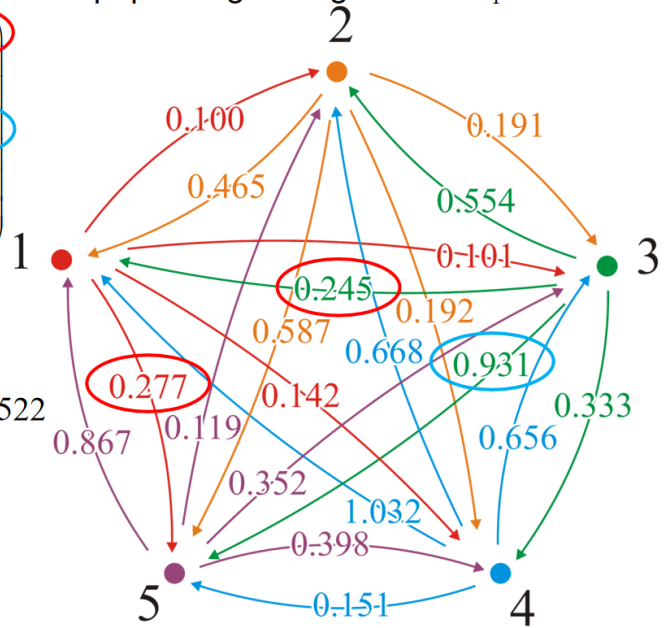
With the first pass, $k = 1$, we attempt passing through vertex v_1

0	0.100	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.931
1.032	0.668	0.656	0	0.151
0.867	0.119	0.352	0.398	0

And a second shorter path:

$(3, 5) \rightarrow (3, 1, 5)$

$$0.931 > 0.245 + 0.277 = 0.522$$



With the next pass, $k = 2$, we attempt passing through vertex v_2

0	0.100	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
1.032	0.668	0.656	0	0.151
0.867	0.119	0.352	0.398	0

There are three shorter paths:

$(5, 1) \rightarrow (5, 2, 1)$

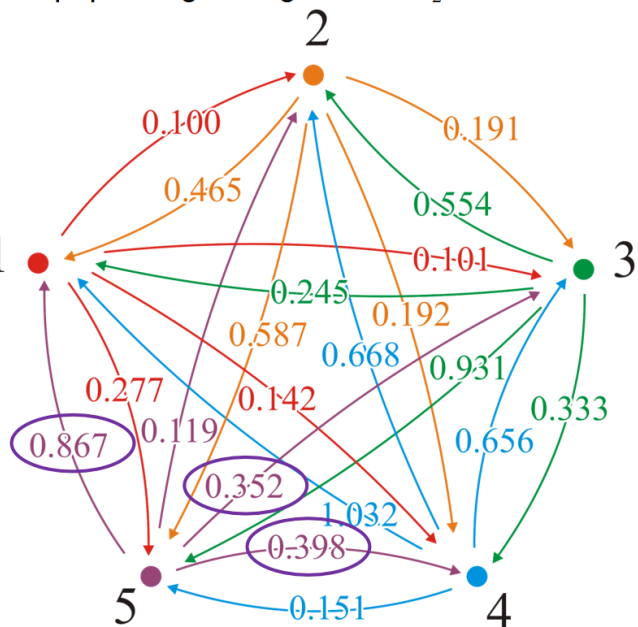
$$0.867 > 0.119 + 0.465 = 0.584$$

$(5, 3) \rightarrow (5, 2, 3)$

$$0.352 > 0.119 + 0.191 = 0.310$$

$(5, 4) \rightarrow (5, 2, 4)$

$$0.398 > 0.119 + 0.192 = 0.311$$



With the next pass, $k = 3$, we attempt passing through vertex v_3

0	0.100	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
1.032	0.668	0.656	0	0.151
0.584	0.119	0.310	0.311	0

There are three shorter paths:

$(2, 1) \rightarrow (2, 3, 1)$

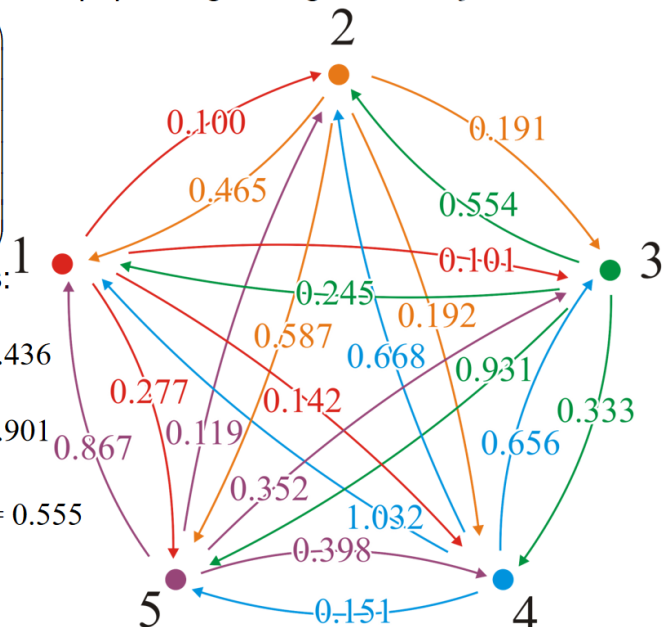
$$0.465 > 0.191 + 0.245 = 0.436$$

$(4, 1) \rightarrow (4, 3, 1)$

$$1.032 > 0.656 + 0.245 = 0.901$$

$(5, 1) \rightarrow (5, 3, 1)$

$$0.584 > 0.310 + 0.245 = 0.555$$



With the next pass, $k = 3$, we attempt passing through vertex v_3

0	0.100	0.101	0.142	0.277
0.465	0	0.191	0.192	0.587
0.245	0.345	0	0.333	0.522
1.032	0.668	0.656	0	0.151
0.584	0.119	0.310	0.311	0

There are three shorter paths:

$(2, 1) \rightarrow (2, 3, 1)$

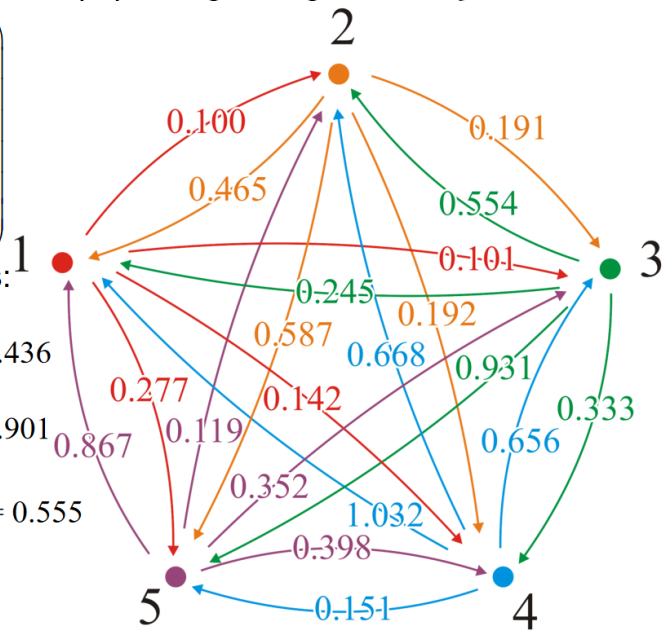
$$0.465 > 0.191 + 0.245 = 0.436$$

$(4, 1) \rightarrow (4, 3, 1)$

$$1.032 > 0.656 + 0.245 = 0.901$$

$(5, 1) \rightarrow (5, 3, 1)$

$$0.584 > 0.310 + 0.245 = 0.555$$



With the last pass, $k = 5$, we attempt passing through vertex v_5

0	0.100	0.101	0.142	0.277
0.436	0	0.191	0.192	0.343
0.245	0.345	0	0.333	0.484
0.901	0.668	0.656	0	0.151
0.555	0.119	0.310	0.311	0

There are three shorter paths:

$(4, 1) \rightarrow (4, 5, 1)$

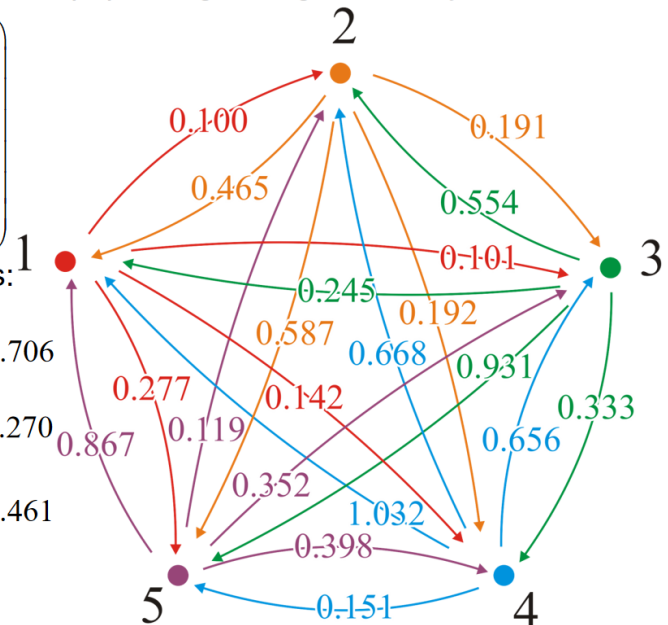
$$0.901 > 0.151 + 0.555 = 0.706$$

$(4, 2) \rightarrow (4, 5, 2)$

$$0.668 > 0.151 + 0.119 = 0.270$$

$(4, 3) \rightarrow (4, 5, 3)$

$$0.656 > 0.151 + 0.310 = 0.461$$



可见，在使用adjacency matrix的时候，在经过的点为矩阵中的第*i*行第*j*列的时候，挑出这两条线上的数据；然后意义比对矩阵中其他数据和投影在两条线上的数据的和，如果和更小，那就更新。比如说*k*=3的时候，对应的是矩阵中第三行第三列的数据，那么例如对1.032来说，位置投影到两条线上得到的数据是0.245,0.656，因此比较1.032与0.245+0.656。

What is the shortest path?

还有一个问题：距离倒是更新了，算法中如何储存路径的节点信息？

This algorithm finds the shortest distances, but what are the paths corresponding to those shortest distances?

- Recall that with Dijkstra's algorithm, we could find the shortest paths by recording the previous vertex
- Here we use a similar approach, but we choose to store the next vertex instead of the previous vertex


```

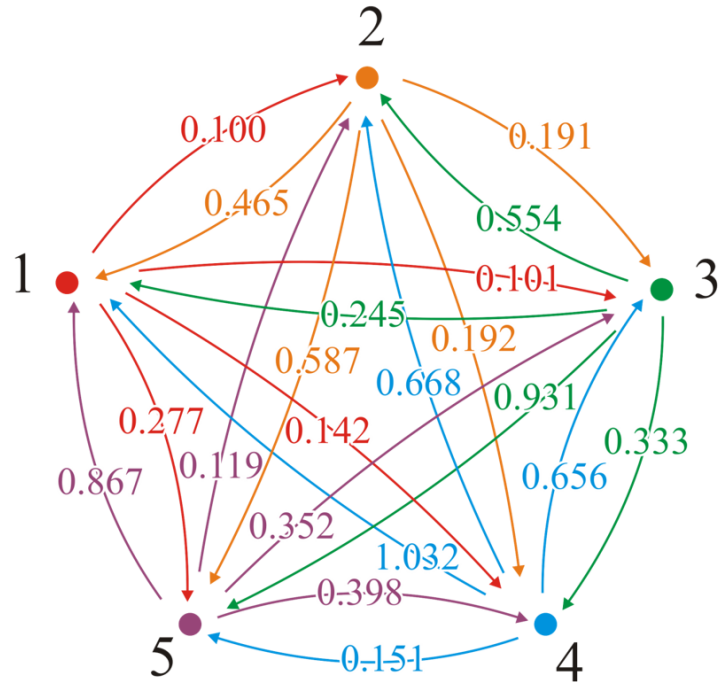
for ( int k = 0; k < num_vertices; ++k ) {
    for ( int i = 0; i < num_vertices; ++i ) {
        for ( int j = 0; j < num_vertices; ++j ) {
            if ( d[i][j] > d[i][k] + d[k][j] ) {
                p[i][j] = p[i][k];
                d[i][j] = d[i][k] + d[k][j];
            }
        }
    }
}

```

In our original example, initially, the next vertex is exactly that:

$$\begin{pmatrix} - & 2 & 3 & 4 & 5 \\ 1 & - & 3 & 4 & 5 \\ 1 & 2 & - & 4 & 5 \\ 1 & 2 & 3 & - & 5 \\ 1 & 2 & 3 & 4 & - \end{pmatrix}$$

This would define our matrix $\mathbf{P} = (p_{ij})$



With the first pass, $k = 1$, we attempt passing through vertex v_1

$$\begin{pmatrix} - & 2 & 3 & 4 & 5 \\ 1 & - & 3 & 4 & 5 \\ 1 & \textcircled{2} & - & 4 & \textcircled{5} \\ 1 & 2 & 3 & - & 5 \\ 1 & 2 & 3 & 4 & - \end{pmatrix} \quad \begin{pmatrix} - & 2 & 3 & 4 & 5 \\ 1 & - & 3 & 4 & 5 \\ 1 & \textcircled{1} & - & 4 & \textcircled{1} \\ 1 & 2 & 3 & - & 5 \\ 1 & 2 & 3 & 4 & - \end{pmatrix}$$

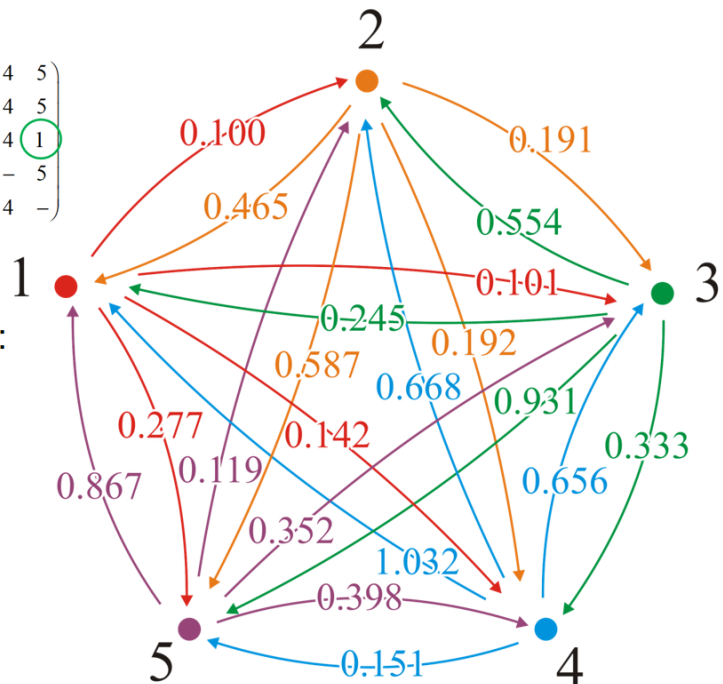
There are two shorter paths:

$(3, 2) \rightarrow (3, 1, 2)$

$$0.554 > 0.245 + 0.100$$

$(3, 5) \rightarrow (3, 1, 5)$

$$0.931 > 0.245 + 0.277$$



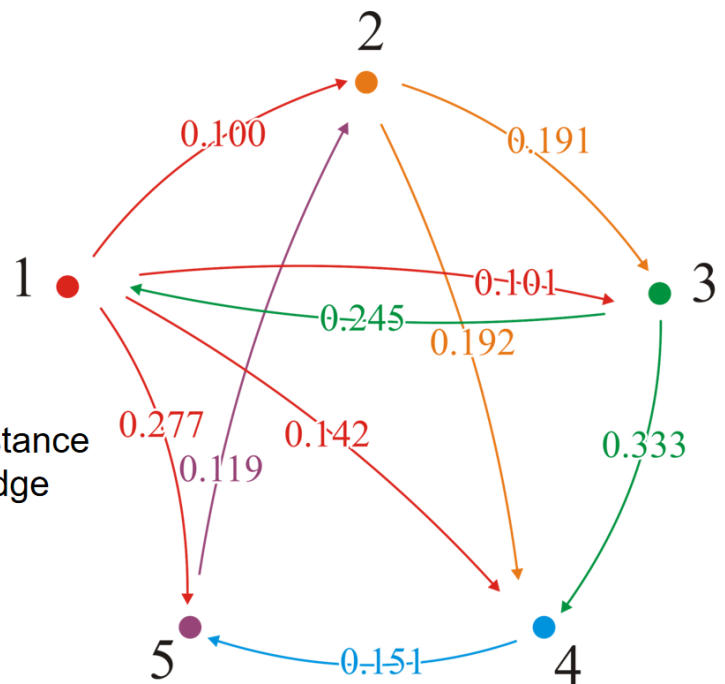
初始化连接顺序矩阵式之后，在distance更新的过程中，如果说这一轮经过的是i顶点，那么在修改distance的连接顺序矩阵中对应位置改成i即可。

These are all the adjacent edges that are still the shortest distance

-	2	3	4	5
3	-	3	4	4
1	1	-	4	4
5	5	5	-	5
2	2	2	2	-

For each of these, $p_{ij} = j$

In all cases, the shortest distance from vertex 1 is the direct edge



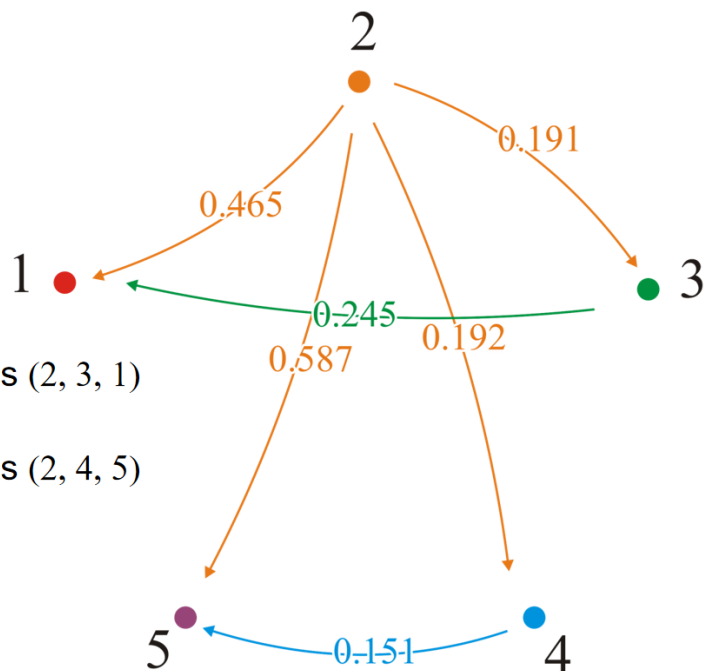
可以观察到，在最后的結果中，如果 $p_{i,j} = j$ ，那么就说明这条相邻的边就是 shortest path 的组成部分。那么如何判断路径呢？假设 $p_{i,j} = v_1, p_{v_1,j} = v_2, \dots, p_{v_t,j} = j$ ，那么路径就是 $(i, v_1, v_2, \dots, v_t, j)$ 。

From vertex v_2 , $p_{2,3} = 3$ and $p_{2,4} = 4$; we go directly to vertices v_3 and v_4

-	2	3	4	5
3	-	3	4	4
1	1	-	4	4
5	5	5	-	5
2	2	2	2	-

But $p_{2,1} = 3$ and $p_{3,1} = 1$;
the shortest path to v_1 is $(2, 3, 1)$

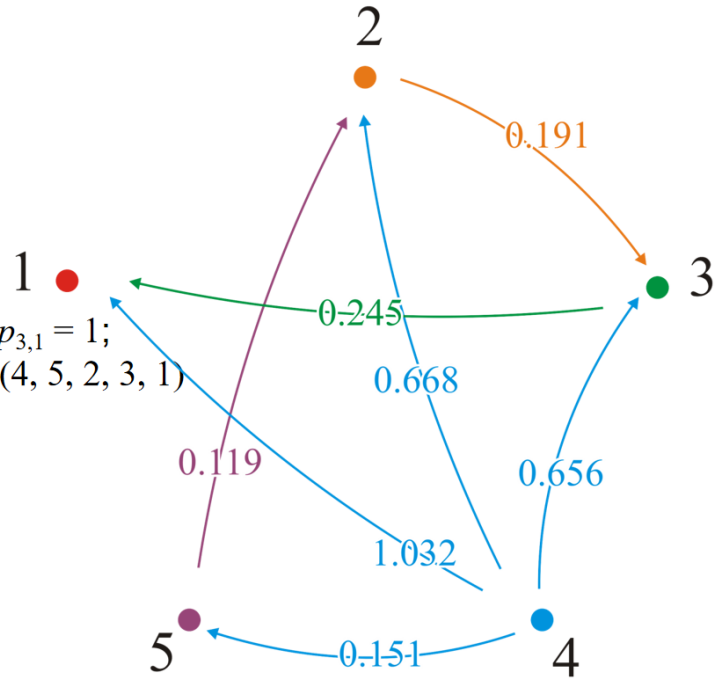
Also, $p_{2,5} = 4$ and $p_{4,5} = 5$;
the shortest path to v_5 is $(2, 4, 5)$



From vertex v_4 , $p_{4,5} = 5$; we go directly to vertex v_5

$$\begin{pmatrix} - & 2 & 3 & 4 & 5 \\ 3 & - & 3 & 4 & 4 \\ 1 & 1 & - & 4 & 4 \\ \color{red}{5} & \color{red}{5} & \color{red}{5} & - & \color{red}{5} \\ 2 & 2 & 2 & 2 & - \end{pmatrix}$$

But $p_{4,1} = 5$, $p_{5,1} = 2$, $p_{2,1} = 3$, $p_{3,1} = 1$;
the shortest path to v_1 is (4, 5, 2, 3, 1)



Which vertices are connected?

最后，如果我们只关心两点之间是否有路径存在呢？那么我们可以仍然利用Floyd-Warshall算法。维护一个连接信息的矩阵，一开始初始化的时候是 $tc[i][j] = true$ 如果有 i 指向 j 的边。那么随着之后的Floyd-Warshall算法，如果对应位置的distance更新，但是 tc 矩阵中对应的是false，那么就更新为true。通过上述的操作，我们就能找到strong connected component。

Painpoint

1. Dijkstra算法产生的生成树不一定是最小生成树
2. 暴力Dijkstra使用adjacency list or matrix不影响最终的时间复杂度，都是 $\Theta(n^2)$
3. 如果是稠密图，暴力Dijkstra反而比使用二叉堆或优先队列的 $\Theta(|E|\log|V|)$ 更好；斐波那契堆方法是最优的
4. 如果有向图，那么Dijkstra算法依然不需要改变
5. Bellman-Ford算法比Dijkstra慢
6. Dijkstra不是在所有的DAG中都是万能的，如果有负数边长的存在就用不了
7. Dijkstra暴力+使用adjacency list的时间复杂度为： $O(|V|^2 + |E|)$
8. Dijkstra's algorithm could only find the shortest path on non-negative weighted graph
9. 通过修改 Dijkstra 算法，在所有最短路径中选择边数最少的路径：给所有的边长都加上 ϵ ，其中 $\epsilon < \delta/|E|$ ，然后运行Dijkstra算法。这是因为在两条路径拥有相同权重的同时，边长数量较少的路径将有较小的 ϵ 综合。其中 δ 代表的是所有边长的最小值。
10. If we add a constraint that each edge can only appear at most once in the shortest path, Dijkstra's algorithm still works for positive-weighted graphs.
11. 'Dijkstra's algorithm cannot work for graph with both positive and negative weights but can work for graph whose weights are all negative' 这个说法是错误的！
12. Bellman-Ford algorithm can be extended to find the negative circle for directed / undirected graphs with negative weights.
13. Dijkstra算法可以处理有环的有向图，只不过是不能处理包含了负数权重边长的图。
14. Dijkstra algorithm can be viewed as a special case of the A* Graph Search algorithm where the heuristic function from any vertex u to the terminal z is $h(u, z) = 0$. Furthermore, if we use heuristic function $h(u) = c$ for any $u \in V$ where c is a positive constant, then the A* search algorithm will be the same with Dijkstra's algorithm.
15. Floyd-Warshall and Bellman-Ford算法都是需要图中没有负环
16. Puzzle问题中的hamming and mahattan距离，以及距离最终状态的真实移动最少步数距离，都是admissible heuristics. (注意：discrete距离并不是admissible的！)
17. 'Suppose the unique shortest path from source s to a vertex t has l edges. It is impossible that we find this shortest path from s to t in less than l iterations.' 这句话是错误的，因为事实上一次迭代是至少会使最短路的边数至少+1.
18. Dijkstra算法和Bellman-Ford算法都是用于找到图中从一个起点到所有其他顶点的最短路径的算法。然而，它们找到的最短路径所组成的树并不一定是最小生成树 (MST)
19. Dijkstra中，If vertex u is marked visited before v , then $\text{dist}[u] \leq \text{dist}[v]$ ；但是在A星算法中，'if vertex u is marked visited before v , then $\text{dist}[u] \leq \text{dist}[v]$, where $\text{dist}[u]$ represents the real distance from start vertex to u '这个说法就是错误的，这是因为其实判断依据是启发式距离，而不是真实距离。换言之：A*算法中使用consistent heuristic function，如果 u 点在 v 之前被标记，那么一定说明： $d(u) + h(u) \leq d(v) + h(v)$
20. For A* search algorithm with admissible and consistent heuristic, heuristic $h_a(x)$ is always better than $h_b(x)$ if $\forall x : h_a(x) \geq h_b(x)$.
21. 如果heuristic是consistent的，那么也必然是admissible，但是反过来不成立

22. Floyd-Warshall使用的不是贪心算法，而是动态规划
23. Dijkstra, Bellman, A*都是只能求出从一个点出发的到所有点的最短路径，而Floyd可以是任意两点之间
24. 如何用Floyd判断是否存在负环？运行完算法后，对于矩阵中对角元素 $dist[i][i]$, $i \in [0, |V| - 1]$ ，如果存在负数，那么就说明有负环；同时Floyd也能判断有向图中任意两个点是否是强连通的： $u, v, d_{u,v} \neq \infty, d_{v,u} \neq \infty$
25. With a simple optimization, applying Floyd-Warshall algorithm to an undirected graph of N nodes just needs to take about half of the time of applying it to a directed graph of N nodes. (Do not need to be strictly 1/2)
26. Floyd算法所消耗的时间仅与顶点数量有关
27. Bellman-Ford算法中，经过k次最外面的循环，则v1到vn的包含至多k条边的最短路径会被计算。这是因为算法经过k次循环，那么从source到每一个点的所有最多含k条边的路径中最短的一条就会被找到。但是其实依次循环使得最短路径长度是至少+1。
28. Floyd寻找path的操作中，如果初始化是 $p[i][j] = j, i \neq j$ ，且在k outer loop过程内， $p[i][j] = p[i][k]$ ，那么路径的确定就如之前note中所示；但是如果初始化是 $p[i][j] = -1$ ，且更新过程中 $p[i][j] = k$ ，那么SOLUTION(u,v,p)，代表寻找u点到v点的最短路径，设计如下：

```
# SOLUTION(u,v,p)
if u==v then
    return {u}
end if
return {u} + FIND-PATH(u,v,p) + {v}

# FIND-PATH(u,v,p)
if p[u][v] == -1
    return {}
end if
k = p[u][v]
return FIND-PATH(u,k,p) + {k} + FIND-PATH(k,v,p)
```

28. Dijkstra & A* pseudocode:

Q is a min-heap storing a tuple: (key, value), and sorted by value. And you have the following operations for Q :

1. **Q.push({u, val})**: put a tuple (u, val) into the heap.
2. **{u, val} = Q.pop()**: get the tuple with minimum value in the heap, and then pop the tuple out of the heap.
3. **Q.update({u, val})**: find the tuple in the heap whose key is 'u', and update its value into 'val'.

Algorithm 1 Single Source Shortest Path Algorithm

- 1: **Input:** Weight map w , min-heap Q , Source node s , heuristic function h .
 - 2: **Output:** The shortest distance from s to all other nodes, and their previous node in the shortest path.
 - 3: **for** $i \leftarrow 0$ **to** V **do**
 - 4: $dist[i] \leftarrow \text{Inf}$
 - 5: $pre[i] \leftarrow \text{NULL}$
 - 6: $Q.\text{push}(\{i, dist[i]\})$
 - 7: **end for**
 - 8: $dist[s] \leftarrow 0$
 - 9: $Q.\text{update}(\{s, 0\})$
 - 10: **while** Q is not empty **do**
 - 11: **Fill this part with your pseudo code**
 - 12: ...
 - 13: **end while**
 - 14: **return** $dist, pre$
-

Dijkstra:

Solution:

```
1 {u, _} = Q.pop()
2 for each neighbor v of u:
3     if dist[v] > dist[u] + w(u, v):
4         dist[v] = dist[u] + w(u, v);
```

```
5         pre[v] = u;
6         Q.update({v, dist[v]})
```

A*:

Solution:

```
1 {u, _} = Q.pop()
2 for each neighbor v of u:
3     if dist[v] > dist[u] + w(u, v):
4         dist[v] = dist[u] + w(u, v);
5         pre[v] = u;
6         Q.update({v, dist[v]+h(v)})
```

29. If we modify the outer loop of Dijkstra's algorithm to execute $(|V| - 1)$ iterations instead of $(|V|)$ iterations (i.e., only pop $(|V| - 1)$ times from the heap), the algorithm can still find the shortest path on a positive-weighted graph.
30. We can modify Bellman-Ford algorithm to detect whether there exists a negative cycle or not in a directed graph.
31. If we modify the outer loop of Bellman-Ford algorithm to execute $(|V|)$ iterations instead of $(|V| - 1)$ iterations (i.e., apply update rule to each edge for $(|V|)$ times), the algorithm can still find the shortest path on a positive-weighted graph.
32. We can modify Floyd-Warshall algorithm to detect whether there exists a negative cycle or not in a directed graph.
33. Floyd-Warshall算法如果被用来只求出single source最短路径，那么依然需要 $\Theta(V^3)$ 的时间复杂度，因为这个算法是绑定产出所有点的最短路径的。
34. Bellman-Ford算法每次循环操作实际上是对相邻节点的访问，**第 n 次循环操作保证了所有深度为 n 的路径最短**。由于图的最短路径最长不会经过超过 $|V| - 1$ 条边，所以可知贝尔曼-福特算法所得为最短路径。加黑斜体的内容是十分关键的！
35. '超级源点'思想十分重要！'双生顶点'手法很重要！在很多时候有奇效。

Dynamic Programming

Introduction

考虑如下函数：

```
double F( int n ) {
    return ( n <= 1 ) ? 1.0 : F(n - 1) + F(n - 2);
}
```

它的运行时间复杂度应该和斐波那契数列非常接近： $T(n) = O(2^n)$ 。问题在于，为了计算，比如说， $F(44)$ 的时候，我们需要计算得到 $F(43)F(42)$ ，但是 $F(43)$ 的计算过程中又包含了 $F(42)$ 。事实上这个问题十分严重，比如说这个例子中， $F(10)$ 被计算了9227465次。这完全是多余的。

一种解决方法是将所有的中间计算过程都储存在表格中。把中间结果储存起来的过程叫做**memorization**。把中间计算结果储存起来是为了后续可能的使用，这样就在需要这个数据的时候不需要重新计算了。

因此：Dynamic programming is actually a fancy name for caching intermediate results in a table for later use.

In solving optimization problems, the top-down approach may require repeatedly obtaining optimal solutions for the same sub-problem

动态规划和分治是不同的，分支中子问题都实际上应该是unique的，但是在动态规划中发生的是：一个问题被分解成多个小问题（子问题），这些子问题可能会被多次遇到。如果子问题再次发生，那么这个问题就称为重叠子问题。

因此如下总结贪心、分治和动态规划三者的概念：

Greedy. Process the input in some order, myopically making irrevocable decisions.

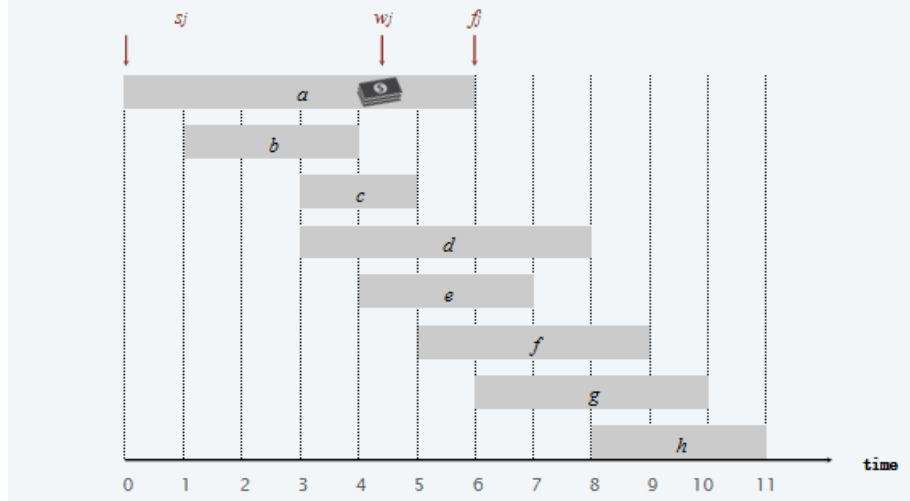
Divide-and-conquer. Break up a problem into independent subproblems; solve each subproblem; combine solutions to subproblems to form solution to original problem.

Dynamic programming. Break up a problem into a series of overlapping subproblems; combine solutions to smaller subproblems to form solution to large subproblem.

Weighted Interval Scheduling

考虑如下的任务：

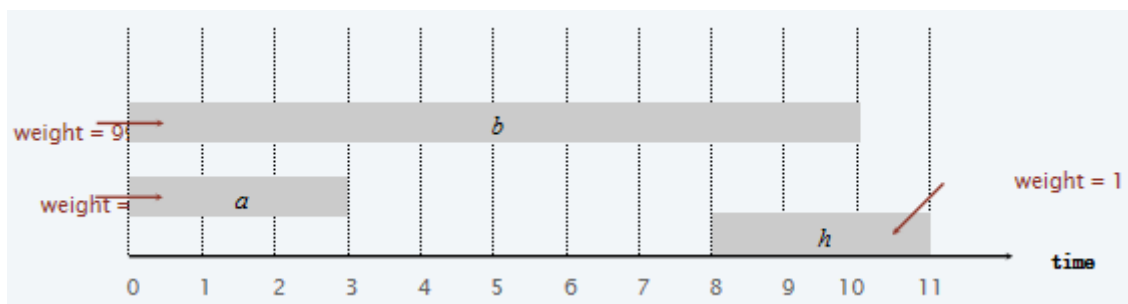
- Job j starts at s_j , finishes at f_j , and has weight $w_j > 0$.
- Two jobs are **compatible** if they don't overlap.
- Goal: find max-weight subset of mutually compatible jobs.



注意到这个任务貌似和贪心中介绍的任务很像，换言之，上述图片的任务中如果所有的task权重都是一样的，那么贪心就应该是最优的。

Earliest finish-time first: Consider jobs in ascending order of finish time. Add job to subset if it is compatible with previously chosen jobs.

但是能够观察到，如果权重不相同呢？那么贪心算法就不是最优的。下面是一个非常简单的反例：



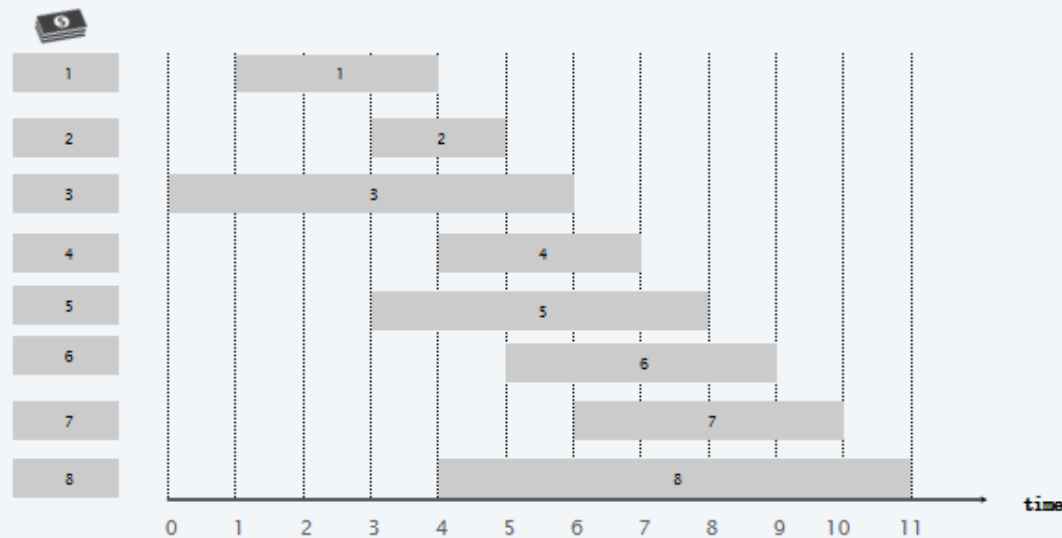
接下来介绍一种新的解决方式，首先定义一些内容：

Convention. Jobs are in ascending order of finish time: $f_1 \leq f_2 \leq \dots \leq f_n$.

Def. $p(j)$ = largest index $i < j$ such that job i is compatible with j .

Ex. $p(8) = 1, p(7) = 3, p(2) = 0$.

i is leftmost interval that ends before j begins



Def. $OPT(j)$ = max weight of any subset of mutually compatible jobs for subproblem consisting only of jobs 1, 2, ..., j .

Goal. $OPT(n)$ = max weight of any subset of mutually compatible jobs.

Case 1. $OPT(j)$ does not select job j .

- Must be an optimal solution to problem consisting of remaining jobs 1, 2, ..., $j - 1$.

Case 2. $OPT(j)$ selects job j .

- Collect profit w_j .
- Can't use incompatible jobs $\{p(j) + 1, p(j) + 2, \dots, j - 1\}$.
- Must include optimal solution to problem consisting of remaining compatible jobs 1, 2, ..., $p(j)$.

optimal substructure property
(proof via exchange argument)

Bellman equation.
$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max \{ OPT(j - 1), w_j + OPT(p(j)) \} & \text{if } j > 0 \end{cases}$$

首先定义的是 $p(j)$ 代表的是j任务之前第一个能和j兼容的任务的index, 然后定义了 $OPT(j)$ 代表使用1, 2, ..., j号任务的所获得的权重最大值。因此顺势展开分类: 如果j号任务是需要的, 那么就说明之前所有任务中能够兼容的任务的最大编号为 $p(j)$, i.e., $p(j) + 1, \dots, j - 1$ 这些编号的任务都是用不上了的, 相当于是子问题是 $OPT(p(j))$; 而如果j没用到, 那么就说明子问题是 $OPT(j - 1)$ 。因此对于 $OPT(j)$ 来说, 有两种可能性, 那么因此它的值应该是两种scenario中的最大值, 因此Bellman Equation如上图所示。

因此可以设计如下伪代码:

```

Brute-Force (n, s1, ..., sn, f1, ..., fn, w1, ..., wn)
Sort jobs by finish time and renumber so that  $f_1 \leq f_2 \leq \dots \leq f_n$ .
Compute  $p[1], p[2], \dots, p[n]$  via binary search.
Return Compute-Opt(n).

Compute-Opt(j)
If (j = 0)
    return 0.
Else
    Return  $\max \{ \text{Compute-Opt}(j-1), w_j + \text{Compute-Opt}(p[j]) \}$ .

```

对于上面这个recursive algorithm来说很慢，因为有overlapping problem，因此时间复杂度成指数增长趋势。但是我们可以利用memoriation来避免重复的计算。考虑用数组来储存子问题的结果：

Top-down dynamic programming (memoization).

- Cache result of subproblem j in $M[j]$.
- Use $M[j]$ to avoid solving subproblem j more than once.

TOP-DOWN($n, s_1, \dots, s_n, f_1, \dots, f_n, w_1, \dots, w_n$)

Sort jobs by finish time and renumber so that $f_1 \leq f_2 \leq \dots \leq f_n$.

Compute $p[1], p[2], \dots, p[n]$ via binary search.

$M[0] \leftarrow 0$. —— global array

RETURN M-COMPUTE-OPT(n).

M-COMPUTE-OPT(j)

IF ($M[j]$ is uninitialized)

$M[j] \leftarrow \max \{ \text{M-COMPUTE-OPT}(j-1), w_j + \text{M-COMPUTE-OPT}(p[j]) \}$.

RETURN $M[j]$.

在函数中判断如果M[j]没有被初始化过，说明是递归的底层开始第一次真正计算了这个值，那么这个值就储存起来，而且不会再更改。之后遇到浅层的递归需要这个值的时候，直接访问数组即可。那么现在时间复杂度是多少呢？sort by finish time的时间复杂度是 $\Theta(n \log n)$ ，计算 $p[j]$ ， $j \in [1, n]$ 的时间复杂度是 $O(n \log n)$ （使用二分搜索），然后在M-COMPUTE-OPT中，实际上只有第一次常数时间内计算M[j]的时候才纳入时间，储存起来后能够常数时间内被调出然后使用， $j \in [1, n]$ ，所以时间复杂度是 $O(n)$ 。综上所述，总的时间复杂度是 $O(n \log n)$

那么在最终求出最优解的时候，实现下列的函数即可，因为实际上就是找出一系列的任务编号：

FIND-SOLUTION(j)

IF ($j = 0$)

RETURN $\emptyset$.

ELSE IF ($w_j + M[p[j]] > M[j-1]$)

RETURN $\{j\} \cup \text{FIND-SOLUTION}(p[j])$.

ELSE

RETURN FIND-SOLUTION($j-1$).

$M[j] = \max \{ M[j-1], w_j + M[p[j]] \}$.

由于每一次判断时间是常数，而最多判断 n 次，所以时间复杂度是 $O(n)$ 。在上述的实现中，M-COMPUTE-OPT采用了top-down递归的形式。但是其实也可以采用bottom-up的方式来实现：

BOTTOM-UP($n, s_1, \dots, s_n, f_1, \dots, f_n, w_1, \dots, w_n$)

Sort jobs by finish time and renumber so that $f_1 \leq f_2 \leq \dots \leq f_n$.

Compute $p[1], p[2], \dots, p[n]$.

$M[0] \leftarrow 0$.

FOR $j = 1$ **TO** n

$M[j] \leftarrow \max \{ M[j-1], w_j + M[p[j]] \}.$

previously computed values

上述算法的时间复杂度依然是 $O(n \log n)$ 。因此这个任务成功完成了。非常好的是，我们不需要证明这个DP算法的正确性，因为**top-down**就是一个合理的算法解释，而bottom-up是一个实现方法，而top-down and bottom-up是等效的。

Segmented Least Squares

Least Squares是什么意思呢？假设平面上有 n 个点，然后拟合一条直线，计算MSELoss，换言之也就是：

$$SSE = \sum_{i=1}^n (y_i - ax_i - b)^2$$

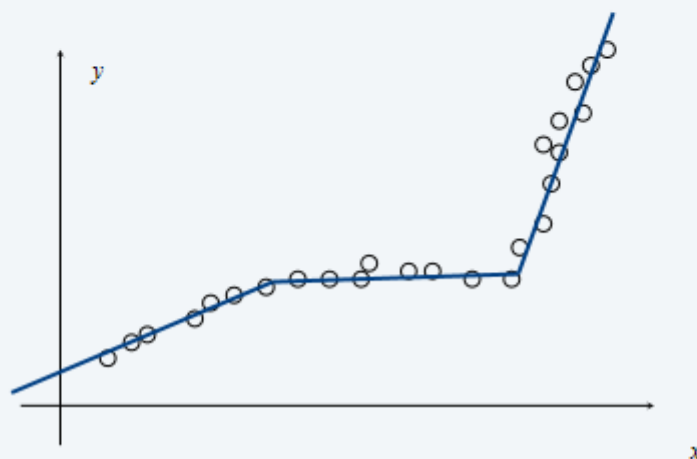
数学中有公式能求出使得SSE最少的参数 a ， b 。但是上述内容的假设是点的分布十分接近于一条直线的。那么如果：Points lie roughly on a sequence of several line segments，任务是find a sequence of lines that minimize $f(x)$ 。

Segmented least squares.

- Points lie roughly on a sequence of several line segments.
- Given n points in the plane: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ with $x_1 < x_2 < \dots < x_n$, find a sequence of lines that minimizes $f(x)$.

Goal. Minimize $f(x) = E + cL$ for some constant $c > 0$, where

- E = sum of the sums of the squared errors in each segment.
- L = number of lines.



那么如何设计DP呢？假设 $OPT(j)$ 代表的是对于 $1-j$ 个点的最小的cost，那么如果考虑 $i \leq j$ ， $p_i, \dots, p_j$ 是最后一条线段拟合的点，因此就能够计算cost之间的关系了：

$$OPT(j) = e_{ij} + c + OPT(i-1), i \leq j$$

$$\text{where } e_{ij} = SSE(p_i, \dots, p_j)$$

那么在设计Bellman equation的时候， i 可以任取 $[1, j]$ 里面的数字，因此方程设计如下：

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \min_{1 \leq i \leq j} \{ e_{ij} + c + OPT(i-1) \} & \text{if } j > 0 \end{cases}$$

Theorem. [Bellman 1961] DP algorithm solves the segmented least squares problem in $O(n^3)$ time and $O(n^2)$ space.

Pf.

- Bottleneck = computing SSE e_{ij} for each i and j .

$$a_{ij} = \frac{n \sum_k x_k y_k - (\sum_k x_k)(\sum_k y_k)}{n \sum_k x_k^2 - (\sum_k x_k)^2}, \quad b_{ij} = \frac{\sum_k y_k - a_{ij} \sum_k x_k}{n}$$

- $O(n)$ to compute e_{ij} . ■

Remark. Can be improved to $O(n^2)$ time.

- For each i : precompute cumulative sums $\sum_{k=1}^i x_k$, $\sum_{k=1}^i y_k$, $\sum_{k=1}^i x_k^2$, $\sum_{k=1}^i x_k y_k$.
- Using cumulative sums, can compute e_{ij} in $O(1)$ time.

Knapsack Problem

考虑下述的问题:

Goal. Pack knapsack so as to maximize total value.

- There are n items: item i provides value $v_i > 0$ and weighs $w_i > 0$.
- Knapsack has weight capacity of W .

Assumption. All input values are integral.

Ex. { 1, 2, 5 } has value \$35 (and weight 10).

Ex. { 3, 4 } has value \$40 (and weight 11).



Creative Commons Attribution-Share Alike 2.5
by Dake

i	v_i	w_i
1	US\$1	1 kg
2	US\$6	2 kg
3	US\$18	5 kg
4	US\$22	6 kg
5	US\$28	7 kg

knapsack instance
(weight limit $W = 11$)

Def. $OPT(i)$ = max-profit subset of items $1, \dots, i$.

Goal. $OPT(n)$.

Case 1. $OPT(i)$ does not select item i .

- OPT selects best of $\{1, 2, \dots, i-1\}$.

Case 2. $OPT(i)$ selects item i .

- Selecting item i does not immediately imply that we will have to reject other items.
- Without knowing which other items were selected before i , we don't even know if we have enough room for i .

optimal substructure property
(proof via exchange argument)

Conclusion. Need more subproblems!

但是此处有限重的要求，因此设计的OPT应该带上weight信息：

Def. $OPT(i, w)$ = max-profit subset of items $1, \dots, i$ with weight limit w .

Goal. $OPT(n, W)$.

Case 1. $OPT(i, w)$ does not select item i .

- $OPT(i, w)$ selects best of $\{1, 2, \dots, i-1\}$ using weight limit w .

possibly because $w_i > w$

Case 2. $OPT(i, w)$ selects item i .

- Collect value v_i .
- New weight limit = $w - w_i$.
- $OPT(i, w)$ selects best of $\{1, 2, \dots, i-1\}$ using this new weight limit.

optimal substructure property
(proof via exchange argument)

Bellman equation.

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

首先考虑的是在 $OPT(i, w)$ 中， i 物品是否选入，这可以分为两种情况：如果选入，那么 $OPT(i, w) = OPT(i-1, w - w_i) + v_i$ ；如果不选入，那么 $OPT(i, w) = OPT(i-1, w)$ ；至于两种情况中的哪一种，那么就是直接考虑max了。当然还需要考虑：如果 $w_i > w$ ，那么显然 $OPT(i, w) = OPT(i-1, w)$

可见，这里储存的结果应该是和两个变量有关，因此应该储存在二维数组中；同时从代码实现上，外面的循环从subset of items从小到大开始，每一次循环中进行 w 的从小到大的一次for循环。

KNAPSACK($n, W, w_1, \dots, w_n, v_1, \dots, v_n$)

FOR $w = 0$ TO W

$M[0, w] \leftarrow 0$.

FOR $i = 1$ TO n

FOR $w = 0$ TO W

IF ($w_i > w$) $M[i, w] \leftarrow M[i-1, w]$.

ELSE $M[i, w] \leftarrow \max \{ M[i-1, w], v_i + M[i-1, w - w_i] \}$.

RETURN $M[n, W]$.

previously computed values



那么时间复杂度和空间复杂度都显然是: $\Theta(nw)$

一定要注意背包问题中，物品是否无限？还是每件只能拿一个？或者是重量是正整数吗？还是正实数！（正实数就是NPC问题了）

More Examples

House Coloring Problem

Goal. Paint a row of n houses red, green, or blue so that

- No two adjacent houses have the same color.
- Minimize total cost, where $cost(i, color)$ is cost to paint i given color.



	A	B	C	D	E	F
Red	7	6	7	8	9	20
Green	3	8	9	22	12	8
Blue	16	10	4	2	5	7

Subproblems.

- $R[i] = \text{min cost to paint houses } 1, \dots, i \text{ with } i \text{ red.}$
- $G[i] = \text{min cost to paint houses } 1, \dots, i \text{ with } i \text{ green.}$
- $B[i] = \text{min cost to paint houses } 1, \dots, i \text{ with } i \text{ blue.}$
- Optimal cost = $\min \{ R[n], G[n], B[n] \}.$

Dynamic programming equation.

- $R[i+1] = \text{cost}(i+1, \text{red}) + \min \{ B[i], G[i] \}$
 - $G[i+1] = \text{cost}(i+1, \text{green}) + \min \{ R[i], B[i] \}$
 - $B[i+1] = \text{cost}(i+1, \text{blue}) + \min \{ R[i], G[i] \}$
- overlapping subproblems

Running time. $O(n).$

Maximum Subarray Problem

Goal: Given an array x of n integer (positive or negative), find a contiguous subarray whose sum is maximum.

定义: $OPT(i) = \text{max sum of any subarray of } x \text{ whose rightmost index is } i.$

Bellman equation.
$$OPT(i) = \begin{cases} x_1 & \text{if } i = 1 \\ \max \{ x_i, x_i + OPT(i-1) \} & \text{if } i > 1 \end{cases}$$

Running time. $O(n).$

↑
take only
element i
↘
take element i
together with best subarray
ending at index $i-1$

对于 $OPT(i)$ 来说, 如果 $OPT(i-1)$ 是一个正数, 那么将 x_i 加入这个subarray中非常nice; 但是如果是负数呢? 那么还不如不加这个subarray, 因为 x_i 这个数字根据 $OPT(i)$ 的定义是不得不加的, 那么还不如这个负数不要加上。

Maximum Rectangle Problem

Goal. Given an n -by- n matrix A , find a rectangle whose sum is maximum.

$$A = \begin{bmatrix} -2 & 5 & 0 & -5 & -2 & 2 & -3 \\ 4 & -3 & -1 & 3 & 2 & 1 & -1 \\ -5 & 6 & 3 & -5 & -1 & -4 & -2 \\ -1 & -1 & 3 & -1 & 4 & 1 & 1 \\ 3 & -3 & 2 & 0 & 3 & -3 & -2 \\ -2 & 1 & -2 & 1 & 1 & 3 & -1 \\ 2 & -4 & 0 & 1 & 0 & -3 & -1 \end{bmatrix}$$

13

Applications. Databases, image processing, maximum likelihood estimation, technical job interviews, ...

Assumption. Suppose you knew the left and right column indices j and j' .

$$A = \begin{bmatrix} -2 & 5 & 0 & -5 & -2 & 2 & -3 \\ 4 & -3 & -1 & 3 & 2 & 1 & -1 \\ -5 & 6 & 3 & -5 & -1 & -4 & -2 \\ -1 & -1 & 3 & -1 & 4 & 1 & 1 \\ 3 & -3 & 2 & 0 & 3 & -3 & -2 \\ -2 & 1 & -2 & 1 & 1 & 3 & -1 \\ 2 & -4 & 0 & 1 & 0 & -3 & -1 \end{bmatrix} \quad x = \begin{bmatrix} -7 \\ 4 \\ -3 \\ 6 \\ 5 \\ 0 \\ 1 \end{bmatrix}$$

$0 - 5 - 2$
 solve maximum subarray problem in this array

An $O(n^3)$ algorithm.

- Precompute cumulative row sums $S_{ij} = \sum_{k=1}^j A_{ik}$.
- For each $j < j'$:
 - define array x using row-sum differences: $x_i = S_{ij'} - S_{ij}$
 - run Kadane's algorithm in array x

Open problem. $O(n^{3-\epsilon})$ for any constant $\epsilon > 0$.

Coin Changing Problem

Problem. Given n coin denominations $c_1, c_2, \dots, c_n$ and a target value V , find the fewest coins needed to make change for V (or report impossible).

Def. $OPT(v)$ = min number of coins to make change for v .

Goal. $OPT(V)$.

Multiway choice. To compute $OPT(v)$,

- Select a coin of denomination c_i for some i .
- Select fewest coins to make change for $v - c_i$.

optimal substructure property
(proof via exchange argument)

Bellman equation.

$$OPT(v) = \begin{cases} \infty & \text{if } v < 0 \\ 0 & \text{if } v = 0 \\ \min_{1 \leq i \leq n} \{ 1 + OPT(v - c_i) \} & \text{otherwise} \end{cases}$$

Running time. $O(nV)$.

CS101常见题型

CS101会设计三类DP设计模型:

- 前缀: $f(i)$ 表示前 i 个东西/以第 i 个东西结尾的...
- 背包: $f(i, j)$ 表示前 i 个东西, 某属性之和... j
- 区间: $f(l, r)$ 表示下标区间为 $[l, r]$ 中的这几样东西...

值得注意的是, 后缀也是可能考的! 如21年Fall-Final的题:

You have n lectures numbered $1, 2, \dots, n$ in the time order. Attending lecture i will spend you t_i time. You may not be able to take all the lectures, because after you attend lecture i and spend t_i time on it, you will feel very tired and cannot continue to take any of the following y_i lectures after lecture i . For example, if you take the i -th lecture with $y_i = 2$, you will miss lecture $i + 1$ and lecture $i + 2$.

However, as a hardworking ShanghaiTech student, you want to spend more time on lectures, i.e. you want to maximize the total time you can spend on those lectures.

Suppose that you are given the t_i and y_i values for all n lectures as input. Design a **dynamic programming** algorithm to find the maximum total time you can spend on all the lectures.

如果考虑的是 $dp[i]$ 代表前 i 节课所花的最多的时间，那么由于考虑的是 $j < i$ 如果上的话，将会有 y_j 的疲劳，因此 $j + y_j < i$ ，符合条件的 j 将会是转移的状态，而且要对所有的可能状态取 $\max$ 。这样太麻烦了。

由于疲劳时间是作用向后的，因此可以考虑倒过来考虑： $dp[i]$ 代表从倒 n 这些课程所花的最长时间，那么考虑第 i 节课，如果上的话，那么转移的状态是 $dp[i + y_i + 1]$ ，而且不需要什么取 $\max$ 操作，非常的方便。时间复杂度是 $O(n)$ 。

Reductions

Cook (Turing) Reduction

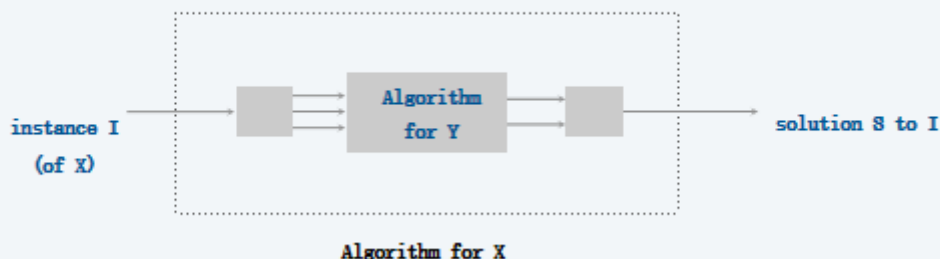
在实践中什么样的问题我们认为是“可解决的”呢？在工程中一般认为：能够在多项式时间内实现，那么认为问题是“可解决的”。因此将问题按照能否在多项式时间内解决为标准进行分类。可惜的是，当今大量的基础问题时间复杂度都超出了多项式时间范畴内。

那么假设我们可以在多项式时间内解决 Y 问题，那么根据这个信息，还有哪些问题能够在多项式时间内结局？因此就有 Reduction Theorem：

Reduction. Problem X polynomial-time (Cook) reduces to problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y .

computational model supplemented by special piece of hardware that solves instances of Y in a single step



如果 X 问题的标准计算步骤是多项式时间级别的，而且调用 Y 算法的次数也是多项式级别的，那么问题 X 也是多项式级别的。这个就是 Cook Reduction。Notation 记为：

$$X \leq_P^T Y$$

Karp (Many-One) Reduction

除了 Cook Reduction，还有一种说明 Reduction 的方法：Karp Reduction。其实 Karp 更像是一种 Transform：

Problem X polynomial transforms (Karp) (polynomial-time many-one reduces) to problem Y if we can construct a mapping function f that maps instances of X to instances of Y such that given any instance x of X , the instance $y = f(x)$ of Y is a yes instance **iff** x is a yes instance of X .

Notation 为： $X \leq_m^P Y$ ，或者是简化版本的： $X \leq_P Y$ 。根据上述的定义，我们可以得知如何证明 X 可以 Karp 归约到 Y ：设计 mapping，说明如果 x 是 X 的 yes，则 $f(x)$ 是 Y 的 yes；如果 $f(x)$ 是 Y 的 yes，则 x 是 X 的 yes。

值得注意的是： $X \leq_P Y$ 和 $Y \leq_P X$ 是不一样的东西。该记号可以这么理解： **X is no harder than Y** 。

那么因此对于 XY 两个问题：如果 $X \leq_P Y$ ， Y 可以多项式时间内解决，那么 X 也能多项式时间内解决，但是**反之不亦然**；同时，如果 X 无法在多项式时间内解决，那么 Y 也无法在多项式时间内解决。那么如果 $X \leq_P Y$ ，且 $Y \leq_P X$ ，那么记为：

$$X \equiv_P Y$$

那么这种情况下： X 可以在多项式时间内解决**当且仅当** Y 能够在多项式时间内解决。从上述的一系列 Reduction 相关定义可以看出：Reductions classify problems according to **relative** difficulty.

在本章的内容中，几乎所有的归约都指的是Karp归约。

Example1

对于一个图来说，给定一个规定的正整数 k ，能不能找到顶点集的一个含 k 个（或者更多）顶点的子集，使得子集中的每两个点之间都是不相连的？这个问题称为**Independent-Set**问题。

对于一个图来说，给定一个规定的正整数 k ，能不能找到顶点集的一个含有 k 个（或者更少）顶点的子集，使得图中每一条边的两个顶点中至少有一个在子集里面。这个问题称为**Vertex-Cover**问题。

那么就有如下的定理：

$$\text{Theorem : Independent - Set} \equiv_p \text{Vertex - Cover}$$

Pf. We show S is an independent set of size k iff $V - S$ is a vertex cover of size $n - k$.

$\Rightarrow$

- Let S be any independent set of size k .
- $V - S$ is of size $n - k$.
- Consider an arbitrary edge $(u, v) \in E$.
- S independent $\Rightarrow$ either $u \notin S$, or $v \notin S$, or both.
 $\Rightarrow$ either $u \in V - S$, or $v \in V - S$, or both.
- Thus, $V - S$ covers (u, v) . ▀

$\Leftarrow$

- Let $V - S$ be any vertex cover of size $n - k$.
- S is of size k .
- Consider an arbitrary edge $(u, v) \in E$.
- $V - S$ is a vertex cover $\Rightarrow$ either $u \in V - S$, or $v \in V - S$, or both.
 $\Rightarrow$ either $u \notin S$, or $v \notin S$, or both.
- Thus, S is an independent set. ▀

考虑另一个新问题：给定一个含有 U 个元素的集合，且给定 S 个 U 的子集，和一个正整数 k ，是否有 $\leq k$ 个子集的Union等于 U ？则有如下定理：

$$\text{Theorem : Vertex - Cover} \leq_p \text{Set - Cover}$$

Pf. Given a VERTEX-COVER instance $G = (V, E)$ and k , we construct a SET-COVER instance (U, S, k) that has a set cover of size k iff G has a vertex cover of size k .

Construction.

- Universe $U = E$.
- Include one subset for each node $v \in V$: $S_v = \{e \in E : e \text{ incident to } v\}$.

Pf. $\Rightarrow$ Let $X \subseteq V$ be a vertex cover of size k in G .

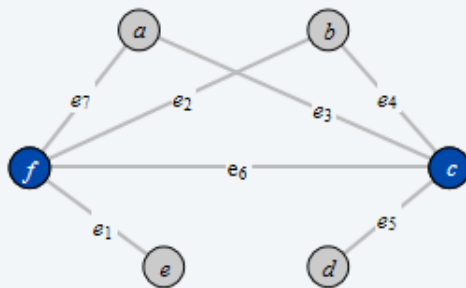
- Then $Y = \{S_v : v \in X\}$ is a set cover of size k . ▀

“yes” instances of VERTEX-COVER are solved correctly

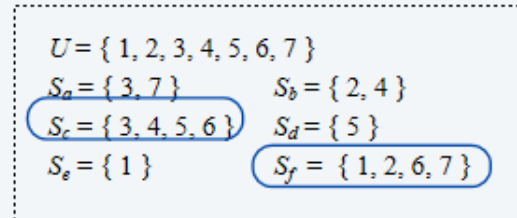
Pf. $\Leftarrow$ Let $Y \subseteq S$ be a set cover of size k in (U, S, k) .

- Then $X = \{v : S_v \in Y\}$ is a vertex cover of size k in G . ■

"no" instances of VERTEX-COVER are solved correctly



vertex cover instance
($k = 2$)



set cover instance
($k = 2$)

22

Example2

现在将介绍一类新的问题：3-SAT问题。一个布尔变量和它的否定 (negation) 为： $x_i \bar{x}_i$ 。同时定义命题的合取范式 (CNF: Conjunctive Normal Form)：

$$\Phi = C_1 \wedge C_2 \cdots \wedge C_n$$

C is called a clause: $C_j = x_1 \vee \bar{x}_2 \vee \cdots \vee x_n$

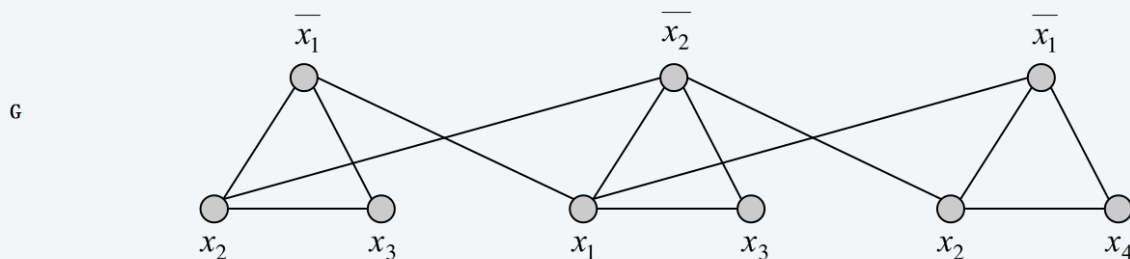
那么考虑一个CNF，是否存在一组clause，i.e.，存在一组boolean variable使得这个CNF为真？这就称为SAT问题。那么**3-SAT**问题就是在SAT的基础之上加上条件：each clause contains exactly 3 literals。那么对于3-SAT问题找到一组使CNF为真的布尔变量，时间复杂度是多少？很可惜的是：

Scientific hypothesis: There does not exist a poly-time algorithm for 3-SAT.

Theorem : $3 - SAT \leq_p Independent - Set$

Construction.

- G contains 3 nodes for each clause, one for each literal.
- Connect 3 literals in a clause in a triangle.
- Connect literal to each of its negations.



$k = 3$

$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$

那么假设有一组assignment使得CNF为真，那么这些布尔变量带入到上图中所构建的graph中，所有的true顶点就构成了一个independent set。可见这个set的元素数量至少是 $|\Phi|$ 。同时考虑构建size为 k 的independent set，如果两个顶点都选在了一个三角形里面，那么就不符合，可见每一个clause其实都只有一项能够为true，那么这个assignment其实就是一种分配。通过上述的说明，我们其实说明了：

Lemma: Φ is satisfiable iff G contains an independent set of size $k = |\Phi|$

通过example 1 and 2，利用reduction的transitivity性质，其实我们能够得知：

Theorem : $3 - SAT \leq_p Independent - Set \equiv_p Vertex - Cover \leq_p Set - Cover$

Example3

还是考虑Vertex-Cover问题：原问题中说的是是否存在一个vertex cover of size less than k。那么如果对于以下问题呢：

- **Search Problem**(Find-Vertex-Cover): Find a vertex cover of size less than k
- **Optimization Problem**(Find-Min-Vertex-Cover): Find a vertex cover of minimum size

同时，原本的问题定义为Decision Problem。那么有如下的定律：

$$Vertex - Cover \equiv_p Find - Vertex - Cover \equiv_p Find - Min - Vertex - Cover$$

Theorem. $VERTEX-COVER \equiv_p FIND-VERTEX-COVER.$

Pf. $\leq_p$ Decision problem is a special case of search problem. ■

Pf. $\geq_p$

To find a vertex cover of size $\leq k$:

- Determine if there exists a vertex cover of size $\leq k$.
- Find a vertex v such that $G - \{v\}$ has a vertex cover of size $\leq k - 1$.
(any vertex in any vertex cover of size $\leq k$ will have this property)
- Include v in the vertex cover.
- Recursively find a vertex cover of size $\leq k - 1$ in $G - \{v\}$. ■

delete v and all incident edges

FIND-VERTEX-COVER. Find a vertex cover of size $\leq k$.

FIND-MIN-VERTEX-COVER. Find a vertex cover of minimum size.

Theorem. $FIND-VERTEX-COVER \equiv_p FIND-MIN-VERTEX-COVER.$

Pf. $\leq_p$ Search problem is a special case of optimization problem. ■

Pf. $\geq_p$ To find vertex cover of minimum size:

- Binary search (or linear search) for size k^* of min vertex cover.
- Solve search problem for given k^* . ■

Example4

3-Color Problem: 有一个无向图G，能够找到一种给所有顶点的染色安排，使得任意两个相邻的顶点都没有相同的颜色？这个问题就是三色问题。

当然之前是见过两色问题的，解决方式就是借助运行BFS or DFS算法来进行染色，因为运行graph search其实就是对一个顶点的所有的unvisited neighbor进行访问。

时间复杂度是 $O(|E| + |V|)$

Use a breadth-first traversal for a connected graph:

- Choose a vertex, mark it belonging to V_1 and push it onto a queue
- While the queue is not empty, pop the front vertex v and
 - Any adjacent vertices that are already marked must belong to the set not containing v , otherwise, the graph is not bipartite (we are done);
 - Any unmarked adjacent vertices are marked as belonging to the other set and they are pushed onto the queue
- If the queue is empty, the graph is bipartite

第一次入栈的用红色，第二次的是蓝色，第三次的是红色，以此类推；然后弹栈的过程中，这个元素周围的元素，应该要么是 unvisited，要么是另外一个颜色的，i.e., 不能和自己的颜色相同；一旦发现了反例，那么立马直到这个元素不是二分图。

K-Register-Allocation Problem: Assign program variables to machine registers so that no more than k registers are used and no two program variables that are needed at the same time are assigned to the same register.

将程序变量分配给机器寄存器，使得使用的寄存器不超过 k 个，并且没有两个在同一时间需要的程序变量被分配到同一个寄存器。

另外定义：**Interference Graph**: Nodes are program variables; edge between u and v if there exists an operation where both u and v are “live” at the same time.

干扰图：节点是程序变量；如果存在一个操作，其中 u 和 v 同时“活跃”，则在 u 和 v 之间有一条边。

因此有以下的事实：

Theorem : Can solve register allocation problem iff interference graph is k -colorable.

$$3 - \text{Color} \leq_p K - \text{Register} - \text{Allocation}, \text{any } k \geq 3$$

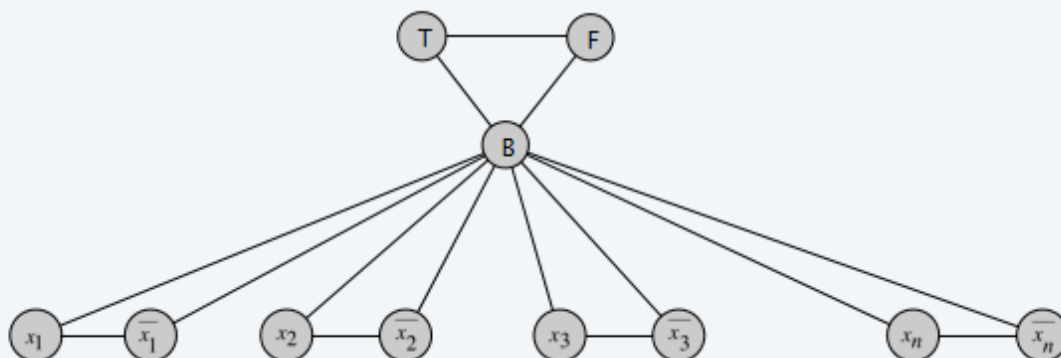
同时也有定理： $3 - \text{SAT} \leq_p 3 - \text{Color}$ 。为了证明它，需要考虑构建两者之间的关系：对于3-SAT问题的解，如何构建到3色问题中的结构，说明3色问题有解当且仅当3-SAT问题解被满足。

Construction.

- Create a graph G with a node for each literal.
- Connect each literal to its negation.
- Create 3 new nodes T , F , and B ; connect them in a triangle.
- Connect each literal to B .
- For each clause C_j , add a gadget of 6 nodes and 13 edges.



to be described later



下面先说明：假设 G 图是3-colorable的。如果图是3-colorable的，那么不失广泛性地，假设节点 T 的颜色是黑色， F 是白色， B 是蓝色。考虑给所有的assignment赋值：所有的true都是黑色，所有的false都是白色。这样保证了以下的性质：

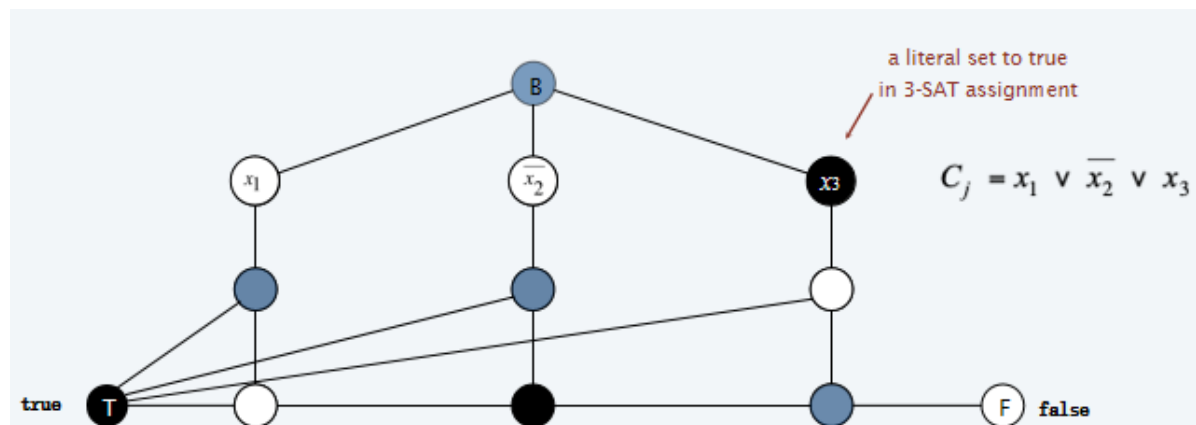
- ensures each literal is colored either black or white.
- ensures that each literal is white if its negation is black (and vice versa).
- ensures at least one literal in each clause is black.

suppose, for the sake of contradiction,
that all 3 literals are white in some 3-coloring

$$C_j = x_1 \vee \bar{x}_2 \vee x_3$$

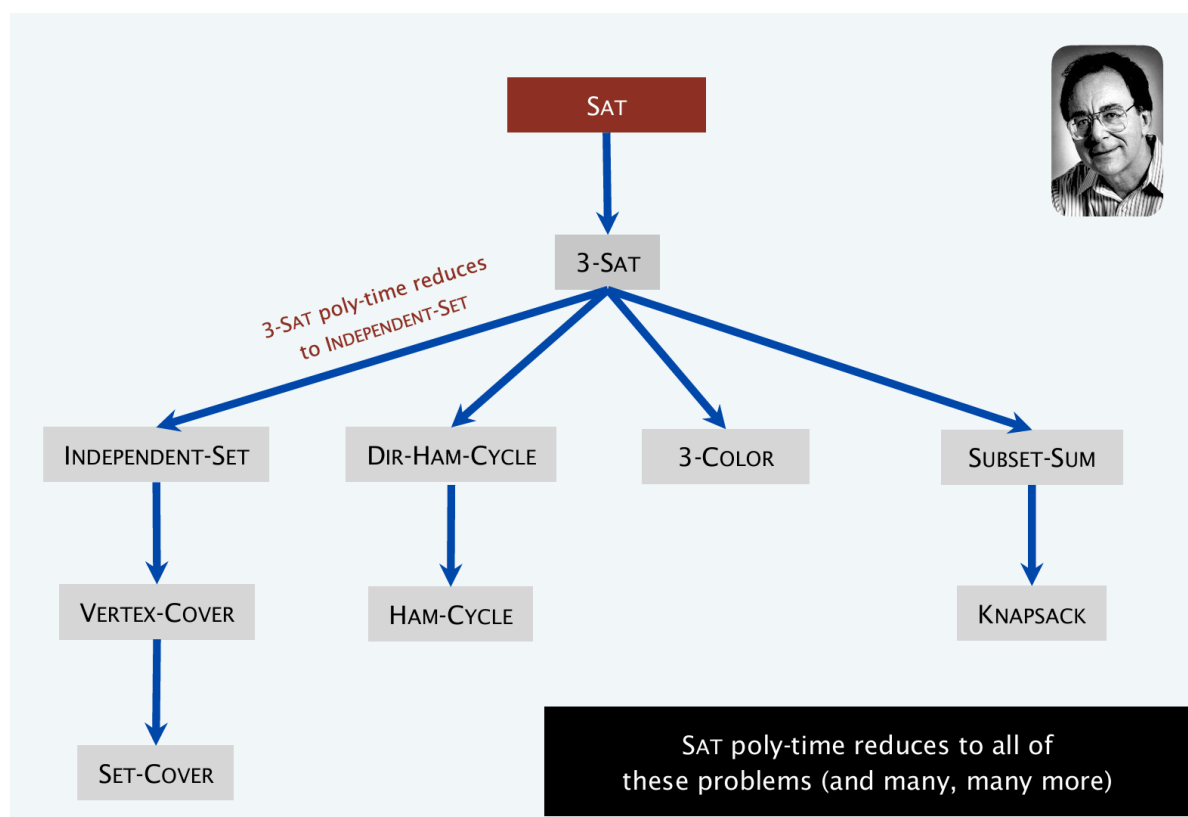
contradiction
(not a 3-coloring)

true **T** **F** false



通过上述的说明，最终证明了: **Lemma: Graph G is 3-colorable iff Φ is satisfiable.**

在上述的例子中，介绍了很多问题，也说明了问题之间的归约关系，如下头，箭头指向代表‘归约到’的关系。



P vs. NP

Definition

PNP也许之前听说过，但是具体的定义是什么呢？P代表的是polynomial，多项式，因此定义如下：

Def: A polynomial time (polytime) algorithm is one that runs in $O(n^k)$ time, for some constant k , when input has size n .

Def: P is the set of all problems that can be solved by a polytime algorithm.

注意！这里的时间复杂度的 n 是输入的size！跟输出的range没有任何关系！

关于P的定义是很容易理解的，那么NP的定义是什么呢？是指的是Non-Polytime吗？实则不然。课程中先前的很多算法都是多项式时间的算法，即对于规模为 n 的输入，在最坏的情况下运行时间都是 $O(n^k)$ 。但是不是所有的问题都可以在多项式时间内解决。一般认为，就如之前所说的那样，多项式时间内可接的问题都是易处理的问题，在超出多项式时间内解决的问题都是不易处理的问题。那么NP类问题指的是什么呢？

NP类问题是指那些在多项式时间内可以被证明的问题。那么‘可被证明’是什么意思？即如果已知一个问题解的证书（certificate），那么可以证明（verifier / **certifier** algorithm）此问题中在该输入规模下能在多项式时间内解决。事实上，verifier并不是在找到问题的解，而仅仅是检查已知的问题解。

简而言之，就是对于NP问题，就是能在多项式时间内验证一个解是否正确的问题；而对于求出这个问题的解，我们是不清楚能不能在多项式时间内求出的。相反地，P问题就是能够确定有多项式时间算法能够给出问题解的问题。

所有的P问题同时也是NP问题，因为如果一个问题P问题，那么不用任何证书（certificate）就可以在多项式时间内解决它。因此 $P \subseteq NP$ ，但是是不是真子集，目前还是一个悬而未决的问题。

关于**certifier**有如下的说明：

- certifier的输入是一个问题的实例（i.e., instance），和一个证书 y （也就是一个可能解）
- certifier的输出是either yes or no
- 如果 x 是一个符合要求的实例，那么应该存在一个 y 输出1；vice versa for x 不符合问题的要求
- certifier应该能在多项式时间内运行出结果

因此，现在对于certifier给出如下的正式定义：

Def Given a decision problem A, a polynomial time verifier(certifier) V for A is an algorithm that does the following

- V's input is an instance x of A, and a certificate string y .
- $V(x, y) \in \{0, 1\}$, representing "reject(no)" and "accept(yes)", respectively.
- If x is a yes instance, there exists a y for which V outputs 1, i.e. $\exists y: V(x, y) = 1$.
- If x is a no instance, every y makes V output 0, i.e. $\forall y: V(x, y) = 0$.
- V runs in polynomial time.

NP is the set of all decision problems with polytime verifiers(certifiers).

其中有一些定义的细节：首先是decision problem，指的是这个问题的答案最终通常是'yes' or 'no'，比如说3-SAT问题中，是否有一组literal assignment使得clause的合取范式为true；然后certificate y ，可以认为是一组已知的特解，从而帮助判断任意一个实例instance是否符合题目要求；Verifier (certifier) 接受的是instance and certificate，最终输出的是0或1，代表no or yes。

因此可见如果想要说明一个decision problem是NP的，应该给出一个多项式时间的verifier (certifier)算法从而满足上述的性质；通常来说，需要首先说明证书有哪些，然后说明算法在输入一个任意instance和一个certificate是如何运行的。

举一个例子：3-SAT以及SAT问题是NP的。而一个instance就是一个具体的formula，而certifier就是随便一组解，然后判断的算法(certifier)就是判断每一个代入instance and formula的情境下，formula的clause是不是至少有一个true，而这是多项式算法的；但是求出一组满足的解，目前是没有一个算法能在多项式时间内给出的，而所有的NP问题都可以归约到3-SAT问题。如何更好的理解instance是什么意思？3-SAT问的是：是否对于任意一个满足规定的formula，判断是否有一组truth assignment能够满足formula为真；因此具体的一个formula，就是一个instance，而certificate就是随便的一组truth assignment。

从上述的视角，我们能够重新更准确地定义什么是P问题：首先，重新定义什么是polynomial time；然后重新定义P问题：

Def: Algorithm A runs in polynomial time if for every string s , $A(s)$ terminates in $\leq \text{poly}(|s|)$ "steps," where $\text{poly}(\cdot)$ is some polynomial function

Def: $P = \text{set of problems for which there exists a poly-time algorithm}$

例如判断一个数字是否是素数，这个问题就是decision problem，而这个问题就是P问题，因为存在Agrawal-Kayal-Saxena(2002)算法能够判断，而它是polynomial time的。

因此回答最开始的一个打趣的问题：NP代表的是Non-Polynomial吗？其实不是，而是**Nondeterministic Polynomial**

P are in NP

Def: EXP: Decision problems for which there exists an exponential-time algorithm

有定理：

$$P \subseteq NP, NP \subseteq EXP$$

对于第一个结论来说非常好说明：对于P中的任何问题，根据定义应该有一个算法 $A(s)$ ，则相当于构建了一个证书 $C(s, t)$ ，其中 t 是empty string。而对于第二个结论的证明不予以要求。

这个定理说明：任何的NP问题其实都能在指数复杂度的时间内解决！

那么P和NP最大的不同，其实就是在寻找解的时间复杂度上，因为给定一个instance and certificate，两种问题在判断输出是不是1这方面都是多项式时间。比如说3-SAT问题判断一个instance中的一个certificate解，只需要多项式时间就能够判断，但是如果对于任何一个含有 n 个literal的instance来说，找到一个满足要求的解需要 2^n 次尝试组合。因此，3-SAT是NP的，但不是P。

那么 $P = NP$ 吗？问题早在1971年就被提出来了，但是至今没有解决，而广泛认同的观点是：Probably no。

NP-Complete

首先，在多项式时间角度上，介绍两种问题之间的关系：归约与转化：

Def: Problem X polynomial (Cook) **reduces** to problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y

Def: Problem X polynomial (Karp) **transforms** to problem Y if given any instance x of X, we can construct an instance y of Y such that x is a yes instance of X **iff** y is a yes instance of Y.

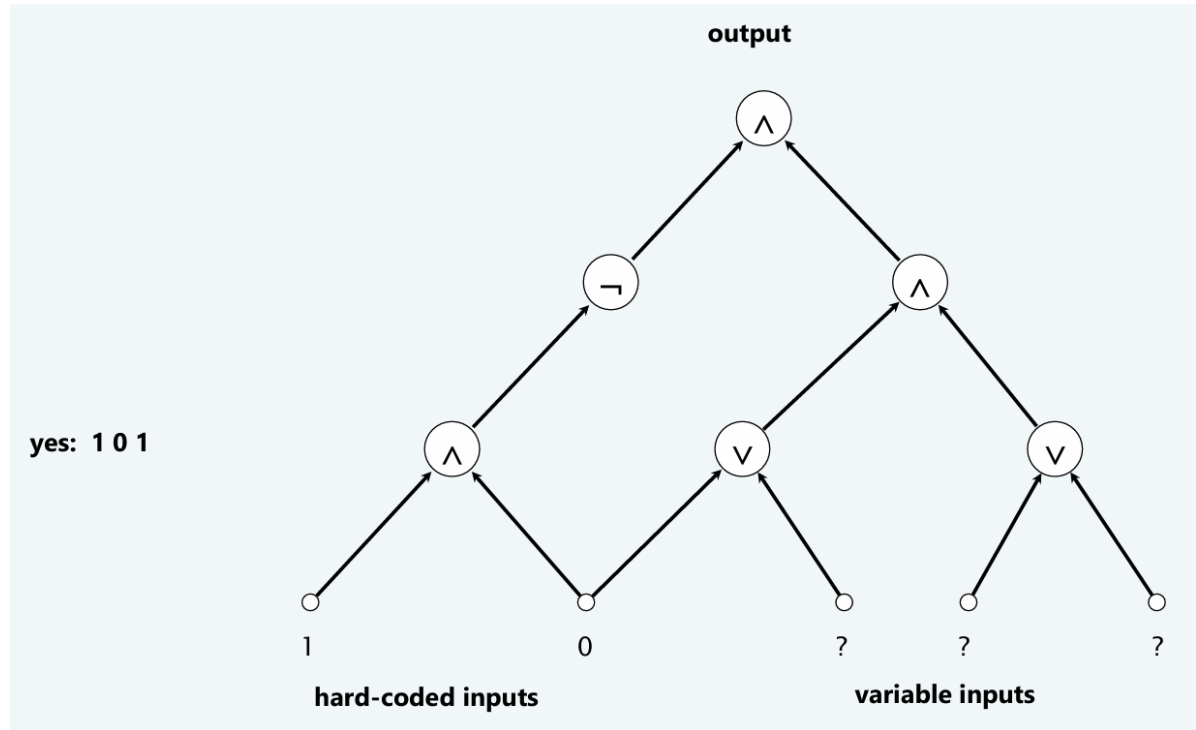
通过上述的定义可见：转化其实就是指调用一次Y算法的归约，而上一节的reduction更指的是这里的transformation。那么对于NP问题来说，这两个概念其实是同一个概念吗？这仍然是一个开放的问题。

那么NP-Complete定义如下：A problem $Y \in NP$ with the property that for every problem $X \in NP$, $X \leq_p Y$ 。这个定义看起来十分的怪：什么叫所有的NP问题都能归约到Y问题？真的有这样的问题吗？Are there any natural NP-complete problems? 答案，当然是有：Circuit-SAT! (**According to the Cook-Levin Theorem**)

Proposition: Suppose $Y \in NP - Complete$. Then, $Y \in P$ iff $P = NP$

注意!! 上面定义中: $Y \in NP$ 是相当必要的! 换言之, 如果上述的描述中去除了这个信息, 那么这句话就是错误的, 因为Y都可能不在NP问题的范畴内!

CIRCUIT-SAT: Given a combinational circuit built from AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?



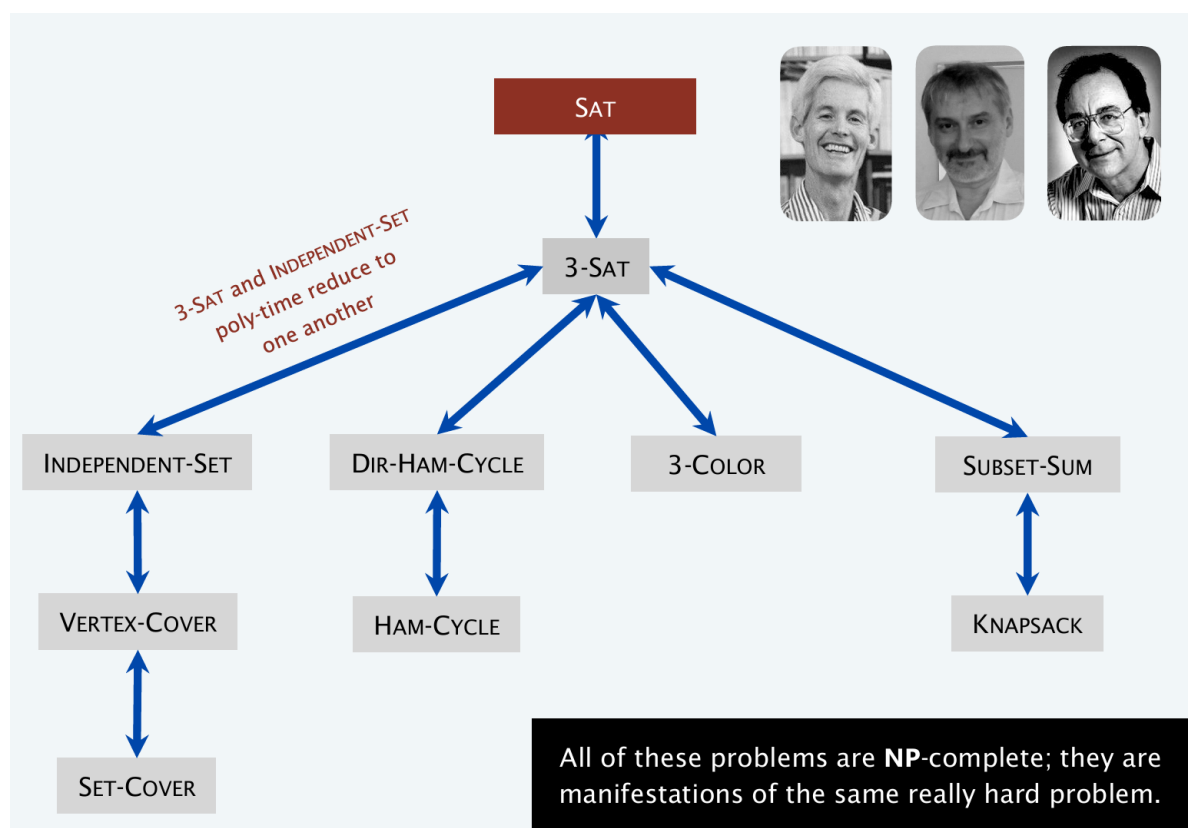
例如上图：有一个由逻辑门构成的circuit然后能不能找到variable inputs solution从而使得最终的输出是1。对于这个circuit-sat问题，很明显是NP的：考虑任何一个含有n个逻辑门的circuit，然后给出随便一组certificate解，那么只需要n次操作就能够求出最后的输出是1还是0。那么是否所有的NP问题都能够归约到Circuit-SAT问题，换言之，是否所有的NP问题A都能够在多项式时间内映射到一个combinational circuit K，从而使得：If A is true, then K is satisfiable; If A is false, then K is not satisfiable。关于这个结论的证明不予以要求。

同时，还有下面这个非常重要的定理：

$$3 - SAT \in NP - Complete$$

如果想要说明上述的结论，只需要证明： $3 - SAT \leq_p CIRCUIT - SAT$ 即可。同样，这个结论的证明不予以要求。

Conclusion



计算机科学家Karp + Cook + Levin的研究显示：上图中所有显示的问题，其实都是相互归约的，而且全部都是NP完全问题。

Knapsack是NPC问题？对于n个物品，W的总重量，DP的时间复杂度是 $O(nW)$ 。这看似是n的多项式时间算法，但是W的取值可能导致这个算法的复杂度不是多项式时间的！事实上，这个算法被认为是：*runs in pseudo-polynomial time*。

Summary of Introduced NPC

Independent-Set: 对于一个图来说，给定一个规定的正整数k，能不能找到顶点集合的一个含k个（或者更多）顶点的子集，使得子集中的每两个点之间都是不相连的？这个问题称为**Independent-Set**问题。

Vertex-Cover: 对于一个图来说，给定一个规定的正整数k，能不能找到顶点集合的一个含有k个（或者更少）顶点的子集，使得图中每一条边的两个顶点中至少有一个在子集里面。这个问题称为**Vertex-Cover**问题。

Set-Cover: 给定一个含有U个元素的集合，且给定S个U的子集，和一个正整数k，是否有 $\leq k$ 个子集的Union等于U？

SAT: 考虑一个合取范式，是否存在一组clause，i.e.，存在一组boolean variable使得这个CNF为真？这就称为SAT问题

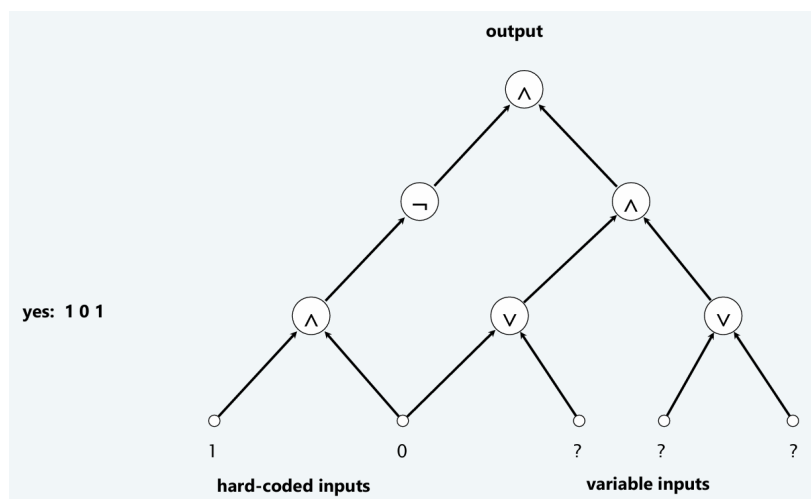
$$\Phi = C_1 \wedge C_2 \cdots \wedge C_n$$

C is called a clause: $C_j = x_1 \vee \overline{x_2} \vee \cdots \vee x_n$

3-SAT: 在SAT基础之上规定每一个clause的boolean variable数量为3

3-Color Problem: 有一个无向图G，能够找到一种给所有顶点的染色安排，使得任意两个相邻的顶点都没有相同的颜色？

CIRCUIT-SAT: Given a combinational circuit built from AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?



Classical Reduction

k-color to 3-color

5.3-Color: Given an undirected graph $G = (V, E)$, determine whether its vertices can be colored **within** 3 different colors such that no adjacent nodes have the same color?

The **yes**-instances of 3-Color is:

$$3\text{-Color} = \left\{ \langle G \rangle \mid \begin{array}{l} G = (V, E) \text{ is an undirected graph such that its} \\ \text{vertices can be colored **within** 3 different colors} \\ \text{such that no adjacent nodes have the same color.} \end{array} \right\}$$

6.k-Color: For fixed positive integer $k \geq 4$, given an undirected graph $G' = (V', E')$, determine whether its vertices can be colored **within** k different colors such that no adjacent nodes have the same color?

The **yes**-instances of k-Color is:

$$k\text{-Color} = \left\{ \langle G' \rangle \mid \begin{array}{l} G' = (V', E') \text{ is an undirected graph such that its} \\ \text{vertices can be colored **within** } k \text{ different colors} \\ \text{such that no adjacent nodes have the same color.} \end{array} \right\}$$

Construct a **correct** **direct** **polynomial-time** **many-one reduction** f_3 that maps instances of 3-Color to instances of k-Color.

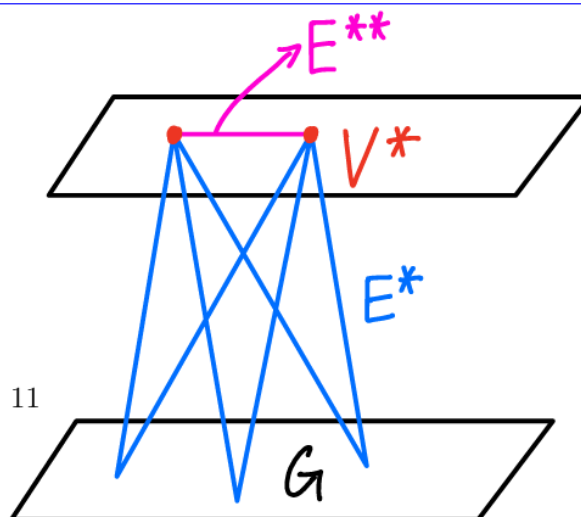
Reminder: You may **not** apply mathematical induction for k here. A **direct** reduction is required.

Solution:

Given an undirected graph $G = (V, E)$, we construct $f_3(\langle G \rangle) = \langle G' \rangle = (V', E')$ as follows:

1. Add $k - 3$ new vertices and define V^* as the set of these newly added vertices.
2. Let $V' = V \cup V^*$.
3. Define $E^* = \{\{u, v\} \mid u \in V, v \in V^*\}$.
4. Define $E^{**} = \{\{u, v\} \mid u, v \in V^*\}$.
5. Let $E' = E \cup E^* \cup E^{**}$.

Our reduction takes polynomial time because generating V^* , E^* and E^{**} can all be done in polynomial time.



Subset-Sum to Partition

Equivalent-Partition: Given an array $B = [b_1, b_2, \dots, b_n]$ of non-negative integers, determine whether there exists a subset $T \subseteq [n]$ such that $\sum_{i \in T} b_i = \sum_{j \in [n] \setminus T} b_j$ (i.e. determine whether there is a way to partition B into two disjoint subsets such that the sum of the elements in each subset is equivalent).

The **yes**-instances of **Equivalent-Partition** is:

$$\text{Equivalent-Partition} = \left\{ \langle b_1, \dots, b_n \rangle \mid \begin{array}{l} n \in \mathbb{Z}^+, b_1, \dots, b_n \in \mathbb{N} \text{ and there exists a} \\ \text{partition of the } b_i\text{'s to two parts whose sums} \\ \text{are equivalent, i.e. } \exists T \subseteq [n] : \sum_{i \in T} b_i = \sum_{j \in [n] \setminus T} b_j \end{array} \right\}$$

We choose **Subset-Sum** to reduce from. Recall that the **yes**-instance of **Subset-Sum** is:

$$\text{Subset-Sum} = \left\{ \langle a_1, a_2, \dots, a_m, k \rangle \mid \begin{array}{l} m \in \mathbb{Z}^+, a_1, \dots, a_m, k \in \mathbb{Z}^+ \text{ and there} \\ \text{exists a subset of the } a_i\text{'s that sum up} \\ \text{to } k, \text{ i.e. } \exists S \subseteq [m] : \sum_{i \in S} a_i = k. \end{array} \right\}$$

Solution:

First, still define $X = \sum_{i \in [m]} a_i$ and let $n = m + 2$. To avoid having k and $X - k$ in the same side of the partition, we add 1 (or any positive number) to both of them. Hence, we define our reduction as

$$\langle b_1, \dots, b_n \rangle = f(\langle a_1, a_2, \dots, a_m, k \rangle) \triangleq \langle a_1, a_2, \dots, a_m, k + 1, X - k + 1 \rangle$$

Our reduction takes polynomial time because all computation including computing the sum of all elements, adding 1 and subtracting k can be done in polynomial time.

(d) Prove the correctness of your reduction by showing:

- i. (1') x is a **yes-instance** of **Subset-Sum** $\Rightarrow f(x)$ is a **yes-instance** of **Equivalent-Partition**.

Solution:

Let $\langle a_1, a_2, \dots, a_m, k \rangle$ be a **yes-instance** of **Subset-Sum** and let $S \subseteq [m]$ be the set of indices such that $\sum_{i \in S} a_i = k$. Then we may deduce that $\sum_{i \in [n] \setminus S} a_i = X - k$. Hence:

$$X - k + 1 + \sum_{i \in S} a_i = (X - k + 1) + k = (k + 1) + (X - k) = k + \sum_{i \in [n] \setminus S} a_i$$

which indicates that $f(\langle a_1, a_2, \dots, a_m, k \rangle) = \langle a_1, a_2, \dots, a_m, k + 1, X - k + 1 \rangle$ is a **yes-instance** of **Equivalent-Partition**.

- ii. (2') x is a **yes-instance** of **Subset-Sum** $\Leftarrow f(x)$ is a **yes-instance** of **Equivalent-Partition**.

Solution:

Let $\langle b_1, \dots, b_n \rangle = f(\langle a_1, a_2, \dots, a_m, k \rangle) = \langle a_1, a_2, \dots, a_m, k + 1, X - k + 1 \rangle$ be a **yes-instance** of **Equivalent-Partition** and let $T \subseteq [n]$ be the set of indices such that $\sum_{i \in T} b_i = \sum_{j \in [n] \setminus T} b_j$. Note that $\sum_{i \in [n]} b_i = (k + 1) + (X - k + 1) + \sum_{i \in [m]} a_i = (k + 1) + (X - k + 1) + X = 2X + 2$, we may deduce that $\sum_{i \in T} b_i = \frac{1}{2}(2X + 2) = X + 1$. Since all $b_i \geq 0, \forall i \in [n]$ while $(k + 1) + (X - k + 1) > X + 1$, we may deduce that $k + 1$ and $X - k + 1$ cannot be on the same side of the partition. Thus, we may conclude that $X - k + 1$ is paired with a subset of element from $a_1, a_2, \dots, a_m$ such that their overall sum is $X + 1$, which indicates that $\exists S \subseteq [m]$ such that $\sum_{i \in S} a_i = (X + 1) - (X - k + 1) = k$. Hence, we claim that $\langle a_1, a_2, \dots, a_m, k \rangle$ is a **yes-instance** of **Subset-Sum**.

Partition to Carpenter

Carpenter: Given an array $L = [l_1, l_2, \dots, l_n]$ of non-negative integers, determine whether there exists a sequence $D = [d_1, d_2, \dots, d_n]$ where $d_i \in \{\pm 1\}$ such that $\max_{j=0}^n \{\sum_{i=1}^j d_i l_i\} - \min_{j=0}^n \{\sum_{i=1}^j d_i l_i\} \leq k$.

Intuitively speaking, give a sequence of rigid rods of various integral lengths connected end-to-end by hinges, can it be folded so that its overall length is at most k ? Correspondingly, l_i indicates the length of i -th rigid rod and d_i indicates the direction of i -th rigid rod (fold left or right). The max and min indicate the leftmost and rightmost positions.

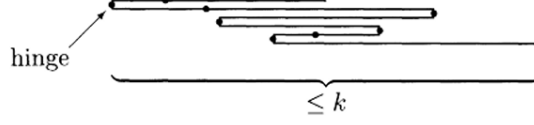


Figure 1: Example of Hinge

The *yes*-instances of Carpenter is:

$$\text{Carpenter} = \left\{ \langle l_1, \dots, l_n, k \rangle \mid \begin{array}{l} n \in \mathbb{Z}^+, l_1, \dots, l_n, k \in \mathbb{N}, \exists D = [d_1, d_2, \dots, d_n], d_i \in \{\pm 1\} \\ \text{i.e. } D \in \{1, -1\}^n \text{ s.t. } \max_{j=0}^n \left\{ \sum_{i=1}^j d_i l_i \right\} - \min_{j=0}^n \left\{ \sum_{i=1}^j d_i l_i \right\} \leq k. \end{array} \right\}$$

We choose Equivalent-Partition to reduce from. Here is the *yes*-instance of it:

$$\text{Equivalent-Partition} = \left\{ \langle b_1, \dots, b_m \rangle \mid \begin{array}{l} n \in \mathbb{Z}^+, b_1, \dots, b_m \in \mathbb{N} \text{ and there exists a} \\ \text{partition of the } b_i\text{'s to two parts whose sums} \\ \text{are equivalent, i.e. } \exists T \subseteq [m] : \sum_{i \in T} b_i = \sum_{j \in [m] \setminus T} b_j \end{array} \right\}$$

- i. (2') Your reduction in the format of (b).ii. (Show both n and L .)

Hint: Try to modify the above reductions into the correct one.

Solution: $n = m + 4$, $S = \sum_{i=1}^m b_i$. (Any $S \geq \sum_{i=1}^m b_i$ suffices. If $S \not\equiv 0 \pmod{2}$, then the instances of **Equivalent-Partition** must be wrong-instances.)

$$\langle l_1, \dots, l_n, k \rangle = f(\langle b_1, \dots, b_m \rangle) \triangleq \langle S, \frac{S}{2}, b_1, \dots, b_m, \frac{S}{2}, S, S \rangle$$

- ii. (2') x is a yes-instance of **Equivalent-Partition** $\Rightarrow f(x)$ is a yes-instance of **Carpenter**.

Solution: If $x = \langle b_1, \dots, b_m \rangle$ is a yes-instance of **Equivalent-Partition**, and $T \in [m]$ is the subset such that $\sum_{i \in T} b_i = \sum_{j \in [m] \setminus T} b_j$ i.e. $\sum_{i=1}^m (-1)^{[i \in T]} b_i = 0$. Let $d_{i+2} = (-1)^{[i \in T]}$ and $d_1 = d_{m+4} = 1, d_2 = d_{m+3} = -1$. Then

$$\forall j \in [m], \sum_{i=1}^{j+2} d_i l_i = \frac{S}{2} + \sum_{i=1}^j (-1)^{[i \in T]} b_i, \max_j \left(\sum_{i=1}^j (-1)^{[i \in T]} b_i \right) \leq \sum_{i \in T} b_i = \frac{S}{2},$$

which indicates $\max_j \{ \sum_{i=1}^j d_i l_i \} = \max(S, \frac{S}{2}, \frac{S}{2}) = S$. Similarly, $\min \{ \sum_{i=1}^j d_i l_i \} = \max(0, \frac{S}{2}, 0) = 0$. Therefore, $\max_j \{ \sum_{i=1}^j d_i l_i \} - \min_j \{ \sum_{i=1}^j d_i l_i \} \leq k$, which indicates that $f(\langle b_1, \dots, b_m \rangle) = \langle S, \frac{S}{2}, b_1, \dots, b_m, \frac{S}{2}, S, S \rangle$ is a yes-instance of **Carpenter**.

- ii. (2') $f(x)$ is a yes-instance of **Carpenter** $\Rightarrow x$ is a yes-instance of **Equivalent-Partition**.

Solution: Let $\langle l_1, \dots, l_n, k \rangle = f(\langle b_1, \dots, b_m \rangle) = \langle S, \frac{S}{2}, b_1, \dots, b_m, \frac{S}{2}, S, S \rangle$ be a yes-instance of **Carpenter** and let $D = [d_1, \dots, d_n]$ be the certificate.

Denote $c_j = \sum_{i=1}^j d_i l_i$. Since $[d_1, \dots, d_n]$ and $[-d_1, \dots, -d_n]$ both can be the valid certificate, we assume $d_1 = 1$. Then $d_2 = -1$ since $c_2 - c_0 = S + d_2 \frac{S}{2} - 0 \in [0, S]$. Moreover, $\forall i \in [n], c_i \in [0, S]$. And since $|c_n - c_{n-1}| = S$, $c_{n-2} = \frac{S}{2}$ no matter $c_n = S$ or $c_n = 0$, which indicates $c_{n-2} - c_2 = 0$ i.e. $\sum_{i=1}^m d_{i+2} b_i = 0$. Denote $T = \{i | d_i = 1\}$ ($[m] \setminus T = \{i | d_i = -1\}$). Then

$$\sum_{i \in T} b_i - \sum_{j \in [m] \setminus T} b_j = 0 \Rightarrow \sum_{i \in T} b_i = \sum_{j \in [m] \setminus T} b_j,$$

which shows that $\langle b_1, \dots, b_m \rangle$ is a yes-instance of **Equivalent-Partition**.

Appendix

How to prove one problem is NPC

(0 points) Tutorial on how to prove that a particular problem is in NP-Complete

To prove problem A is in NP-Complete, your answer should include:

1. Prove that problem A is in NP by showing: What your **polynomial-size** certificate is and what your **polynomial-time** certifier is.
2. Choose a problem B in NP-Complete to reduce from.
3. Construct your **polynomial-time many-one reduction** f that maps instances of problem B to instances of problem A .
(polynomial-time many-one reduction = polynomial transformation = Karp reduction, see presenter notes of page 7 & 61 in lecture slides (.pptx file) for more details.)
4. Prove the correctness of your reduction (i.e. Prove that your reduction f do map *yes*-instance of problem B to *yes*-instance of problem A and map *no*-instance of problem B to *no*-instance of problem A) by showing:
 - (a) x is a *yes*-instance of problem $B \Rightarrow f(x)$ is a *yes*-instance of problem A .
 - (b) x is a *yes*-instance of problem $B \Leftarrow f(x)$ is a *yes*-instance of problem A .

Proof Example: Prove that the decision version of Set-Cover is in NP-Complete.

Recall that the *yes*-instances of the decision version of Set-Cover is:

$$\text{Set-Cover} = \left\{ \langle U, S_1, \dots, S_n, k \rangle \mid \begin{array}{l} n \in \mathbb{Z}^+, S_1, \dots, S_n \subseteq U \text{ and there exist } k \text{ sets } S_{i_1}, \dots, \\ S_{i_k} \text{ that cover all of } U, \text{ i.e., } S_{i_1} \cup S_{i_2} \cup \dots \cup S_{i_k} = U \end{array} \right\}$$

To prove that the decision version of Set-Cover is in NP-Complete, we follow these steps:

1. **Membership in NP:**
 - (a) A set of indices $\{i_1, \dots, i_k\} \subseteq \{1, 2, \dots, n\}$, whose size is polynomial of input size .
 - (b) Check whether $S_{i_1} \cup S_{i_2} \cup \dots \cup S_{i_k} = U$, whose run-time is polynomial of input size.
2. **Reduction from Vertex-Cover:**
 - (a) We choose the decision version of Vertex-Cover as the problem to reduce from.

$$\text{Vertex-Cover} = \left\{ \langle G, k' \rangle \mid \begin{array}{l} G \text{ is an undirected graph and there exists a set of } \\ k' \text{ vertices that touches all edges in } G. \end{array} \right\}$$

- (b) Given an undirected graph $G = (V, E)$ and a positive integer $k' \in \mathbb{Z}^+$, we construct the reduction $f(\langle G, k' \rangle) = \langle U, S_1, \dots, S_n, k \rangle$ as follows: $U = E$. $n = |V|$ and $k = k'$. For each $i \in \{1, \dots, n\}$, $S_i = \{e \in E \mid e = (v_i, u) \text{ for some } u \in V \setminus \{v_i\}\}$.
- (c) We prove the correctness of the reduction:
 - “ $\Rightarrow$ ”: If $\langle G, k' \rangle$ is a *yes*-instance of Vertex-Cover, then there exists a set $V^* = \{v_{i_1}, \dots, v_{i_{k'}}\}$ of vertices that covers all edges. By construction, $\{S_{i_1}, \dots, S_{i_{k'}}\}$ covers all edges in U , so $\langle U, S_1, \dots, S_n, k \rangle$ is a *yes*-instance of Set-Cover.
 - “ $\Leftarrow$ ”: Conversely, if $\langle U, S_1, \dots, S_n, k \rangle$ is a *yes*-instance of Set-Cover, then there exists a set $\{S_{i_1}, \dots, S_{i_k}\}$ of sets that covers all edges in U . By construction, the corresponding set of vertices $\{v_{i_1}, \dots, v_{i_k}\}$ covers all edges in G , so $\langle G, k' \rangle$ is a *yes*-instance of Vertex-Cover.

Hence, the decision version of Set-Cover is in NP-Complete.

How to prove two P problems can reduce to each other

In fact, any two problems in P can reduce to each other in polynomial time, indicating they share the same “hardness.” To illustrate, consider decision problems A and B in $\mathcal{P}$. Given an instance x of A :

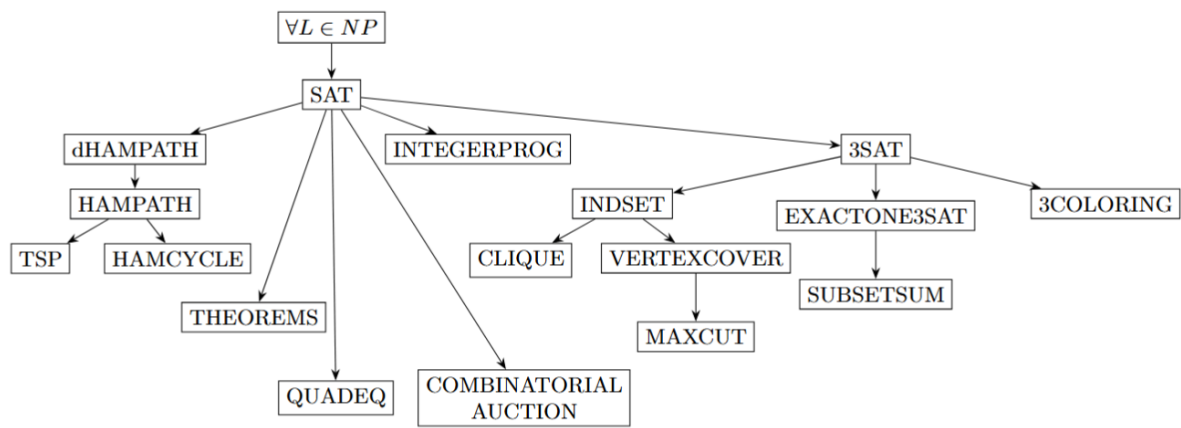
1. Prepare polynomial-time copies of known *yes*- and *no*-instances of B .
2. Determine if x is a *yes*-instance of A in polynomial time.
3. Return the corresponding *yes*- or *no*-instance of B .

For example, let A be the minimum spanning tree cost problem and B be the shortest path cost problem in undirected weighted graphs. Given an instance G of A :

1. Prepare graphs G_1 and G_2 with known shortest path costs relative to c' .
2. Find G ’s minimum spanning tree cost using Kruskal’s algorithm and compare it with c .
3. Return G_1 if the cost is no more than c , otherwise return G_2 .

NPC map

A brief map of common NP-Complete problems:

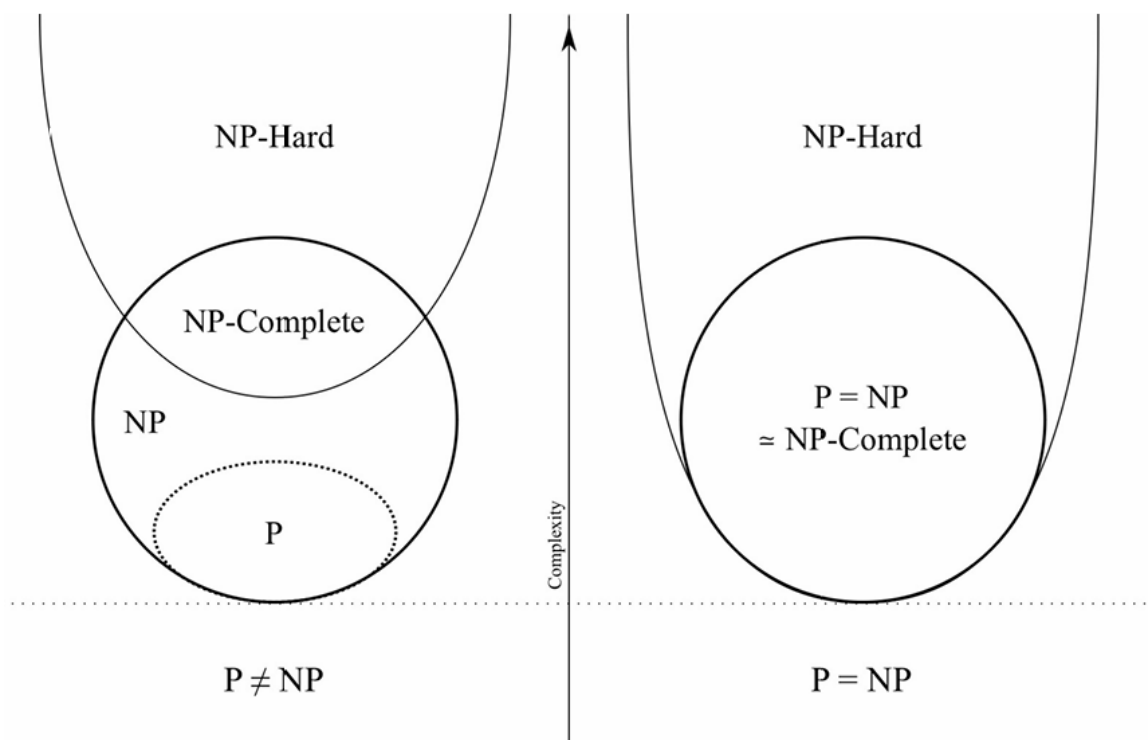


Reference: *Computational Complexity: A Modern Approach* by Sanjeev Arora and Boaz Barak.

Common solvable and unsolvable problem

yes	probably no
shortest path	longest path
min cut	max cut
2-satisfiability	3-satisfiability
planar 4-colorability	planar 3-colorability
bipartite vertex cover	vertex cover
matching	3d-matching
primality testing	factoring
linear programming	integer linear programming

P v.s. NP



$P \neq NP$ if and only if $P \cap NP - Complete = \emptyset$

$P = NP$ if and only if $NP = NP - Complete$

Warning

在CS101考试和作业中，如果没有说 $P =$ OR $\neq NP$ 的大前提，那么在判断一个选项的时候就需要考虑两种情况；同时，NP-Hard的情况也需要予以考虑，虽然课堂内容没有cover到。NP-Hard与其他三种问题的关系如上图所示。